

密 级： 公开

加密论文编号：

论文题目： 面向云原生应用权限提升漏洞的智能化
防控技术研究

学 号： 论文作者学号

研 究 生： 本论文作者

专 业 名 称： 计算机科学与技术

2026 年 5 月 1 日

面向云原生应用权限提升漏洞的智能化防控技术研究

**Research on Dynamic Defense and Monitoring
Technologies for Privilege Escalation
Vulnerabilities in Cloud-Native Applications**

研究生：本论文作者

指导教师：本论文导师

北京科技大学 计算机与通信工程学院

北京 100083，中国

Doctor Degree Candidate: 本论文作者

Supervisor: 本论文导师

School of Computer and Communication Engineering

University of Science and Technology Beijing

30 Xueyuan Road, Haidian District

Beijing 100083, P.R.CHINA

中图分类号:

学校代码:

10008

U D C:

密 级:

公开

北京科技大学硕士学位论文

论文题目: 面向云原生应用权限提升漏洞的智能化防控技术研究

研究生: _____ 本论文作者

指 导 教 师: 本论文导师 单位: 北京科技大学 职称: 教授

指 导 小 组 成 员: _____ 单位: _____ 职称: _____

_____ 单位: _____ 职称: _____

论文提交日期: 2026 年 5 月 1 日

学位授予单位: 北 京 科 技 大 学

摘 要

Kubernetes 生态中的第三方云原生应用存在可导致权限提升的高危漏洞。在漏洞披露至补丁落地的窗口期内，Kubernetes 集群面临较高的被攻击风险。现有针对该类漏洞的防控方法主要依赖静态核查与配置审计，难以围绕具体漏洞快速生成可落地的临时防控策略，相关安全工具的规则编写与策略制定也高度依赖专家经验。

针对 Kubernetes 权限提升漏洞问题，本文提出一种基于多智能体安全策略管理的智能化漏洞防控技术，基于多种安全工具的策略化代码生成、评估与部署实现对权限提升相关漏洞的动态防护与监控，开发了对应的原型系统。本文的主要工作如下。

(1) 多智能体安全策略管理的漏洞防控技术：首先，本文收集了近年来 Kubernetes 第三方云原生应用中的权限提升漏洞样本并设计了漏洞成因分类框架，同时基于开源代码仓库中等多源信息构建了云原生权限提升安全知识库。然后，构建由安全知识、代码生成与安全评审等六个智能体组成的工作流，进行针对特定权限提升漏洞防护与监控策略代码的自动生成、迭代优化与质量评估。最后，针对安全策略代码的实际应用需求，设计了覆盖有效性、非干扰性与可部署性的三维质量评价指标。

(2) 开发原型系统：设计并实现原型系统 KPEAgent，集成知识库管理、策略生成工作流、专家评审排序与策略部署验证等功能模块，形成面向云原生权限提升漏洞的策略化防控流程。

(3) 实验验证与案例分析：在 55 个漏洞样本上，基于 5 种大语言模型开展消融实验与模型对比实验。结果表明，完整工作流在三个评价维度上均取得最优表现，其中反馈优化对策略质量的影响最为显著，移除该模块后策略有效性下降 79%。与四种静态扫描工具相比，本文方法能够围绕具体漏洞生成更具针对性的临时防控方案。进一步地，本文选取 7 个覆盖不同成因类型的典型 CVE，在隔离集群中完成漏洞复现与策略部署验证，从三个维度进一步验证了所提方法的实际效果。

关键词：云原生安全；权限提升漏洞；策略代码生成；补丁窗口期防控；多智能体协同

Research on Dynamic Defense and Monitoring Technologies for Privilege Escalation Vulnerabilities in Cloud-Native Applications

ABSTRACT

Third-party open-source applications in the Kubernetes ecosystem commonly contain high-risk vulnerabilities that may lead to privilege escalation. During the window between vulnerability disclosure and patch deployment, Kubernetes clusters remain exposed to a considerable risk of attack. Existing defense methods for such vulnerabilities mainly rely on static inspection and configuration auditing, and therefore struggle to rapidly produce actionable interim mitigation strategies for specific vulnerabilities. In addition, the formulation of security rules and mitigation strategies still depends heavily on expert experience.

To address the problem of privilege escalation vulnerabilities in Kubernetes, this thesis proposes an intelligent vulnerability prevention and control technique based on multi-agent security strategy management. By generating, evaluating, and deploying strategy code for multiple security tools, the proposed method enables dynamic protection and monitoring for privilege-escalation-related vulnerabilities, and a corresponding prototype system is developed. The main work of this thesis is as follows.

(1) Vulnerability prevention and control technique based on multi-agent security strategy management: First, this thesis collects privilege escalation vulnerability samples from third-party open-source applications in the Kubernetes ecosystem in recent years, and designs a root-cause classification framework for such vulnerabilities. Meanwhile, a cloud-native security knowledge base for privilege escalation is constructed based on multi-source information, including open-source code repositories. Then, a workflow consisting of six agents for security knowledge, code generation, and security review is built to support the automatic generation, iterative optimization, and quality evaluation of protection and monitoring strategy code for specific privilege escalation vulnerabilities. Finally, to meet the practical requirements of security strategy code, a three-dimensional quality evaluation framework

is designed, covering effectiveness, non-interference, and deployability.

(2) Prototype system development: A prototype system, named KPEAgent, is designed and implemented. It integrates functional modules for knowledge base management, strategy generation workflow, expert review and ranking, and strategy deployment verification, thereby forming a strategy-based prevention and control process for cloud-native privilege escalation vulnerabilities.

(3) Experimental validation and case studies: Ablation studies and model comparison experiments are conducted on 55 vulnerability samples using five large language models. The results show that the complete workflow achieves the best performance across all three evaluation dimensions. Among all components, feedback optimization has the most significant impact on strategy quality: removing this module reduces strategy effectiveness by 79%. Compared with four static scanning tools, the proposed method can generate more targeted interim mitigation strategies for specific vulnerabilities. Furthermore, seven representative CVEs covering different root-cause categories are selected for vulnerability reproduction and strategy deployment verification in an isolated cluster, which further demonstrates the practical effectiveness of the proposed method from the three evaluation dimensions.

Key Words: Cloud-native security; Privilege escalation vulnerabilities; Strategy code generation; Patch-window defense and control; Multi-agent collaboration

目 录

摘要	I
ABSTRACT	III
符号清单与术语表	XI
1 引言	1
1.1 背景与意义	1
1.2 研究内容与成果	2
1.3 论文组织结构	3
2 背景介绍	5
2.1 相关概念与技术	5
2.1.1 容器	5
2.1.2 容器集群系统 Kubernetes	6
2.1.3 Kubernetes 权限模型	7
2.1.4 Kubernetes 权限提升漏洞	7
2.1.5 云原生安全技术	8
2.2 国内外研究现状	9
2.2.1 系统权限安全研究现状	9
2.2.2 Kubernetes 安全研究现状	11
2.2.3 前期工作	12
2.3 小结	14
3 面向云原生权限提升漏洞的智能化防控技术	15
3.1 研究动机	15
3.2 方法框架	17
3.2.1 基本概念定义	18
3.2.2 云原生权限提升安全知识库	19
3.2.3 基于多智能体协同的策略生成与优化	24
3.3 方法示例	30
3.4 小结	34
4 面向云原生权限提升漏洞的智能化防控系统设计与实现	35
4.1 需求分析	35
4.1.1 用户角色与用例分析	35
4.1.2 功能需求	36
4.2 总体设计	36
4.2.1 系统架构设计	36

4.2.2	数据模型设计	37
4.2.3	数据流转与处置流程设计	38
4.2.4	交互时序设计	38
4.3	详细设计与实现	39
4.3.1	CVE 知识库管理	39
4.3.2	策略生成与优化	39
4.3.3	专家评审与排序	40
4.3.4	策略部署管理	41
4.4	小结	41
5	实验研究	53
5.1	研究问题	53
5.2	实验对象	53
5.3	实验设计	54
5.3.1	度量指标	54
5.3.2	基线方法	55
5.3.3	实验过程	55
5.4	结果分析与讨论	56
5.4.1	消融实验与 workflow 模块贡献	56
5.4.2	生成模型能力评估	57
5.4.3	评审方法可信度评估	59
5.4.4	静态扫描工具的处置价值	62
5.4.5	资源消耗与效率评估	63
5.5	小结	66
6	案例分析	67
6.1	案例分析实验设计	67
6.1.1	CVE 案例选择标准	67
6.1.2	实验环境与复现流程	67
6.2	案例一：CVE-2022-24768 内部权限问题	69
6.2.1	漏洞详解与复现	69
6.2.2	策略部署	70
6.2.3	三因素分析	71
6.3	案例二：CVE-2025-2241 (Hive ClusterProvision 敏感信息泄露)	72
6.3.1	漏洞详解与复现	72
6.3.2	策略部署	74
6.3.3	三因素分析	74

6.4 案例三：CVE-2023-30512（CubeFS CSI 过度权限配置）	75
6.4.1 漏洞详解与复现	75
6.4.2 策略部署	77
6.4.3 三因素分析	78
6.5 案例四：CVE-2020-4062 Conjur OSS 缺乏网络隔离策略	78
6.5.1 漏洞详解与复现	78
6.5.2 策略部署	80
6.5.3 三因素分析	80
6.6 案例五：CVE-2022-24877（Flux kustomize-controller 路径穿 越）	81
6.6.1 漏洞详解与复现	81
6.6.2 策略部署	82
6.6.3 三因素分析	82
6.7 案例六：CVE-2022-29171（Sourcegraph gitserver 远程代码执 行）	83
6.7.1 漏洞详解与复现	83
6.7.2 策略部署	84
6.7.3 三因素分析	84
6.8 案例七：CVE-2023-22482（Argo CD OIDC aud 未校验导致越权 访问）	85
6.8.1 漏洞详解与复现	85
6.8.2 策略部署	86
6.8.3 三因素分析	86
6.9 小结	88
7 结论与展望	89
7.1 总结	89
7.2 展望	90
附录 A 云原生权限提升漏洞分类与 CWE 对应关系	99
A.1 KubeGuard 权限提升检测规则汇总	99
附录 B CVE202224768 复现步骤与策略清单	101
B.1 漏洞复现详细步骤	101
B.2 策略代码清单	102
B.2.1 Kubernetes RBAC：收敛 AppProject 的写权限	102
B.2.2 Argo CD RBAC：默认只读 + 高风险能力隔离	103
B.2.3 准入控制：Gatekeeper（推荐）	103
B.2.4 准入控制：Kyverno（替代方案）	104

附录 C CVE-2025-2241 复现步骤与策略清单	107
C.1 漏洞复现详细步骤	107
C.2 策略代码清单	108
C.2.1 1) 立刻收敛 ClusterProvision 的读取权限 (RBAC)	108
C.2.2 2) 对读取行为启用审计并联动告警	108
C.2.3 3) 缩短暴露窗口: 供应完成后清理 ClusterProvision	109
C.2.4 4) vCenter 凭据轮换与强化 (止血)	109
C.2.5 5) “敏感字段守护”检测 (Gatekeeper/Kyverno, 建议审计模式)	110
附录 D CVE-2023-30512 复现步骤与策略清单	113
D.1 漏洞复现详细步骤	113
D.2 策略代码清单	113
D.2.1 1) RBAC 最小化: 移除 ClusterRole 中的 secrets 权限	114
D.2.2 2) ServiceAccount 硬化 (按需评估)	114
D.2.3 3) 准入防回归 (OPA Gatekeeper): 禁止创建包含 secrets 读 取动词的 ClusterRole	114
D.2.4 4) 审计与检测: 对 secrets 枚举行为进行可观测化	115
D.2.5 5) 网络收敛 (可选): 限制 CSI Pod 出站访问面	115
附录 E CVE-2020-4062 复现步骤与策略清单	117
E.1 漏洞复现详细步骤	117
E.2 策略代码清单	118
E.2.1 1) 缓解措施: NetworkPolicy, 默认拒绝并仅放通 Conjur Server 访问 Postgres 5432	118
E.2.2 2) 命名空间隔离与 RBAC 收敛	118
附录 F CVE-2022-24877 复现步骤与策略清单	121
F.1 漏洞复现详细步骤	121
F.2 策略代码清单 (优先级方案)	122
F.2.1 1) 工作区缓解: 在 CI/CD 中校验 kustomization.yaml 路径 策略	122
F.2.2 2) 多租户变更治理与运行时硬化 (纵深防御)	123
附录 G CVE-2022-29171 复现步骤与策略清单	125
G.1 漏洞复现详细步骤	125
G.2 策略清单	126
G.2.1 1) 快速缓解: 禁用或移除 Gitolite code host (若业务允许) ..	126
G.2.2 2) 网络隔离: 限制 gitserver 的可达面 (默认拒绝并按需 放通)	126

G.2.3 3) 权限分离与暴露面收敛：限制管理面与观测面的高权限 ...	127
附录 H CVE-2023-22482 复现步骤与策略清单	129
H.1 漏洞复现详细步骤	129
H.2 策略清单	130
H.2.1 1) 身份提供方治理：独立客户端与唯一受众	130
H.2.2 2) 入口补偿控制：前置强制校验 aud	131
H.2.3 3) 权限收敛：RBAC 最小化与项目边界约束	131
致谢	133
作者简历及在学研究成果	135
独创性说明	137
关于论文使用授权的说明	139
学位论文数据集	141

符号清单与术语表

本文中频繁使用的主要符号、核心术语与常用缩写汇总如下。为避免与正文重复，本页重点列出第三章方法定义与后续评价分析中反复出现的记号与表述。

符号清单

基本对象与集合

符号	含义说明
$\mathcal{V}$	漏洞实例空间，表示全部待分析漏洞构成的集合。
$v = \langle id, d, m \rangle$	单个漏洞实例，由编号、描述与元数据组成。
id	漏洞标识符，如 CVE 编号。
d	漏洞描述文本，用于刻画漏洞事实。
m	漏洞元数据，如严重性、受影响组件与版本范围等。
$\mathcal{C}$	安全上下文空间，表示与漏洞相关的证据集合空间。
c	安全上下文，即带来源标识的有限证据集合。
t_i	第 i 条证据内容，如代码片段、公告文本或文档片段。
src_i	第 i 条证据来源，如文档路径、链接或代码位置。
Π	候选策略全集，表示所有候选监控策略与防护策略的总集合。
$\pi = \langle type, method, action \rangle$	单个安全策略，由策略类型、控制机理与执行表达组成。
$type$	策略类型，取值为监控型或防护型。
$method$	控制机理序列，用于表示策略采用的控制手段。
$action$	执行表达序列，对应可部署的规则、配置或命令。
Π_v	漏洞 v 的候选策略集合，由生成阶段针对漏洞 v 产出。
$\mathcal{T} = \{\text{mon}, \text{prot}\}$	策略类型集合，分别表示监控型与防护型。
τ	类型变量，表示某一给定策略类型。
Π_v^τ	同类型候选子集，表示漏洞 v 在类型 τ 下的候选集合。

评价与聚合记号

符号	含义说明
$\Delta = \{E, I, D\}$	评价维度集合，分别对应有效性、无干扰性与可部署性。
$q(\pi)$	三维质量向量，表示策略 π 在三个维度上的质量。
$E(\pi)$	有效性，用于衡量对主要攻击路径与关键风险面的覆盖程度。
$I(\pi)$	无干扰性，衡量对正常业务的扰动程度及误报风险。
$D(\pi)$	可部署性，衡量策略在现有环境中的落地难度。
$\mathcal{H}$	评价主体集合，表示多个评审主体构成的集合。
$r_{\delta}^{(h)}$	秩函数，表示评价主体 h 在维度 δ 上给出的排序位置。
$s_{\delta}^{(h)}(\pi)$	归一化秩得分，用于将排序位置归一化到 $[0, 1]$ 区间。
$\hat{S}_{\delta}(\pi)$	共识得分，表示多个评价主体在维度 δ 上的聚合结果。
$U_{\delta}(\pi)$	分歧度量，用于刻画不同评审结果的一致性程度。
$Q(\pi)$	总体质量函数，表示综合三维共识得分后的最终质量。
$\alpha_E, \alpha_I, \alpha_D$	维度权重，分别对应有效性、无干扰性与可部署性的权重。

术语表

为统一全文表述，本文使用频率较高的核心术语与常用缩写说明如下。

核心术语

术语	英文/缩写	说明
云原生	Cloud-native	本文研究对象所处的应用与基础设施环境。
Kubernetes	K8s	本文默认指 Kubernetes 容器编排平台，K8s 为其常用缩写。
权限提升	Privilege Escalation	攻击者突破原有权限边界并获取更高权限的过程。
补丁窗口期	Patch-gap window	漏洞披露至补丁可用或补丁完成部署之间的时间段。
临时防控	Interim mitigation	在补丁窗口期内用于降低风险的临时控制措施。
安全策略	Security strategy	监控策略与防护策略的上位概念。
监控策略	Monitoring strategy	面向异常行为感知、告警与取证的策略。
防护策略	Protection strategy	面向攻击阻断、权限收敛与攻击面控制的策略。
安全上下文	Security context	与具体漏洞相关的证据集合及其来源标识。
安全知识库	Security knowledge base	存放多源证据、向量索引及元数据的知识支撑体系。
多智能体协同	Multi-agent collaboration	由多个功能角色协同完成分析、生成与评审的工作方式。
检索增强	RAG	Retrieval-Augmented Generation
思维链推理	CoT	Chain-of-Thought，思维链。
反馈优化	Feedback	基于评审结果对候选策略进行修订与收敛的过程。

常用缩写

缩写	英文全称	中文说明
CVE	Common Vulnerabilities and Exposures	通用漏洞与暴露。
CWE	Common Weakness Enumeration	通用弱点枚举。
CVSS	Common Vulnerability Scoring System	通用漏洞评分系统。
NVD	National Vulnerability Database	国家漏洞数据库。
CNCF	Cloud Native Computing Foundation	云原生计算基金会。
K8s	Kubernetes	Kubernetes 的常用缩写。
RBAC	Role-Based Access Control	基于角色的访问控制。
LLM	Large Language Model	大语言模型。

1 引言

本章介绍课题的研究背景与意义、研究内容与成果，以及论文的组织结构。

1.1 背景与意义

服务化发展推动了现代软件系统的大规模部署^[1]。服务计算场景下, 软件体系结构由单体应用转向面向服务架构 (SOA), 并发展至微服务架构^[2]。微服务系统由多个可独立开发、部署和演化的微服务单元组成, 单物理机部署难以满足微服务系统弹性扩缩、持续交付和高可用运行等要求。云原生软件范式将微服务和容器、云计算结合, 实现了大规模微服务系统的多节点集群部署, 保障软件构建与运行中可用性、安全性、可观测性^[3,4]。

云原生计算推动了集群管理基础设施的发展。容器技术依托内核级资源隔离机制和标准化镜像分发方式, 提高了微服务封装、交付和迁移效率^[5]; 容器部署的多节点资源调度、服务编排和集群管理需求的增长, 谷歌公司开发了容器管理系统 Borg^[6], 并以此为参考在 2014 年开源容器编排平台 Kubernetes(K8S), 后捐赠给云原生基金会 CNCF 管理。Kubernetes 通过主从式集群架构与声明式 API 实现对容器资源与应用状态的分布式管理, CNCF 2025 年度调查报告^[7] 显示, 生产企业的 Kubernetes 采用率已达 98%。

云原生基础设施维护依赖众多开源云原生应用, 但存在第三方权限提升风险。为实现日志采集、监报告警、身份认证等能力, 系统管理方引入例如 Fluentd、Prometheus 和 Keycloak 等开源云原生应用。这些云原生应用能够增强平台能力、降低开发运维成本, 但其组件深度参与集群运行与控制过程, 关联 Kubernetes 集群系统安全。其中, **权限安全风险是容器集群基础设施的重要风险来源**。云原生应用为完成资源访问、状态同步或集群控制等功能, 通常需要通过 RBAC 机制^[8] 获取一定的集群访问权限。若相关组件存在过度授权^[9]、授权检查缺陷、敏感信息泄露^[10] 或网络隔离配置不当等问题, 攻击者便可能借此扩大访问范围并进一步获取更高权限, 进而导致攻击横向移动, 甚至控制整个集群^[11]。此类因应用实现缺陷、权限配置不当或安全边界失效而使攻击者获得超出预期权限的安全问题, 属于云原生应用的**权限提升漏洞**。

云原生应用的权限提升漏洞管理受制于漏洞补丁滞后性与防控技术局限

性。一方面，漏洞补丁滞后会暴露攻击窗口：开源社区云原生组件在暴露漏洞后的补丁发布时间存在不确定性，且漏洞补丁部署到生产环境中存在流程和兼容问题；另一方面，现有防控技术有效性不足：现有云原生安全技术和研究为 Kubernetes 安全提供了基础，配置扫描^[12,13]、运行时检测^[14]、准入控制^[15,16] 和网络隔离^[17] 等方法可实现暴露面收敛^[18,19]，但这些技术面向通用场景，应对具体高危 CVE 时缺乏有效性或需要专业安全人员编写领域特定代码。

因此，如何在补丁尚未落地的前提下，结合已有安全技术面向具体漏洞快速生成可落地的防控方案，是云原生权限提升漏洞防控中的关键问题。本文围绕这一问题，研究结合大语言模型与安全技术策略的面向具体 CVE 的防控方法，提出集合漏洞语义向可执行安全策略代码的安全体系，并开发了支持工具。基于搜集的云原生应用权限提升工具进行了实验，验证显示所提方法在策略生成、模型对接与部署上具有优秀的性能。

1.2 研究内容与成果

围绕补丁窗口期内 Kubernetes 权限提升漏洞难以及时处置的问题，本文以面向具体漏洞生成可部署的临时防控策略为目标，研究基于多智能体安全策略管理的智能化漏洞防控技术，并完成原型系统实现以及实验验证。本文的工作既关注漏洞证据的组织与分析，也关注策略代码的生成、评估与部署，力图形成从漏洞披露到临时防控落地的完整闭环。主要研究内容与成果如下。

(1) 提出一种基于多智能体安全策略管理的漏洞防控技术。围绕补丁窗口期内权限提升漏洞的临时防控需求，本文构建云原生权限提升安全知识库，设计覆盖漏洞分析、策略生成、评审优化与质量评价的多智能体协同 workflow，形成由漏洞证据组织到防控策略生成的技术方法，为面向具体漏洞生成可部署的临时处置策略提供支撑。

(2) 实现面向云原生权限提升漏洞的原型系统。基于所提方法，本文设计并实现了原型系统 KPEAgent，集成知识管理、workflow编排、模型对接、人工交互与部署验证等功能，建立从漏洞输入到策略输出与结果复核的运行支撑平台，为方法的工程化落地提供实现载体。

(3) 实验验证与案例分析。本文通过样本实验、对比实验与典型案例验证，对所提方法在策略有效性、无干扰性和可部署性等方面进行了系统评估。结果表明，该方法能够较好支持面向具体漏洞的临时防控策略生成，并在实

际部署场景中体现出较好的针对性与可行性。

1.3 论文组织结构

第一章， 引言，交代课题的研究背景与意义、研究内容与成果。

第二章， 背景介绍，梳理相关概念及技术、云原生安全技术与国内外研究现状。

第三章， 面向云原生权限提升漏洞的智能化防控技术，包括总体方法概述与基本概念定义、云原生权限提升安全知识库、基于多智能体协同的策略生成与优化以及策略评估与质量控制。

第四章， 智能化防控系统的设计与实现，包括需求分析、总体设计、详细设计与实现。

第五章， 实验研究，围绕研究问题、实验对象与设计以及结果分析展开。

第六章， 案例分析，选取典型 CVE 案例进行漏洞复现与策略部署验证。

第七章， 研究工作的总结以及未来的展望。

2 背景介绍

本章介绍相关概念与技术，并梳理 Kubernetes 权限提升漏洞相关的国内外研究现状。

2.1 相关概念与技术

2.1.1 容器

计算服务的部署形态经历了从物理机、虚拟机到容器化的持续演进过程（见图 2-1）。虚拟机部署依赖 KVM 等虚拟化技术，能够提供完整操作系统级别的隔离，但也带来了较高的虚拟化开销、较长的启动时间和较大的镜像体积，从而影响应用交付效率与资源利用率。相比之下，容器通过操作系统内核级的资源隔离共享宿主机内核，在资源利用率和启动速度上更具优势。容器以**镜像**为交付单元，将应用及其运行依赖整体打包。其实现主要依赖以下三类内核机制：

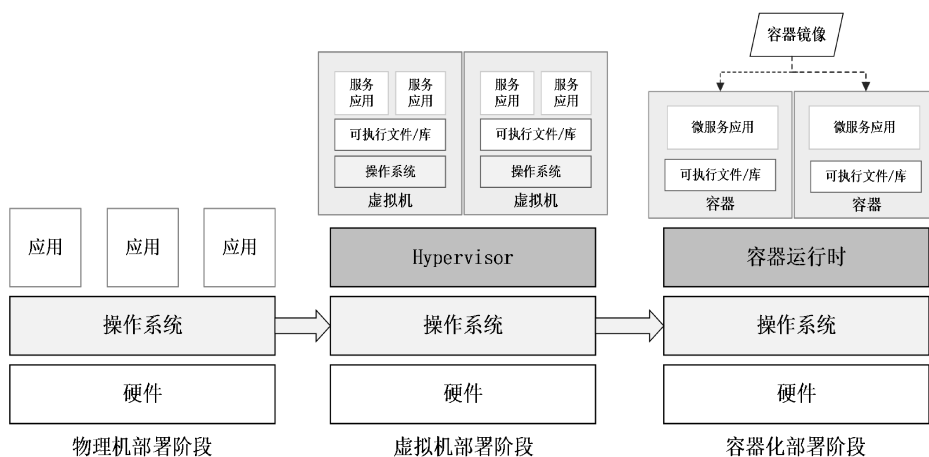


图 2-1: 服务部署形式演进

Fig. 2-1: Evolution of service deployment paradigms

- **命名空间（Namespace）隔离**：通过隔离进程、网络、挂载点等，使容器内运行的程序有独立系统抽象资源；
- **控制组（Cgroup）限制**：负责对容器施加 CPU、内存等硬件资源使用的强制规划，提供物理资源隔离；
- **只读分层与隔离挂载**：利用 OverlayFS 等联合文件系统^[20]，通过复用底层只读镜像并叠加独立的顶层读写区域，实现容器文件系统隔离。

2.1.2 容器集群系统 Kubernetes

Kubernetes 是面向容器集群负载的分布式编排系统，简称 K8S。它源自谷歌内部的集群编排系统 Borg^[6]，并于 2014 年正式开源。其核心思想是以**声明式 API** 描述系统期望状态，再由控制平面持续驱动集群向该状态收敛^[21]。图2-2展示了 Kubernetes 的基本体系结构。一个 Kubernetes 集群由**控制平面**和多个**工作节点**构成。控制平面负责集群控制、调度与状态协调，其中：**API Server** 是资源访问与状态变更的统一入口，并结合键值数据库 ETCD 完成认证、授权、准入和资源对象持久化；**调度器**根据资源请求、亲和性与反亲和性等约束为工作负载选择节点；**控制器**负责持续调节各类资源状态，使其向期望状态收敛。工作节点侧，**kubelet** 负责与 API Server 通信并维护本节点工作负载运行，**容器运行时**负责镜像拉取、容器创建和生命周期管理，**kube-proxy** 负责服务转发与网络规则实现。

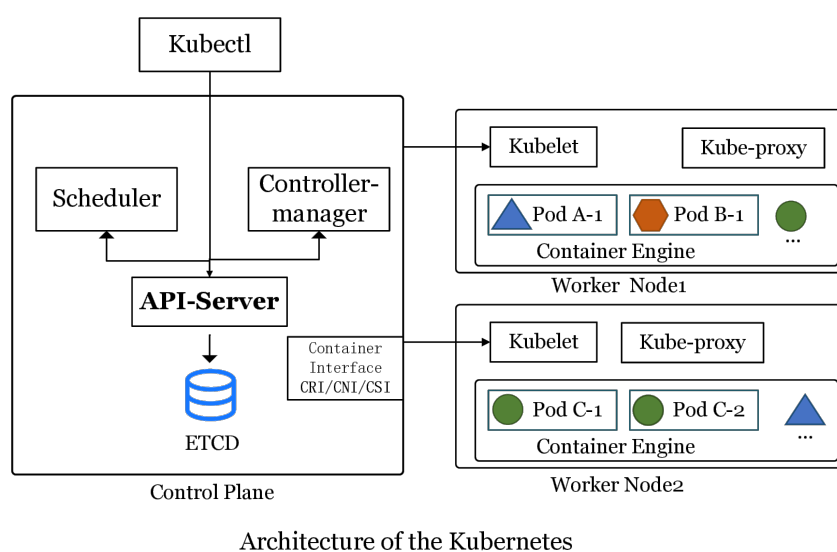


图 2-2: Kubernetes 体系结构示意图

Fig. 2-2: Overview of Kubernetes architecture

Kubernetes 通过一系列持久化**资源对象**描述集群中不同资源的编排与管理。例如，**Pod** 是最小调度单元，包含一个或多个共享网络与存储的容器；**Deployment**、**StatefulSet** 与 **DaemonSet** 分别面向无状态、有状态和节点守护型工作负载提供副本控制与更新策略；**Service** 与 **Endpoint/EndpointSlice** 提供稳定的服务发现与负载均衡抽象；**Ingress** 面向南北向流量提供入口控制；**ConfigMap** 与 **Secret** 用于配置和敏感信息的声明式注入；**Volume**、**PV** 与 **PVC** 则提供存储生命周期与绑定关系的抽象。

2.1.3 Kubernetes 权限模型

基于角色的访问控制（RBAC）是 Kubernetes 授权机制的核心^[21]。图 2-3 左侧展示了权限元模型的形式化表达。该模型可以概括为主语、动作与对象之间的授权关系，即界定何种实体被允许对何种资源执行何种操作，其中：

- **服务账户（ServiceAccount）** 对应模型中的主语节点，表示发起请求的身份载体。在 Kubernetes 中，工作负载通常通过 ServiceAccount 与控制平面交互。
- **权限角色（Role/ClusterRole）** 对应模型中的动作与对象约束，用于声明在特定作用域内针对各类资源（resources）允许执行的操作动词（verbs）集合。
- **角色绑定（RoleBinding/ClusterRoleBinding）** 负责建立服务账户与权限角色之间的关联关系，从而完成授权。

图 2-3 右侧给出了一个典型的 RBAC 授权流程示例：首先创建名为 kada 的 ServiceAccount；随后定义名为 basic-operation 的 Role，并在其 rules 字段中授予对 pods 与 secrets 的 get、list、create 权限；最后通过 RoleBinding 将该 Role 绑定至 kada 账户。这类配置在接入第三方应用时较为常见，但若授予的权限范围超出实际需要，就可能引发权限提升风险。

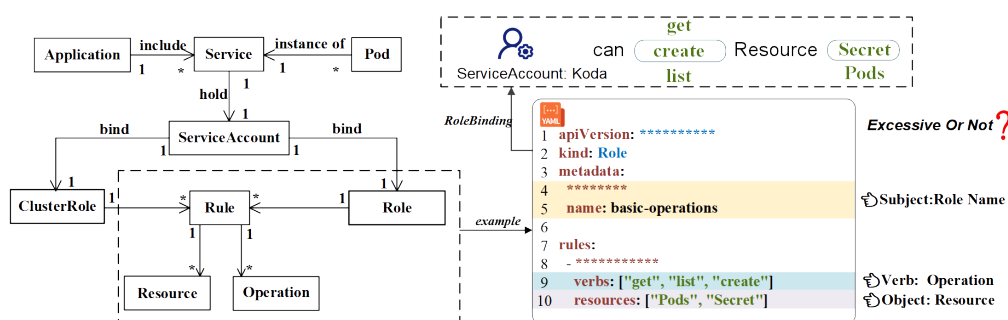


图 2-3: Kubernetes RBAC 权限配置 YAML 文件示意^[10]

Fig. 2-3: Example of Kubernetes RBAC permission configuration in YAML^[10]

2.1.4 Kubernetes 权限提升漏洞

权限提升是指攻击者在仅具备受限权限或初始执行能力的情况下，借助漏洞或错误配置将自身权限提升到更高等级的过程^[22]。在 Kubernetes 场景下，这类问题既可能表现为授权范围扩大或横向获取，也可能表现为节点侧系统权限提升，从而影响更大范围的资源与工作负载。

在云原生环境中，权限提升的危害通常具有更强的放大效应。一方面，单个节点往往承载多个 Pod，一旦节点侧权限提升成功，攻击者便可能对同节点上的多个工作负载实施敏感信息窃取、运行环境篡改或恶意组件注入。另一方面，Kubernetes 的资源对象与运维动作高度集中且自动化，控制平面授权不当也会显著放大攻击后果。结合图 2-3 中的示例，若主体被授予针对 Secret 的 get/list 权限，攻击者即可读取服务账户令牌、数据库口令或 API 密钥等敏感信息；若被授予针对 Pod 的 create 权限，则可能进一步创建携带恶意镜像、特权配置或宿主机挂载的工作负载，作为横向移动、凭据窃取乃至后续节点侧利用的跳板。

图2-4 以 CVE-2024-33522 为例展示了典型的节点侧权限提升漏洞。该漏洞源于 Calico CNI 安装程序的 SUID 位配置不当。若攻击者借助弱口令或服务暴露等前置手段获取节点本地访问权限，便可注入恶意二进制文件，并诱导安装程序以高权限执行。越权成功后，攻击者可以直接接管容器运行时数据、窃取高权限凭据并干扰网络组件，其影响将突破单一工作负载的隔离边界，迅速蔓延至整个集群。

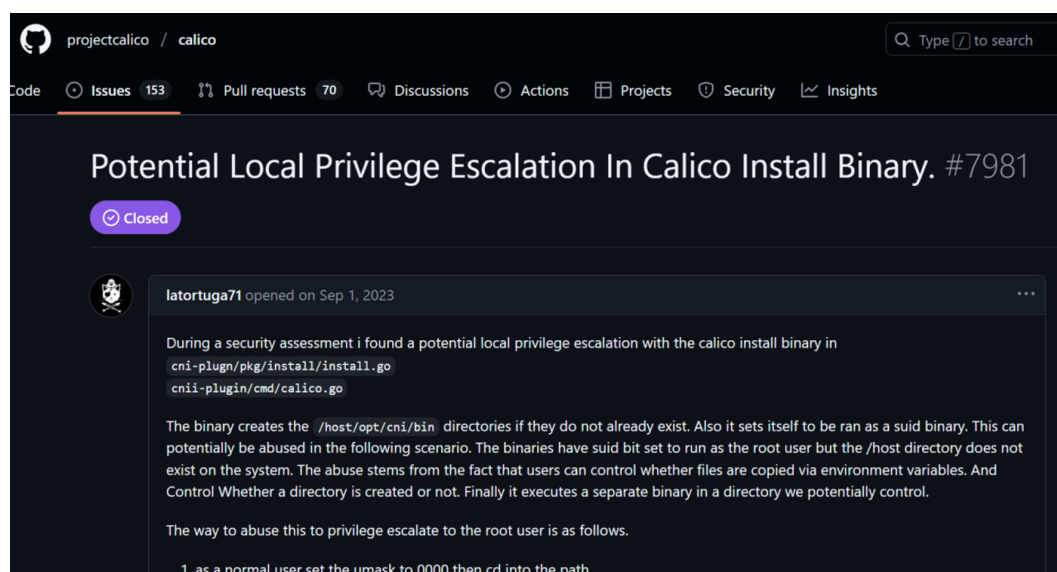


图 2-4: CVE-2024-33522（Calico）节点侧权限提升漏洞报告 ISSUE

Fig. 2-4: Issue report of node-side privilege escalation vulnerability CVE-2024-33522 in Calico

2.1.5 云原生安全技术

云原生安全技术围绕镜像构建、资源准入、网络通信和运行时行为等关键环节形成分层防线，常见工具大多以规则化策略或声明式配置为载体，用于在

不同阶段识别或约束风险。从作用位置看,这类技术大致可归纳为静态扫描与配置审计、准入控制、网络隔离和运行时监控四类,它们共同构成 Kubernetes 集群全生命周期安全治理的基础能力。

静态扫描与配置审计主要用于在镜像构建与部署前发现已知漏洞、弱配置和权限异常。代表性工具包括 Trivy^[23]、Grype/Anchore^[24] 等镜像与依赖漏洞扫描工具, Syft^[25] 等 SBOM 生成工具, 以及 Kubescape^[26]、kube-linter^[27]、kube-score^[28]、KubeSec^[29]、Polaris^[30]、Checkov^[31]、KubiScan^[32]、Krane^[33] 和 RBAC Police^[34] 等面向清单、IaC 与 RBAC 的审计工具。这类方法便于集成到 CI/CD 流程中,能够提前暴露高风险对象,但其判断依据主要来自既有漏洞库、配置基线和审计规则,对特定漏洞的真实利用条件仍缺乏上下文感知能力。

与之相比,准入控制、网络隔离与运行时监控更侧重集群运行阶段的风险约束。OPA/Gatekeeper^[15] 与 Kyverno^[16] 在 API Server 持久化资源前执行策略校验,可用于拦截特权容器、危险挂载和敏感权限变更; NetworkPolicy 及其在 Calico、Cilium 和 Weave Net 等插件中的实现,用于收敛服务间连通范围并降低横向移动风险; Falco^[14] 则依托 eBPF 技术^[35] 采集系统调用和 Kubernetes 事件,并通过规则引擎对异常行为进行实时检测与告警。这些技术分别从事前阻断、横向隔离和事中观测三个层面增强了集群安全性,但整体上仍以通用规则为主,难以直接围绕某一具体漏洞在补丁窗口期快速生成兼顾针对性与可部署性的临时防控策略。

2.2 国内外研究现状

本节按照通用系统权限安全研究、Kubernetes 场景安全研究和课题组已有工作这一顺序展开。前两部分分别概括权限提升风险在一般系统与云原生环境中的研究脉络,最后结合课题组前期成果说明本文工作的承接关系与问题切入点。

2.2.1 系统权限安全研究现状

系统权限安全研究最初聚焦访问控制模型与最小授权原则,随后逐步扩展到对过度授权、运行时提权行为和信息流泄露的综合治理。在基础设计原则层面, Saltzer 与 Schroeder^[36] 于 1975 年指出, **最小权限原则**是实现特权隔离的重要基础。基于这一原则, Ferraiolo 等^[8] 提出基于角色的访问控制模型

(RBAC)，通过角色组织多类权限并将其映射至不同主体，从而缓解规模化授权中的管理难题。然而，**过度授权**现象始终普遍存在，并显著扩大了系统攻击面^[37]。为收敛这类风险，Molloy 等^[38] 以及 Sanders 与 Yue^[39] 分别提出基于日志与算法模型逆向生成应用最小权限集的策略，使权限治理从静态制度设计进一步走向结合实际使用行为的最小权限推断。

随着系统组件化程度不断提高，权限提升风险不再局限于单一主体的越权访问，而更多表现为跨组件、跨应用乃至跨平台的调用链滥用。在典型移动操作系统 Android 中，访问控制的粗粒度特性带来了更突出的权限提升与隐蔽滥用风险。Davi 等人^[40] 较早揭示了**混淆代理**攻击模型，即低权限应用可通过伪造合法请求，诱导高权限组件执行敏感操作。Hutchinson 等^[41] 进一步将该模型概括为三个条件：请求敏感权限、开放导出组件以及存在通向敏感 API 的执行路径。随着攻防对抗持续升级，提权漏洞的利用方式也由单一组件劫持^[42]，演变为多重调用链上的跨应用共谋^[43]，甚至扩展到同一设备内由第三方 SDK 发起的共谋攻击^[44]。

围绕上述风险，现有治理方法大致可归纳为静态架构分析、动态行为监控和数据流追踪三类。其中，静态架构分析主要用于识别权限配置与组件关系中的潜在风险。Au 等^[45] 提出 PScout，通过分析系统源码自动提取底层权限检查图谱；后续研究进一步结合最小执行权限要求引入强制访问控制机制，例如 Bagheri 等^[46] 提出 DelDroid，通过静态分析自动推导并强制实施应用所需的最小权限架构，Wu 等^[42] 提出 SecComp，在组件调用层面约束非授权操作以防御劫持攻击。

与之相比，动态行为监控更关注运行期提权行为的发现与拦截。相关研究结合进程代数建模^[47]、系统级探针监控^[48] 以及深度学习与半监督学习^[49]，实现对权限提升行为的实时预警。进一步地，当研究目标由识别异常操作转向解释敏感数据究竟如何被滥用时，数据流追踪开始成为重要补充。相关研究由面向 Web 的混合污点分析^[50] 逐步扩展到移动系统中的动态污点跟踪。代表性工作 TaintDroid^[51] 在 Android 虚拟化执行环境中引入系统级污点传播机制，可同时跟踪多类隐私敏感数据源，并以较低运行开销实现对第三方应用数据收集与外传行为的实时分析，说明运行时信息流追踪能够为识别隐私滥用提供直接证据。此后，针对 Android 5.0 之后 ART 取代 Dalvik、解释器插桩方案难以继续适用，以及基于模拟器的分析难以部署到真实设备的问题，TaintMan^[52] 进一步通过静态插桩与引用劫持机制，在非 Root 真机上实现了

兼容 ART 的动态污点分析，并将跟踪范围从显式数据依赖扩展到部分由控制依赖引起的隐式信息流。整体来看，这一路径表明权限安全分析正由静态权限归属判断进一步转向对真实调用链利用过程的分析。

上述思路随后被扩展到小程序等宿主内嵌生态的权限安全研究。Zhang 等^[53] 对多个主流小程序生态开展系统性研究，提取了通用的权限控制抽象模型，揭示了跨运行态环境下的多类典型越权漏洞并通过实际攻击加以验证；Wang 等^[54] 则聚焦小程序数据权限滥用问题，定义了多种违规权限申请模式，并设计静态分析工具 MiniChecker 开展规模化检测，指出现有平台权限机制在底层设计上仍存在固有缺陷。总体来看，通用系统中的相关研究为识别跨组件调用、越权传播和后果溯源提供了重要方法基础，但其分析对象和执行环境与 Kubernetes 仍存在明显差异。

2.2.2 Kubernetes 安全研究现状

在云原生场景中，Kubernetes 将配置、控制平面、网络与运行时机制耦合在同一系统之中，权限风险往往沿部署模板、授权链路和组件交互同步扩散。因此，相关研究既继承了通用系统中的最小授权与行为检测思路，又形成了更强调配置语义和多层联动的分析路径。现有工作大体可分为配置风险识别、模板与 IaC 安全分析、架构与权限链风险挖掘以及网络与运行时防护四类。

安全误配置是 Kubernetes 场景中最常见、也最早受到关注的风险来源。早期研究主要用于勾勒配置风险的整体图景：Islam Shamim 等^[55] 系统总结了 Kubernetes 安全实践，Shamim^[56] 进一步指出部署清单偏离最佳实践可能直接演化为可利用的提权路径，Curtis 与 Eisty^[57] 基于 Stack Overflow 数据的实证分析也表明配置安全始终是开发侧的核心关切。在此基础上，Rahman 等^[13] 对开源清单中的典型安全误配置进行了大规模归纳与实证研究。然而，仅依赖静态扫描仍难以完整反映风险的真实可利用性。Shamim 等^[12] 评估了 CIS 推荐配置的合规现状，发现从业人员对安全基线的理解存在显著差异，现有 SAST 工具的检测覆盖也有明显不足；其后续工作^[58] 进一步结合工业界部署实践，论证了将 DAST 动态分析纳入配置核验流程的必要性。上述研究表明，Kubernetes 配置安全研究正由基线合规判断转向实际可利用风险评估。

在识别单份清单风险之外，基础设施即代码（IaC）也已被证明是 Kubernetes 安全风险的重要来源。针对 Helm Charts 带来的依赖传播与供应链风险，

Zerouali 等^[59] 和 Blaise 与 Rebecchi^[60] 分别从生态规模与依存关系角度提供了分析基础；Minna 等^[61] 进一步评估了大语言模型在修复 IaC 配置异味方面的效果与边界。为缓解多平台安全语义碎片化问题，Ul Haque 等^[62] 提出知识图谱框架 KGSecConfig，对安全配置概念进行统一抽取与建模。这类研究表明，Kubernetes 安全治理已由局部规则检查逐步扩展到面向模板、依赖关系与安全语义的自动化分析。

仅分析配置项是否合规仍不足以解释攻击链如何形成，因此相关研究进一步深入到体系结构缺陷与过度授权所引发的链式风险。Dell’Imagine 等^[63] 面向微服务部署定义了一组安全反模式，并实现自动检测工具 KubeHound，表明安全缺陷可能嵌入系统交互结构之中。Yang 等^[11] 系统分析了 CNCF 生态中第三方插件的权限配置，设计了多种攻击路径利用策略，可将工作节点逃逸演化为全集群接管。Gu 等^[9] 提出 EPScan，通过面向容器的程序分析自动比对实际资源访问行为与声明权限，识别可被利用的过度特权。与前述误配置研究相比，这一方向更关注风险沿授权链路的形成与放大机制。

即便控制平面配置与授权链路得到一定约束，底层网络与运行时隔离仍是 Kubernetes 安全防护不可缺少的一环。容器编排引入的网络抽象与传统安全模型存在显著差异，Minna 等^[18] 指出这种差异可能衍生非预期的攻击路径。在网络策略的工程落地层面，Budigiri 等^[19] 基于 eBPF 评估了网络策略的性能代价，Kim 等^[64] 对比了多种容器网络接口插件的安全能力，两项工作共同揭示了协议过滤深度与转发性能之间的固有权衡。在缓解机制方面，研究视角涵盖了从 DevOps 流水线到节点侧运行时的多个层面：Pecka 等^[65] 通过构造流水线攻击场景证明了工具合规并不等同于安全，Cesarano 与 Natella^[66] 提出的 KubeFence 通过裁剪非必要 API 参数实现了比 RBAC 更细粒度的接口过滤，Zhu 与 Gehrmann^[67] 和 Le 等^[68] 则分别在 AppArmor 配置自动生成与系统调用感知调度方向上强化了节点侧防护。综合来看，现有研究已经从清单级合规检查推进到跨组件授权链与运行时隔离的综合分析，但对于权限提升漏洞在暴露窗口期如何快速生成兼顾监控与防护的协同策略，仍缺乏统一的方法框架。

2.2.3 前期工作

在上述研究基础上，课题组围绕 Kubernetes 权限风险检测开展了系统研究，已形成覆盖风险建模、原型实现与实验验证的前期积累。其中，代表性成

果为 KubeGuard 框架。该框架面向过度权限、权限提升和敏感信息泄露三类风险，分别设计了最小权限集构建、规则化审计以及敏感词监测机制^[10]。如图2-5所示，KubeGuard 由过度权限检测、权限提升检测和敏感信息泄露检测三个模块构成。

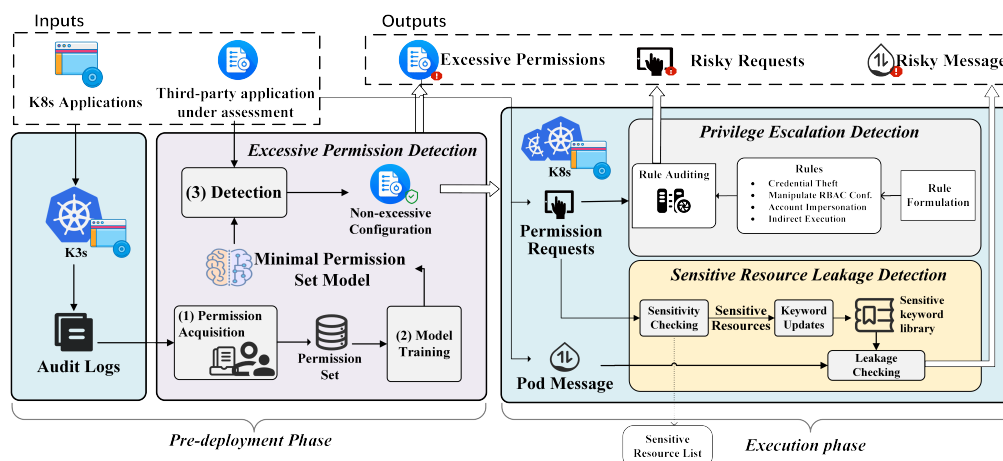


图 2-5: KubeGuard 方法框架图^[10]

Fig. 2-5: Framework of the KubeGuard method^[10]

在方法设计上，KubeGuard 一方面利用 NLP 技术识别应用所需的最小权限集，以判定权限配置是否存在过度授权；另一方面通过动态规则审计检测多类权限提升风险。具体而言，该工作将权限提升场景归纳为凭证窃取、账户冒用、RBAC 操纵和间接执行四类，并针对各类场景提出相应的检测思路。考虑到规则细节较为繁杂，相关风险描述与检测规则在正文中不再展开，具体内容见附录表A-2。此外，该框架还通过监测 Pod 间消息传输并结合关键词匹配，对涉及敏感资源的内容进行告警，从而覆盖敏感信息泄露风险。

在系统实现与效果验证方面，课题组进一步实现了原型系统 KubeGuarder，并在开源 Kubernetes 应用与模拟场景中开展实验评估^[10]。实验结果表明，该系统在过度权限检测方面相较静态基线方法表现出更高的有效性与准确性，同时能够识别多类型的权限提升与敏感信息泄露风险。这说明前期工作已经验证了从权限视角开展风险建模与检测的可行性，并为本文进一步研究提供了方法与系统基础。

不过，现有工作仍主要停留在权限层面的规则化检测，其核心依据是 RBAC 配置与 API 访问行为之间的匹配关系。该方法虽然能够较为系统地覆盖多种权限提升模式，但在真实生产环境中仍存在三方面局限：（1）仅依据权限配置与操作行为的对应关系，难以准确区分正常运维行为与恶意提权行为，因此误报风险较高；（2）该方法主要面向控制平面与资源访问层，尚

未深入节点侧系统层面的权限提升漏洞，对容器逃逸、SUID 配置不当及内核漏洞等底层利用缺乏针对性的检测能力；（3）基于预定义规则的检测方式对未知漏洞与新型攻击手法的泛化能力有限。上述不足也构成了本文进一步研究的直接出发点，即在保留权限视角优势的同时，引入面向真实利用链与运行态场景的检测与防护能力。

2.3 小结

本章介绍了容器、Kubernetes 及其权限模型等基础概念与云原生安全技术，并从系统权限安全、Kubernetes 安全与课题组前期工作三个层面梳理了国内外研究现状，为后续章节的方法设计与实验验证奠定了理论基础。

3 面向云原生权限提升漏洞的智能化防控技术

本章系统阐述面向云原生权限提升漏洞的智能化防控方法，先说明研究动机与方法框架，再介绍基本概念定义，并围绕知识库构建、多智能体策略生成与质量评估三项核心技术展开具体设计。

3.1 研究动机

本章讨论的问题，主要发生在漏洞已经披露但补丁尚未真正落地的处置阶段。对生产环境而言，第三方组件升级通常受到兼容性验证、变更审批与业务连续性要求等因素制约，补丁发布并不意味着系统能够立即完成修复。本文对 2020 至 2025 年间 55 个云原生权限提升漏洞的统计表明，30.4% 的漏洞因补丁延迟而长期处于可利用状态。与此同时，攻击者却可以借助公开公告、修复提交或社区讨论快速还原利用条件。因此，在补丁窗口期内形成临时防控措施，并尽快阻断关键利用条件、收敛攻击面和补足观测能力，已经成为云原生漏洞处置中的独立任务，而不只是正式修复前的附属动作。

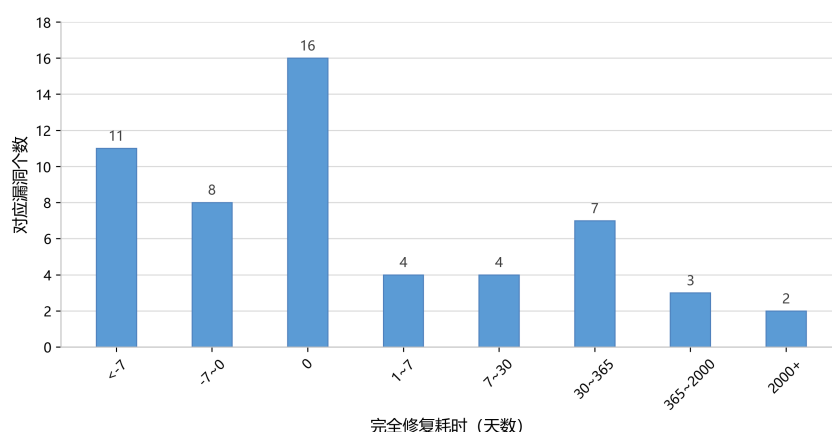


图 3-1: 云原生环境下权限提升漏洞的修复时间分布

Fig. 3-1: Distribution of remediation time for privilege escalation vulnerabilities in cloud-native environments

这一处置任务的难点在于，临时防控策略并不能直接从 CVE 编号或简短公告中得到。权限提升漏洞往往同时涉及 RBAC 权限配置^[8]、控制平面交互、工作负载部署方式、运行时行为和网络可达关系等多个层面，并常与过度授权^[11]、授权检查缺陷、敏感信息泄露^[10] 等问题交织出现。真正有用的处置依据，通常分散在漏洞公告、修复提交、Issue/PR 讨论、配置清单与相关

代码之中。要在较短时间内形成有效策略，需要先重建漏洞语义，再识别关键控制点，并据此判断应采用准入控制、RBAC 收敛、运行时监控还是网络隔离等措施。换言之，这里面对的不是单一规则编写问题，而是面向具体漏洞的证据组织与控制路径选择问题。

围绕云原生安全治理，现有工具已经覆盖了较完整的风险管控周期。静态层面，已有方法可对镜像依赖与 Kubernetes 清单进行扫描^[12,13]；动态层面，则包括基于 eBPF 的运行时代观测与审计^[14]、OPA/Gatekeeper^[15] 和 Kyverno^[16] 等准入控制机制，以及通过网络策略约束横向连通范围的防护手段^[18,19]。这些技术为集群安全提供了必要基础，但大多围绕通用基线核查和既有规则模式展开，更擅长识别普遍性风险；当目标转向某一具体漏洞时，它们往往缺少从多源证据出发、围绕利用条件组合控制措施的能力。实践中真正欠缺的，并不是新的单点工具，而是能够将已有工具与漏洞证据关联起来、面向补丁窗口期快速输出针对性临时策略的方法。

大语言模型提供了新的防控方案。其价值并不在于替代扫描器、准入控制器或运行时系统，而在于能够从 CVE 公告中提取攻击语义、受影响对象与潜在控制点。如果将这种能力与现有安全工具的检测、约束和执行能力结合起来，便有机会形成一条从漏洞证据整理到策略生成、再到部署验证的协同处置路径。因此，本文的出发点并不是构造一种脱离现有生态的全新安全工具，而是探索如何将 LLM 与既有安全工具有效衔接，使其共同服务于补丁窗口期内的临时防控任务。

若要使这一思路真正落到可执行的方法设计上，还需要进一步回答以下三个问题。

- (1) **如何构建面向具体漏洞的安全上下文。** 权限提升漏洞的关键信息分散在代码仓库、漏洞公告、Issue/PR 讨论与修复提交等多类来源中。若缺乏统一的组织方式，模型很难准确把握漏洞成因、利用前提与影响边界。因此，方法首先需要建立可检索、可追溯的领域知识支撑，将分散证据转化为可供生成环节直接使用的安全上下文。
- (2) **如何将漏洞语义映射为可部署的控制措施。** 即便能够理解漏洞语义，临时防控策略的形成仍需要回答“在哪个控制面介入、采用何种约束方式、如何与现有工具配合”这一系列问题。也就是说，方法设计不仅要完成文本层面的归纳，还要把漏洞语义进一步转化为面向 Kubernetes 控制面的具体策略表达。

(3) 如何控制生成结果的不确定性。大语言模型在将漏洞语义转化为策略代码时，仍可能出现幻觉、逻辑矛盾或偏离实际环境的问题。若缺乏有效的验证与校正机制，生成结果不仅可能无法阻断攻击，还可能干扰正常业务。因此，方法还必须考虑如何通过多阶段协同、结果评审与反馈优化，提高策略的有效性、无干扰性与可部署性。

基于上述动机，本文选择以多智能体协同方式组织整个防控流程，将证据整理、漏洞分析、策略生成、结果评审与反馈优化拆分为相互衔接的处理环节。这样的设计既服务于补丁窗口期内的快速响应需求，也为异构知识融合和生成结果控制提供了可操作的实现路径。后续内容将围绕这一思路，依次说明方法框架中的关键对象、知识支撑机制、协同生成流程以及质量控制方法。

3.2 方法框架

基于前文提出的现实需求，本文将补丁窗口期内云原生权限提升漏洞的防控问题建模为面向具体漏洞生成临时策略的自动化过程。按照控制手段的作用方式，相关措施可概括为监控策略与防护策略两个互补层次。其中，防护策略侧重在漏洞利用发生前阻断关键条件并收敛攻击面；监控策略侧重对运行时异常行为进行持续观测、告警与取证。围绕这两个目标，本文构建了基于多智能体安全策略管理的智能化漏洞防控框架。

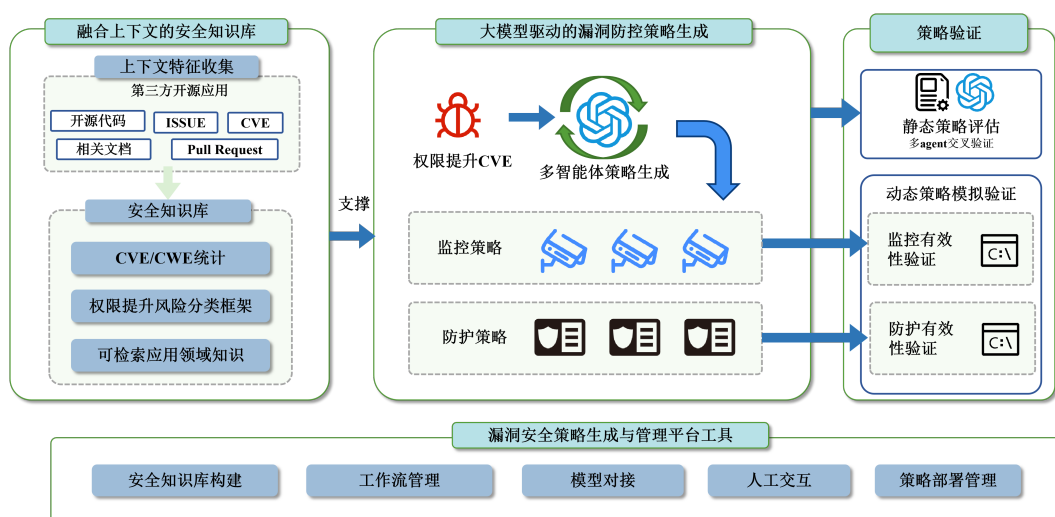


图 3-2: 基于多智能体的策略化漏洞防控研究总体框架

Fig. 3-2: Overall multi-agent framework for strategy-based vulnerability prevention and control based on large language models

如图 3-2所示，该框架以权限提升类 CVE 为输入，依托大语言模型与多

智能体协同，结合检索增强生成、思维链推理与反馈优化等技术，围绕安全知识支撑、策略生成、策略验证和平台工具建设，形成从漏洞披露到输出结构化 JSON 临时防控策略的自动化流程。具体而言，系统首先从代码仓库、漏洞公告、Issue/PR 讨论与补丁提交中提取多源证据，构建可追溯的漏洞安全知识库；随后通过安全知识、漏洞分析、策略生成、策略评审与策略优化等智能体协同，将漏洞语义转化为面向 Kubernetes 平台的监控与防护策略；最后结合质量评价与平台工具，实现结果排序、人工复核与部署验证。下面将依次对该流程涉及的对象定义、知识支撑机制、生成方法与评价过程展开说明。

3.2.1 基本概念定义

基本对象

为形式化描述面向漏洞的安全策略生成问题，本文将方法中的核心对象抽象为漏洞实例、安全上下文与安全策略三类实体。本文涉及的主要记号与核心术语已在前置部分的符号清单与术语表中汇总，这里仅给出与方法定义直接相关的形式化说明。记候选策略全集为 Π 。策略质量评价所涉及的维度与记号将在下一小节中进一步给出。

定义 1 (漏洞实例)。设 $\mathcal{V}$ 为漏洞实例空间。任一漏洞实例表示为

$$v = \langle id, d, m \rangle \in \mathcal{V} \quad (3-1)$$

其中， id 为漏洞标识符（如 CVE 编号）， d 为漏洞描述文本， m 为与漏洞相关的元数据，用于刻画其严重性、受影响组件及版本范围等属性。

定义 2 (安全上下文)。给定漏洞实例 v ，其安全上下文 $c \in \mathcal{C}$ 定义为带来源标识的有限证据集合：

$$c = \{ \langle t_i, src_i \rangle \mid i \in [n_c] \} \quad (3-2)$$

其中， t_i 表示证据内容， src_i 表示来源标识（如文档路径、公告链接或代码位置）。该定义强调上下文不仅提供与漏洞相关的背景事实，而且保留事实来源，以支撑后续策略生成与结果复核。

定义 3 (安全策略)。针对给定漏洞实例 v 的完整安全策略表示为三元组

$$\pi = \langle type, method, action \rangle \quad (3-3)$$

其中， $type \in \{\text{mon}, \text{prot}\}$ 表示策略属于监控型还是防护型； $method = (m_1, \dots, m_k)$

表示控制机理序列， $action = (a_1, \dots, a_k)$ 表示与之对应的执行表达序列。二者满足一一对应关系，即每个 m_i 均对应一个可部署的 a_i ，共同构成完整的防控措施单元。

对给定漏洞实例 v ，生成阶段得到的候选策略集合记为 $\Pi_v \subseteq \Pi$ 。

质量评价指标

由于不同漏洞样本在业务环境、证据完备程度与候选策略规模上存在差异，若直接采用绝对分值，容易引入尺度漂移与比较偏置。与此同时，监控策略与防护策略在优化目标上并不完全同质，前者更强调观测、告警与取证能力，后者更强调阻断、收敛与约束效果。因此，本文在评价阶段先按策略类型划分候选集合，再在各自子集中实施基于排序的相对评价。记类型集合为 $\mathcal{T} = \{\text{mon}, \text{prot}\}$ ，对任意 $\tau \in \mathcal{T}$ ，定义

$$\Pi_v^\tau = \Pi_v \cap \Pi_\tau \quad (3-4)$$

表示漏洞实例 v 在类型 τ 下的同质候选子集。下文若不强调策略类型，统一用 Π_v 表示漏洞实例 v 的候选策略集合；当需要区分监控型与防护型时，再使用 Π_v^τ 表示对应子集。记评价维度集合为 $\Delta = \{E, I, D\}$ ，分别对应有效性、无干扰性与可部署性。

定义 4（三维质量向量）。策略质量由映射 $q: \Pi \rightarrow [0, 1]^3$ 给出：

$$q(\pi) = (E(\pi), I(\pi), D(\pi)) \quad (3-5)$$

其中， E 衡量策略对主要攻击路径与关键风险面的覆盖程度， I 衡量策略对正常业务的扰动程度及误报风险， D 衡量策略在现有环境中的落地难度与执行清晰度。

3.2.2 云原生权限提升安全知识库

如图 3-3 所示，本节围绕漏洞实例 v 构建可追溯云原生权限提升安全知识库 $\mathcal{K}_v$ ，并通过检索形成安全上下文 c ，为后续策略生成提供带来源标识的证据支撑。

数据采集

本文以 Kubernetes 及其生态开源应用为研究对象，首先从 CNCF Landscape 获取云原生应用清单，并对清单进行快照固化，以保证采集过程可复

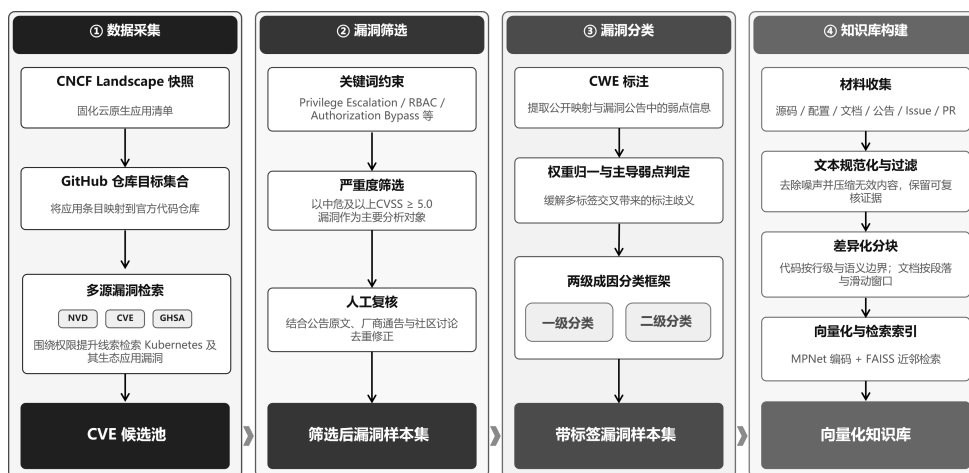


图 3-3: 云原生权限提升漏洞知识库构建

Fig. 3-3: Overview of the cloud-native privilege escalation vulnerability knowledge base construction

现。随后，将清单条目名称映射到其官方代码仓库，形成后续漏洞检索与证据收集的目标集合。

在此基础上，本文综合使用 NIST NVD¹、MITRE CVE 官方库²与安全公告平台 GitHub Security Advisory³三个来源，围绕权限提升线索对 Kubernetes 及其生态应用开展多源漏洞检索，获得初步 CVE 候选池。

漏洞筛选

为从 CVE 候选池中筛选出高质量样本，本文采用关键词约束、严重性阈值与人工复核相结合的重重过滤策略：

- **关键词约束**：在 CVE 描述与参考链接中匹配 Privilege Escalation、Authorization Bypass、Improper Access Control、RBAC、ServiceAccount、ClusterRole、Container Escape/Sandbox Escape/Breakout 等关键词；
- **严重性阈值**：采用 CVSS v3.x 评分，并以 ≥ 5.0 的中危及以上漏洞为主要对象；
- **人工复核**：对候选条目结合公告原文、厂商通告与社区讨论进行复核，剔除重复或与权限提升无关的记录，并补全攻击前提、影响面与可用缓解信息。

最终保留的结构化字段包括 CVE 编号、漏洞摘要、披露时间、受影响组件与版本范围、修复版本、CVSS 评分、参考链接，以及与修复相关的补丁提交记

¹NIST NVD (National Vulnerability Database): <https://nvd.nist.gov/>

²MITRE CVE: <https://cve.mitre.org/>

³GitHub Security Advisory: <https://github.com/advisories>

录等。上述字段与漏洞实例 $v = \langle id, d, m \rangle$ 直接对应：CVE 编号即标识符 id ，漏洞摘要构成描述 d ，其余属性如严重性评分、受影响版本和参考链接等共同构成元数据 m 。

为保证数据构建过程可复现，本文对同一批次样本采用统一快照方式完成采集，并在构建配置中记录应用清单快照、检索时间窗口、纳入与剔除准则以及去重规则。对于来自不同数据源但指向同一漏洞事实的记录，以 CVE 编号为主键进行合并；对于无独立编号但经公告与补丁信息确认指向同一漏洞事件的记录，则视为重复项而不重复计数。

漏洞分类

漏洞样本的分类方式直接影响后续策略生成的针对性。为此，本文构建了面向策略生成场景的两级成因分类框架，旨在为模型提供稳定且可解释的风险语义线索，从而约束推理路径、辅助关键控制点与实施路径的选择，提升输出策略的可部署性、可验证性与副作用可控性。

首先，本文依据公开映射与公告信息提取每个 CVE 的 CWE（Common Weakness Enumeration）标签，并统计不同 CWE 的出现频次。CWE 是 MITRE 维护的软件弱点枚举体系，可用于将具体 CVE 归并到更抽象的弱点类型。

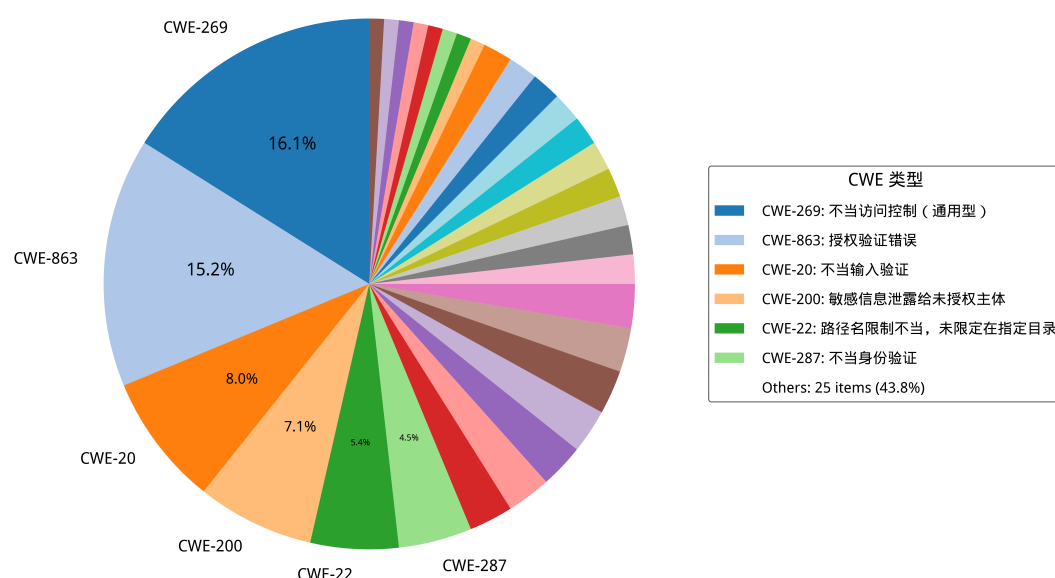


图 3-4: Kubernetes 权限提升 CVE 对应的 CWE 类型占比
Fig. 3-4: Distribution of CWE categories corresponding to Kubernetes privilege escalation CVEs

CWE 体系并不是严格互斥的分类框架。同一漏洞往往会同时对应多个弱点条目，不同条目在抽象层次上也可能彼此交叉。在本文收集的 55 个权限提升漏洞中，有 20 个样本同时关联两个及以上 CWE 标签，占比 36%。为更客观地反映 CWE 分布，统计时令每个 CVE 的总权重为 1，多标签样本按标签数平均分配。结果表明，授权不当、权限管理不当等访问控制类弱点常与输入验证不当等问题同时出现，这说明权限提升漏洞通常具有复合成因。仅依赖单一 CWE 标签，难以为后续防控策略提供稳定的语义约束，因此有必要进一步构建面向防控动作的成因分类框架。

基于这一问题，本文进一步提出面向云原生权限提升漏洞的两级成因分类框架，并为每个漏洞确定一级类和二级类标签。一级分类用于概括高层风险来源，如安全逻辑缺陷、配置缺陷、代码注入和敏感信息泄露；二级分类则进一步细化到更贴近防控动作选择的类型，如授权验证不完善、默认不安全配置等。经过这样的整理，漏洞分析结果可以更自然地映射到 RBAC 最小权限、准入控制、网络隔离、运行时策略与审计等具体控制点。

在策略生成阶段，分类结果作为提示词中的结构化约束，引导模型先识别漏洞的主要成因，再定位关键控制点，最后组织可执行的防控动作，从而减少泛化建议和无关信息，提高策略的针对性与可部署性。完整的一级、二级分类及其与 CWE 的对应关系见附录表 A-1。

为降低多标签条件下的标注歧义，本文对每个样本只保留一个主导的一级类和二级类标签。判定时优先选择直接造成权限边界突破的成因；其余伴随弱点仍保留在 CWE 映射表中，作为补充语义信息。对于边界不清或存在多种解释路径的样本，再结合漏洞公告、修复补丁语义与攻击路径描述进行复核后裁决。

以下对各一级分类分别展开说明。

配置缺陷指应用或基础设施的配置不当导致的安全漏洞，主要包括两种情形：

- **过度权限配置**：Kubernetes RBAC 配置不当、Role/ClusterRole 规则范围过宽或绑定关系不合理等，使低权限主体获得超出其职责的资源访问与高危操作能力。该类问题在第三方应用接入场景中尤为常见，并已被证明可被用于接管集群关键资源^[11,37]。
- **网络隔离不足**：容器、Pod 或服务间缺乏网络隔离，导致攻击者可以横向移动获取其他权限。

表 3-1: 云原生权限提升漏洞分类框架（CWE 对应详见附录表 A-1）
 Tab. 3-1: Classification framework for cloud-native privilege escalation vulnerabilities
 (see Appendix Table A-1 for the CWE mapping)

一级分类	二级分类	示例 CVE
配置缺陷	冗余权限	CVE-2023-30512
	网络隔离不足	CVE-2020-4062
安全逻辑缺陷	访问控制缺陷	CVE-2022-24768
	身份验证绕过	CVE-2023-22482
	输入验证不足	CVE-2023-3955
	授权验证不完善	CVE-2022-21701
敏感信息泄露	API 端口暴露	CVE-2022-1902
	日志泄露	CVE-2019-10165
	配置文件泄露	CVE-2019-3869
代码注入	命令注入	CVE-2021-41254
	路径遍历	CVE-2022-24877

安全逻辑缺陷指应用代码中的安全机制设计或实现不当，涵盖以下情形：

- 访问控制缺陷：应用的权限检查不完全或存在逻辑漏洞，允许绕过权限验证；
- 身份验证绕过：认证机制设计缺陷，攻击者可以冒充他人身份或跳过认证步骤；
- 输入验证不足：对请求参数、资源标识或边界条件的校验不充分，导致攻击者借由异常输入扩大权限边界或触发越权路径；
- 授权验证不完善：权限检查在某些操作路径或边界条件下缺失。

敏感信息泄露指重要的身份、密钥或权限信息意外暴露，常见形式包括：

- API 端口暴露：管理接口、内部 API 或调试端口未受保护暴露在网络上；
- 日志泄露：错误泄露的日志文件中包含密钥、命令历史等敏感信息；
- 配置文件泄露：配置文件包含硬编码的凭证或密钥。

代码注入指应用对用户输入的处理不当，使攻击者得以注入恶意代码，典型子类包括：

- 命令注入：应用直接执行用户提供或可控的命令，攻击者可以执行任意系统命令；
- 路径遍历：应用文件访问检查不完整，攻击者可以访问系统其他部分的文件。

知识库构建

知识库构建阶段的目标是将与漏洞 v 相关的多源材料组织为带来源标注的证据集合，并建立支持按需取证的检索能力。围绕 v 构建知识库 $\mathcal{K}_v$ 后，策略生成阶段即可通过语义检索提取相关证据片段，并按需组装为安全上下文 c ，为后续分析提供可追溯的事实依据。

具体而言，本文首先针对每个漏洞关联的开源应用仓库收集代码、配置与文档等可读文本，同时纳入与漏洞相关的安全公告、议题（Issue）/合并请求（Pull Request）讨论以及修复提交信息。随后，对文本进行最小化规范，并过滤超长或二进制内容，以降低噪声与资源开销。

在完成材料收集后，为兼顾证据粒度与可复核性，本文对不同材料采用差异化分块策略：对代码和配置按行切分，并尽量对齐函数或语义边界；对文档则按段落与窗口切分，并保留适当重叠，以减少跨块语义断裂。

在此基础上，为实现语义层面的证据召回，本文为每个文本块构造向量表示，并建立近邻检索索引。实验实现中，采用 `sentence-transformers` 框架中的 MPNet 句子编码器 `all-mpnet-base-v2` 完成文本表示^[69,70]，并基于 FAISS^[71] 构建相似度索引。为保证可追溯性与可复现性，索引与元数据同步持久化，内容包括向量索引文件、块级元数据以及构建配置。块级元数据记录来源路径或链接、块 ID、块类型与范围等信息；构建配置记录分块参数与检索阈值等关键设置。

上述流程得到的向量知识库为后续策略生成提供了可追溯的证据供给能力。模型在推导控制措施时可以检索到与漏洞相关的关键事实与上下文片段，并通过来源标识完成快速定位与复核。

3.2.3 基于多智能体协同的策略生成与优化

策略生成阶段以漏洞实例 v 与安全上下文 c 为输入，产出候选策略集合 $\Pi_v \subseteq \Pi$ 。按照定义 3，每个候选 $\pi \in \Pi_v$ 都表示为三元组 $\pi = \langle type, method, action \rangle$ 。按策略类型划分， $\Pi_v^{mon} = \Pi_v \cap \Pi_{mon}$ 表示监控策略候选子集， $\Pi_v^{prot} = \Pi_v \cap \Pi_{prot}$ 表示防护策略候选子集。整个流程结合检索增强、思维链推理与反馈优化三类机制，由安全知识、漏洞分析、策略生成、策略评审与策略优化五类智能体分工完成：先整理证据，再形成结构化分析，随后生成候选，并依据评审反馈持续修订，以降低误报与运维噪声并提升可部署性。

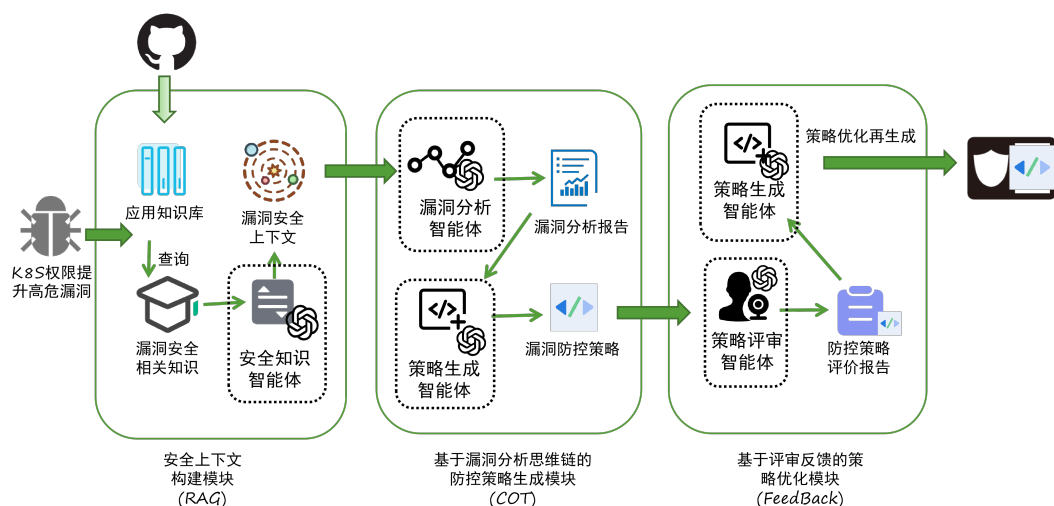


图 3-5: 多智能体协同的策略化漏洞防控工作流

Fig. 3-5: Multi-agent collaborative framework for strategy-based vulnerability prevention and control

安全上下文构建

安全上下文构建的关键在于保证证据可追溯。按照定义 2，证据集合记为 $c = \{ \langle t_i, src_i \rangle \mid i \in [n_c] \}$ 。在知识库阶段，系统为每个片段记录来源元数据，如路径、URL、版本与位置；在生成阶段，再从知识库与安全材料中检索候选证据，并由安全知识智能体完成去重与提炼，输出触发条件、受影响组件、权限边界与可控点，同时保留对应的 src_i 供复核。

为保证不同实验轮次中证据整理过程具有一致的输入输出边界，本文将该智能体的指令约束抽象为由角色、输入、任务和输出构成的四元模式，其在全流程中的统一定义见表 3-2。安全知识智能体的职责是将离散证据整理为带来源标识的结构化摘要，而不是直接给出策略建议。

安全知识智能体提示词简化示例

你是云原生安全知识整理助手。

输入

漏洞实例 v 与候选证据集合 c

任务

提炼受影响组件、触发条件、权限边界、攻击前提与缓解线索，保留每条结论对应的来源标识。

输出

返回结构化证据摘要，不直接生成防控策略。

基于漏洞分析思维链的策略生成

本节将漏洞分析、技术选型与策略生成纳入统一的结构化约束，要求各环节输出可复核的中间结论。与仅给出自然语言建议相比，这种做法能够减少信息缺失与表述歧义。

首先，**漏洞分析智能体**基于漏洞摘要与安全上下文 c 形成结构化分析结果，覆盖漏洞类型、攻击路径、影响范围与风险类别，并与前述成因分类体系保持一致。随后，**策略生成智能体**在结构化指令约束与分步推理引导下，根据分析结论选择控制手段并组织动作序列。具体而言，它会结合平台可用能力，在身份与权限收敛、网络隔离、运行时约束与准入控制等手段之间确定实施路径，并将其映射为最终策略对象；当证据不足以支持唯一结论时，则显式保留待补充信息与必要假设。为保持方法主线清晰，同时保证实验过程可复现，本文保留简化后的提示词示例，并以表 3-2 概括各智能体的统一约束。

策略生成阶段需要在统一的数据模式下将分析结论映射为可部署的策略对象，因此除任务目标外，还需明确输出字段与约束边界。表 3-2 中漏洞分析与策略生成两类智能体的约束，共同保证了这一映射过程的稳定性。

漏洞分析智能体提示词简化示例

你是 Kubernetes 漏洞分析助手。

输入

漏洞实例 v 与安全上下文 c

任务

给出漏洞类型、攻击路径、影响范围与成因分类，
并指出导致权限提升的关键环节。

输出

返回结构化分析结果；证据不足处显式标注 `assumptions`。

表 3-2: 多智能体指令约束摘要
Tab. 3-2: Summary of instruction constraints for the multi-agent workflow

智能体	输入	核心任务	输出约束
安全知识智能体	漏洞实例与候选证据片段	抽取受影响组件、触发条件、攻击前提、权限边界与缓解线索	输出带来源标识的结构化证据摘要
漏洞分析智能体	漏洞实例与安全上下文	形成漏洞类型、攻击路径、影响范围与风险类别的结构化分析	输出满足预定义模式的分析结果
策略生成智能体	中间分析结论、控制手段选择结果与安全上下文	生成监控或防护策略, 并保持字段完整、一致且可验证	输出包含 <code>type</code> 、 <code>method</code> 与 <code>action</code> 的结构化策略对象
策略评审智能体	漏洞实例、候选策略集合与补充约束	从有效性、干扰风险与验证需求等方面形成评审意见	输出包含总体结论与逐策略点评的结构化评审结果
策略优化智能体	漏洞实例、候选策略集合与评审结论	吸收有效控制、剔除高噪声建议, 并补齐前置条件与回滚边界	输出优化后策略及相对原候选的改进说明
评价智能体	匿名化与随机化后的同类型候选策略集合	在 E 、 I 、 D 三个维度上给出严格排序	输出满足预定义模式的维度排序结果及必要判据

策略生成智能体提示词简化示例

你是云原生安全策略生成助手。

输入

中间分析结论、可用控制手段与安全上下文 c

任务

围绕监控或防护目标生成策略 $\pi =$
 $\rightarrow \langle \mathit{type}, \mathit{method}, \mathit{action} \rangle$,
 优先给出可部署、可验证、可回滚的措施。

输出

返回结构化策略对象; 若信息不足, 显式说明前置假设。

为与前述基本概念定义保持一致, 策略对象统一采用 `type`、`method` 与 `action` 三字段模式。其中, `type` 用于区分监控型与防护型策略, `method` 用于描述控制机理, `action` 用于承载与之对应的可执行表达, 如规则片段、配置约束或执行命令。当一项策略包含多条控制措施时, `method` 与 `action` 均

表示为等长序列，并通过位置对应关系保持语义一致。

为保证候选策略具有可比性与交付价值，本文进一步要求其至少满足以下条件：覆盖明确的防控目标，包含可验证的实施步骤，并对潜在副作用与回滚边界作出清晰说明。

候选策略集合 Π_v 通过多候选并行生成方式获得，即围绕同一漏洞构造多个结构化策略对象，以探索不同控制思路与实施路径。为避免异质目标混合比较，生成结果在进入评价阶段前按策略类型划分为 Π_v^{mon} 与 Π_v^{prot} ，并分别进入后续评价与排序环节。

在进入交付与评估阶段之前，系统还需对生成结果进行规范化整理，包括统一字段格式、补齐必要步骤、清理冗余信息，并在证据不足处显式标记假设条件与前置依赖。

基于评审反馈的策略优化

在候选策略生成之后，流程进入评审、反馈与再生成的迭代优化环节，以降低误报与运维噪声，提升可部署性与一致性。系统首先通过并行生成获得多份候选，形成 Π_v ；随后由**策略评审智能体**给出结构化评审，包括有效点、风险、验证要点与缺失信息；最后由**策略优化智能体**合并可行措施、剔除不可行建议，并补齐前置条件与回滚方案。

为保证评审阶段具有一致的比较口径，本文将评审与优化两个环节的指令约束统一纳入表 3-2 所示模式，并保留简化后的提示词示例，用以展示其输入边界、判断依据与输出结构。

策略优化阶段的目标是将多份候选与评审反馈压缩为单一、可交付的最终策略，因此其关键不在于模板表述本身，而在于前序评审信息的吸收边界、修订依据与回滚约束是否得到显式保留。

策略评审智能体提示词简化示例

你是云原生安全策略评审助手。

输入

漏洞实例 v 、候选策略集合 Π_v 及补充约束

任务

从有效性、无干扰性与验证需求三个方面比较候选，

指出优势、风险与缺失信息。

输出

返回结构化评审结果，包含总体结论与逐策略点评。

策略优化智能体提示词简化示例

你是云原生安全策略优化助手。

输入

漏洞实例 v 、候选策略集合 $\setminus P_{i_v}$ 与评审结论

任务

保留有效控制，删除高噪声或不可行措施，
补齐前置条件、验证步骤与回滚边界。

输出

返回优化后的最终策略，说明保留、删除与修订原因。

基于排序的质量评价

基于共识得分可得到维度 δ 的**共识排序** $\bar{R}_\delta$ ，即按 $\hat{S}_\delta(\pi)$ 从大到小进行排序。为保证输出为严格排序，若出现数值相等，则按预先约定的确定性规则打破平分，例如依次比较各评审主体的原始排名，并以匿名编号作为最终判别。

为刻画评审一致性与不确定性，可进一步定义分歧度量，例如秩的加权方差：

$$U_\delta(\pi) = \sum_{h \in \mathcal{H}} w_h \cdot (r_\delta^{(h)}(\pi) - \bar{r}_\delta(\pi))^2, \quad \bar{r}_\delta(\pi) = \sum_{h \in \mathcal{H}} w_h \cdot r_\delta^{(h)}(\pi) \quad (3-6)$$

当 $U_\delta(\pi)$ 较大时，表示该策略在该维度上存在明显分歧，需在后续决策中优先人工复核。

在获得三维共识质量后，进一步给出总体质量函数 $Q: \Pi_v^r \rightarrow [0, 1]$ ：

$$Q(\pi) = \alpha_E \hat{S}_E(\pi) + \alpha_I \hat{S}_I(\pi) + \alpha_D \hat{S}_D(\pi), \quad \alpha_E + \alpha_I + \alpha_D = 1 \quad (3-7)$$

其中 $\alpha_E, \alpha_I, \alpha_D$ 是三个维度的权重，用于将有效性、无干扰性和可部署性的偏好映射到最终排序。该权重可按业务场景配置：例如在应急与临时缓解场

景下可提高 α_E, α_D ；在生产稳定性优先场景下可提高 α_I 。

在本文的安全防控语境下，若无额外业务约束，默认偏好应体现有效性优先，即 $\alpha_E > \alpha_I > \alpha_D$ 。例如可取 $\alpha_E = 0.5, \alpha_I = 0.3, \alpha_D = 0.2$ 作为默认基线，具体数值可结合组织的风险偏好与运维能力进行微调。

为避免权重选择导致结论出现偶然变化，实验中可对 α 做简单敏感性分析，例如在合理范围内扰动 α 并观察总排名是否稳定，再将不稳定样本优先交由专家复核。对任一固定类型 τ ，最终总排名 $\bar{R}^\tau$ 由 $Q(\pi)$ 从大到小排序得到；为保证输出为严格排序，若出现 $Q(\pi)$ 相等，则按 $(\hat{S}_E, \hat{S}_I, \hat{S}_D)$ 的词典序进行细化，并在仍无法区分时以匿名编号作为最终判别。由此，系统可分别得到监控型与防护型候选的可审计交付结论。

3.3 方法示例

为更直观地说明本文方法如何从漏洞证据出发生成可部署的临时防控策略，下面结合 CVE-2024-33522 给出一个完整示例。按照定义 1，可将该漏洞表示为

$$v = \langle id, d, m \rangle = \langle \text{CVE-2024-33522}, d, m \rangle \quad (3-8)$$

其中， d 对应漏洞描述文本， m 则包含受影响组件、版本范围、修复版本与参考资料等元数据。该漏洞属于典型的节点侧权限提升问题，其风险主要不体现在控制平面授权关系，而体现在节点文件系统中存在可被滥用的高权限执行路径，因此适合用来展示本章方法从证据归并到策略交付的全过程。

步骤 1：构建安全上下文 c

按照定义 2，系统首先围绕漏洞实例 v 从漏洞公告、版本修复说明、Calico 组件部署配置、CNI 安装路径说明以及节点加固建议等材料中检索证据，并组织为带来源标识的安全上下文

$$c = \{ \langle t_i, src_i \rangle \mid i \in [n_c] \}. \quad (3-9)$$

对该案例而言，安全知识智能体归并后的关键证据主要包括：

- **利用前提：**攻击者已经获得节点本地访问能力，或能够借助宿主机挂载、容器逃逸等方式接触节点文件系统；
- **关键控制点：**Calico CNI 安装二进制文件的权限位、`/opt/cni/bin` 等节点侧敏感路径的写入面，以及由此延伸出的高权限执行链；
- **缓解线索：**升级到修复版本、移除危险权限位、限制高风险宿主机目录

挂载，并对异常写入与执行行为进行持续监控。

由此，分散证据被收敛为可直接进入后续推理的安全上下文 c ，并将该漏洞转化为三个明确的处置目标：消除当前可利用条件、限制节点侧高风险暴露面，以及补足对异常行为的持续观测。

步骤 2：由 v 和 c 生成候选策略集合 Π_v

在此基础上，漏洞分析智能体基于 v 与 c 识别出该漏洞的核心问题是节点侧高权限执行链可被滥用，而非控制平面 RBAC 越权。因此，策略生成智能体围绕“权限收敛、路径约束、运行时观测”三类控制思路生成候选策略集合 $\Pi_v \subseteq \Pi$ ，并进一步划分为防护型子集 Π_v^{prot} 与监控型子集 Π_v^{mon} 。

例如，可首先得到一个初始防护候选 $\pi_1 \in \Pi_v^{\text{prot}}$ ：

$$\pi_1 = \langle \text{prot}, \text{method}_1, \text{action}_1 \rangle \quad (3-10)$$

其中， method_1 对应“节点权限收敛”， action_1 对应具体的临时 DaemonSet 修复动作。其实现重点包括以下几个方面：

- 扫描节点上的受影响目录，并移除残留的 SUID 位；
- 在执行修复动作后输出当前文件权限状态，保留可复核的处置证据；
- 将升级到修复版本作为后续正式修复动作，而在补丁窗口期内优先通过权限收敛降低风险暴露面。

对应的策略片段可写为：

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: calico-cni-suid-remediate
  namespace: kube-system
spec:
  template:
    spec:
      hostPID: true
      tolerations:
        - operator: Exists
      containers:
        - name: fixer
          image: busybox:1.36
          securityContext:
            privileged: true
            command: ["/bin/sh", "-c"]
          args:
            - |
              find /host/opt/cni/bin -maxdepth 1 -perm -4000 -exec chmod u-s {} \;
              find /host/opt/cni/bin -maxdepth 1 \( -perm -4000 -o -perm -2000 \) -ls
```

```

volumeMounts:
  - name: host-cni
    mountPath: /host/opt/cni/bin
volumes:
  - name: host-cni
    hostPath:
      path: /opt/cni/bin

```

除 π_1 外，系统还可生成监控型候选 $\pi_2 \in \Pi_v^{\text{mon}}$ ，用于发现 `/opt/cni/bin` 下的异常写入、权限变更与可疑执行；以及额外的防护型候选 $\pi_3 \in \Pi_v^{\text{prot}}$ ，用于限制普通工作负载直接挂载高风险宿主机目录。这样，候选集合 Π_v 不再是单一方案，而是覆盖“止血、约束、观测”三类控制目标的多候选集合。

步骤 3：基于评审反馈优化 Π_v

初始候选生成后，策略评审智能体按照有效性、无干扰性与可部署性三个维度对 Π_v 进行审查。对于 π_1 ，评审意见指出其能够快速消除当前节点上的危险权限位，因此具有较高的有效性 $E(\pi_1)$ ；但它主要处理“当前是否仍保留危险权限位”，缺乏对后续权限漂移的持续观测，且无法限制高风险宿主机路径再次暴露。基于这一反馈，策略优化智能体进一步吸收 π_2 与 π_3 中有效的控制措施，形成更完整的组合式候选。

评审后补入的关键措施

- 在运行时监控侧，补入针对 CNI 目录异常写入、权限变更和可疑执行行为的 Falco 规则；
- 在准入控制侧，补入针对高风险宿主机目录挂载的 Kyverno 审计策略，限制普通工作负载直接挂载 `/opt/cni/bin` 与 `/var/lib/calico`；
- 在优化结果中保留节点权限收敛动作，并补齐验证步骤与后续升级路径。

相应的补充配置示例如下：

```

- rule: Write under CNI bin directory
  desc: Detect unexpected writes or permission changes under /opt/cni/bin
  condition: open_write and fd.name startswith /opt/cni/bin
  output: "Unexpected write to CNI binaries (user=%user.name file=%fd.name
  ↪ proc=%proc.cmdline)"
  priority: WARNING

apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: restrict-cni-bin-hostpath

```

```
spec:
  validationFailureAction: Audit
  rules:
    - name: forbid-cni-bin-mount
      match:
        any:
          - resources:
              kinds: ["Pod"]
      exclude:
        any:
          - resources:
              namespaces: ["kube-system", "calico-system"]
```

经过优化后,可得到防护型最终候选 π_{prot}^* 与监控型最终候选 π_{mon}^* 。例如,前者可形式化写为

$$\pi_{\text{prot}}^* = \langle \text{prot}, (\text{权限收敛}, \text{路径约束}), (\text{DaemonSet 修复动作}, \text{Kyverno 审计策略}) \rangle \quad (3-11)$$

而后者则可表示为

$$\pi_{\text{mon}}^* = \langle \text{mon}, (\text{运行时观测}), (\text{Falco 检测规则}) \rangle. \quad (3-12)$$

步骤 4: 基于三维质量向量完成最终选择

在最终聚合阶段,系统分别在 Π_v^{prot} 与 Π_v^{mon} 内,对保留下来的候选对象计算三维质量向量

$$q(\pi) = (E(\pi), I(\pi), D(\pi)), \quad (3-13)$$

并基于共识排序得到各维度排序 $\bar{R}_E, \bar{R}_I, \bar{R}_D$, 再通过总体质量函数

$$Q(\pi) = \alpha_E \hat{S}_E(\pi) + \alpha_I \hat{S}_I(\pi) + \alpha_D \hat{S}_D(\pi) \quad (3-14)$$

完成同类型候选的最终比较。在本文默认的有效性优先设置下, π_{prot}^* 往往优于仅包含单次权限修复的 π_1 , 因为它在保持较高有效性的同时,通过路径约束补足了暴露面控制;而 π_{mon}^* 则作为与防护型策略互补的监控结果,保留对节点侧异常行为的持续观测能力。

因此,对 CVE-2024-33522 的最终交付结果并不是一句泛化的“尽快升级版本”,也不是单一的临时修补动作,而是围绕漏洞实例 v 、由安全上下文 c 支撑、经由候选生成 Π_v 、评审优化与质量聚合后得到的一组可复核、可部署且便于运维执行的策略对象。这一过程也直观说明了本章方法如何把形式化定义、智能体协同流程与具体安全工具配置衔接起来。

3.4 小结

本章围绕云原生权限提升漏洞的防控需求，给出了从知识组织、候选生成到多评审聚合的完整方法链条。具体而言，本文统一定义了漏洞实例、安全上下文、策略对象与评价过程，构建了面向开源应用漏洞的可追溯知识库，并基于多智能体协同完成策略生成、评审与优化，最终通过三维评价与多源排名聚合模型实现对候选策略的可审计选择。上述设计为下一章的系统设计与实现以及后续实验验证奠定了方法基础。

4 面向云原生权限提升漏洞的智能化防控系统设计与实现

本章在第三章方法基础上实现 KPEAgent 原型系统。系统不将检索、推理和反馈视为彼此割裂的功能点，而是围绕第三章提出的“安全上下文构建、结构化漏洞分析、候选策略生成、评审反馈优化与质量排序确认”这一方法链条进行工程化落地，形成覆盖漏洞知识管理、会话驱动生成、专家评审和策略交付的完整闭环。下文从需求分析、总体设计和详细设计三个层面展开说明。

4.1 需求分析

4.1.1 用户角色与用例分析

系统围绕安全工程师与专家评审员两类角色组织核心用例，见图 4-1。安全工程师负责选择漏洞实例、发起分析会话、查看证据与候选策略，并完成部署与验证；专家评审员负责对同类型候选策略进行三维质量评审、排序确认与反馈修订。两类角色共同服务于第三章所定义的候选策略集合 Π_v 的生成、优化与最终交付。

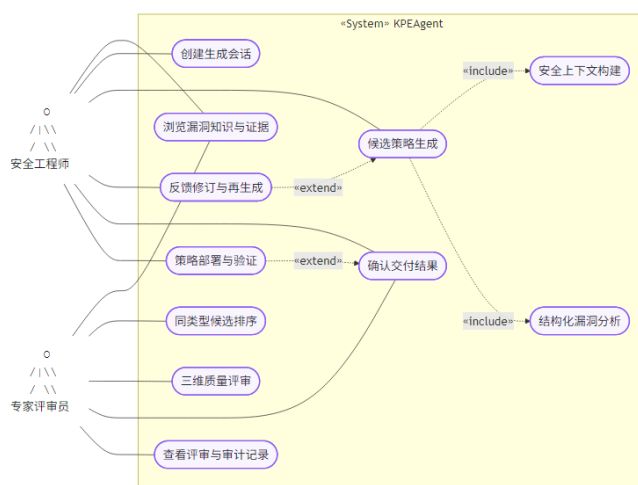


图 4-1: 系统用户角色与核心用例 (UML 用例图)

Fig. 4-1: System user roles and core use cases (UML use case diagram)

4.1.2 功能需求

结合两类角色的职责以及第三章的方法链条，系统需要具备以下五项核心能力：

- (1) **漏洞知识与证据管理**：导入、检索与维护 CVE 条目及其关联证据，为后续构建安全上下文 c 提供可追溯材料。
- (2) **安全上下文构建与候选策略生成**：围绕漏洞实例组织候选证据、形成结构化分析，并输出同类型候选策略集合。
- (3) **评审反馈驱动优化**：接收专家评审意见，对候选策略进行修订、补全与再生成，以降低噪声并提升可部署性。
- (4) **三维质量评审与排序确认**：围绕有效性、无干扰性和可部署性对同类型候选进行排序比较，并形成可审计的确认结果。
- (5) **策略交付与部署验证**：将通过确认的候选策略下发至目标环境，并记录执行状态、验证结果与审计信息。

4.2 总体设计

4.2.1 系统架构设计

系统采用分层架构，自顶向下分为界面层、服务层、智能引擎层和数据层，见图 4-2。该分层并不是对方法步骤的简单堆叠，而是将第三章中的知识组织、分析生成、评审优化与结果交付拆解为具有明确职责边界的实现单元：

- (1) **界面层**：基于 Vue3 构建单页应用（SPA），采用 Pinia 管理应用级状态，并集成 ECharts 可视化组件，负责 CVE 详情展示、策略交互与评审结果呈现。
- (2) **服务层**：采用 Flask 实现 REST API 网关，负责请求路由、访问控制与接口编排；同时通过任务调度器（Scheduler）管理耗时较长的会话任务，并协调知识检索、候选策略生成、评审提交与部署验证等流程。
- (3) **智能引擎层**：封装系统核心分析能力，由 KPEAgent 引擎统一调度安全知识整理、漏洞分析、候选策略生成、评审反馈吸收与策略优化等处理单元，完成从漏洞实例到候选策略集合的迭代生成。
- (4) **数据层**：负责数据持久化与检索支撑，包括存放 CVE 元数据、证据片段、中间分析结果与候选策略对象的文件存储，以及支持安全上下文构建的向量索引库。

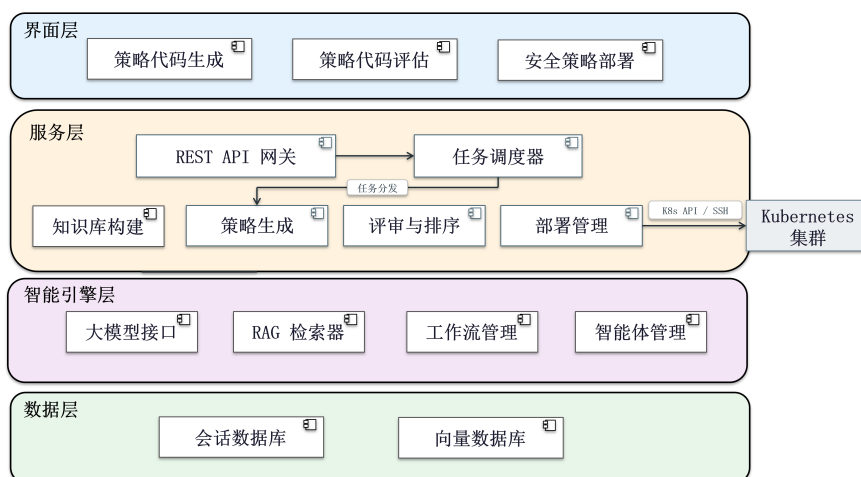


图 4-2: 系统总体架构与分层设计 (UML 组件图)

Fig. 4-2: Overall system architecture and layered design (UML component diagram)

4.2.2 数据模型设计

系统的数据持久化围绕四类核心实体展开, 见图 4-3。这些实体与第三章中的漏洞实例 v 、安全上下文 c 、候选策略集合 Π_v 及其评审结果一一对应; 图中按照 UML 标准标注了可见性修饰符 (– 表示私有属性, + 表示公有操作)、多重性约束与关联方向:

- (1) **CVE 实体**: 作为数据源头, 包含 `cve_id`、`summary`、`cvss_score` 和 `severity` 等基础属性, 并以枚举类型 `status` 标记当前处置状态 (未处理、分析中、已部署)。每个 CVE 与后续生成的会话呈一对多关系。
- (2) **会话实体 (Session)**: 表示一次完整的分析会话, 记录所选模型 `model_name`、提示词模板 `prompt_template` 与会话状态。会话既承载候选证据组织过程, 也承载候选策略生成与优化过程, 因此与策略呈一对多关系, 支持同一漏洞任务下的多版本迭代。
- (3) **策略实体 (Strategy)**: 记录单次生成的完整策略结果, 包括策略内容 `content`、中间分析快照 `reasoning_chain`、证据召回片段 `rag_snippets` 以及版本号 `version`, 用于支撑候选策略的来源追溯、版本对比和增量追踪。
- (4) **排序实体 (Ranking)**: 存储专家决策结果, 其核心属性包括专家评审员标识、决策时间戳、策略排序结果以及多维评分信息。该实体同时关联会话与策略, 用于刻画专家对特定候选策略集的即时评价。

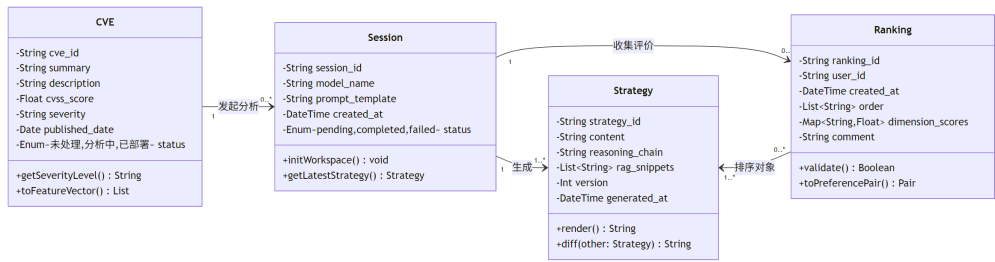


图 4-3: 系统核心实体类图（UML 类图）

Fig. 4-3: Core entity class diagram (UML class diagram)

4.2.3 数据流转与处置流程设计

为保证关键决策可追溯，系统构建了图 4-4 所示的闭环数据流。与第三章方法定义一致，该流程不再将系统简单理解为“检索后生成”的单段链路，而是拆解为安全上下文构建、漏洞分析与候选策略生成、评审反馈与策略优化、质量排序与结果确认，以及最终的审计归档五个环节。漏洞实例首先与候选证据共同进入上下文构建阶段，形成带来源标识的安全上下文；随后系统围绕该上下文输出候选策略集合及其中间分析结果；专家评审意见继续驱动候选修订与结果确认，最终将排序结果和审计记录写入持久层，形成可复核的处置证据链。

系统进一步采用 UML 状态图（图 4-5）描述 CVE 处置生命周期，刻画漏洞从导入到部署完成的状态演化过程。该生命周期包含未处理、策略生成与优化中、待评审、已批准、部署中和已部署六个顶层状态。其中，“策略生成与优化中”用于统摄安全上下文构建、结构化漏洞分析、候选策略生成与反馈修订等活动；提交评审、驳回修改和部署失败重试等迁移条件与第三章的方法规则保持一致。为进一步说明这一核心阶段，图 4-6 给出了候选策略生成与优化流程图，展示从候选证据整理到待排序候选集合输出的主要处理链路。

4.2.4 交互时序设计

候选策略生成涉及证据检索、结构化分析、候选组织与反馈修订等多轮处理，单次任务耗时较长，因此系统采用异步交互时序，见图 4-7 和图 4-8。安全工程师发起生成请求后，后端先返回会话标识，再在后台构建安全上下文并产出候选策略；前端通过轮询或 WebSocket 获取任务状态。图 4-7 展示了会话创建与安全上下文构建/候选策略生成两个阶段；图 4-8 展示评审反馈驱动优化与质量排序确认两个阶段。这种时序设计既降低了前端阻塞风险，也

保留了多轮人工干预入口。

4.3 详细设计与实现

本节按功能模块说明系统的交互流程与实现方式。各模块遵循前述分层架构与数据模型约束，并围绕 CVE 处置生命周期中的关键状态迁移展开。下文依次介绍知识库管理、策略生成与优化、专家评审与排序以及策略部署管理四个模块。

4.3.1 CVE 知识库管理

知识库管理模块承担漏洞条目的汇聚、检索、证据关联与状态维护，是安全上下文构建的入口环节。该模块对应服务层的 CVE 知识库组件，直接影响 CVE 实体的 `status` 字段及后续会话流程。如图 4-9 所示，系统主界面以列表视图展示已录入的 CVE 条目，核心字段包括 CVE 编号、漏洞摘要、CVSS 风险评分和当前处置状态。界面上方提供关键词检索与状态筛选功能，支持安全工程师依据漏洞编号、漏洞类型或关键描述快速定位目标记录；选择具体条目后，系统即可进入详情视图并发起分析会话，为后续候选证据检索和安全上下文整理提供统一入口。

4.3.2 策略生成与优化

策略生成与优化模块负责将漏洞证据转化为可交付的候选策略，并通过评审反馈闭环持续修订结果。该模块由 KPEAgent 核心引擎及其处理单元组成，并由服务层的策略生成组件和任务调度器协同驱动。系统以会话 (Session) 为基本单元，维护单个漏洞分析任务的上下文与执行状态，避免不同漏洞之间相互干扰。

如图 4-10 所示，会话初始化界面左侧给出历史会话列表，记录既有分析任务的漏洞对象与时间信息；右侧主工作区通过阶段条标识当前所处环节，即启动、证据检索、漏洞分析与候选生成。在初始化阶段，安全工程师可选择大语言模型后端及提示词模板，为后续安全上下文构建和候选策略生成设定运行配置。

证据检索阶段负责补充候选策略生成所需的外部知识上下文。如图 4-11 所示，系统先从漏洞实例中抽取关键语义特征，再在本地知识库中召回语义相关的 Top-K 文档片段。界面同时展示来源文档和相关度得分，便于安全工

工程师核验证据质量；筛选后的有效片段将进入后续的安全上下文构建过程，以增强生成结果与具体漏洞场景的对应关系。

结构化漏洞分析阶段负责展开漏洞机理分析与控制路径选择。如图 4-12 所示，左侧提供结构化分析模板，右侧展示攻击路径分解、权限边界识别以及 Kubernetes 控制点映射等中间步骤。点击具体步骤可进入详情视图（图 4-13），系统按背景识别、因果分析与结论推导组织结果，便于人工复核。

完成证据整理与漏洞分析后，系统进入候选策略生成阶段。如图 4-14 所示，生成结果预览界面以结构化代码视图展示候选策略对应的防护规则，如 NetworkPolicy 或 OPA Rego 脚本，并提供复制、导出与简要说明，便于安全工程师在部署前审查规则内容。对于同一漏洞实例，系统可保留多份监控型或防护型候选结果，供后续评审与排序使用。

针对复杂漏洞场景，系统进一步引入评审与优化闭环机制。专家评审员提出修改意见后，安全工程师可在优化面板发起再生成请求。如图 4-15 所示，界面并列展示原始版本与修订版本，并高亮变更位置，便于沿“候选生成—评审反馈—策略优化”的轨迹持续修正。

4.3.3 专家评审与排序

专家评审与排序模块负责将候选策略转化为可用于决策和模型对齐的偏好数据。该模块接收策略生成模块输出的同类型候选策略集合，并输出满足排序实体（Ranking）结构约束的结果。系统通过“三维质量评审+同类型排序确认”两个阶段引入专家知识，并为后续模型对齐提供偏好信号。

评审阶段侧重采集细粒度定性反馈。如图 4-16 所示，结构化评审详情页要求专家评审员围绕有效性、潜在干扰风险和实施前置条件等因素对候选策略进行逐项核查，并允许输入自由文本意见。这些反馈可作为策略优化与再生成的重要依据。

排序阶段侧重同类型多候选方案之间的相对比较与一致性判别。如图 4-17 所示，拖拽式排序主界面左侧列出同一 CVE 的多个候选策略，右侧设置有效性、可部署性与无干扰性等维度评价区，并提供必要的排序提示。相较于绝对打分，拖拽排序更有利于控制评分尺度漂移；维度评价区则为排序结果提供解释依据。系统以排序结果作为主信号，以维度判断和文本备注作为辅助信号，形成后续一致性分析与模型对齐所需的偏好样本。

为支持细粒度比较，系统允许专家评审员展开单个候选策略的详细信息。

如图 4-18 所示，专家评审员可在该界面对不同方案的规则逻辑、关键参数与实现细节进行逐项核对，重点关注策略对攻击路径与权限边界的覆盖范围、规则前置条件与约束是否充分且不过度收紧，以及执行代价与潜在业务影响是否可接受。这种呈现方式减少了因信息分散造成的判断偏差，并提高了排序决策的稳定性。

最终确认的排序结果通过提交界面（图 4-19）写入数据库，并与维度得分、时间戳及专家评审员标识一并存档，形成后续分析与模型对齐所需的偏好数据集。为保证数据质量，系统在提交前执行一致性校验，例如检查排序结果是否构成全序、是否存在缺失条目，并在持久化过程中绑定候选策略版本与会话上下文，避免版本不一致问题。

4.3.4 策略部署管理

策略部署管理模块负责将通过评审的候选策略下发至目标环境，并在 CVE 处置生命周期中完成从已批准到已部署的状态迁移。如图 4-20 所示，部署目标配置界面为 Kubernetes 集群和传统主机环境提供两类配置方式。对于 Kubernetes 集群，系统通过 Python Kubernetes SDK 与 API Server 建立连接，安全工程师需配置集群地址、访问令牌与命名空间；对于传统主机环境，系统支持基于 SSH 的部署方式，需要输入主机地址、端口和认证密钥。正式部署前，系统提供连通性测试。

部署完成后，系统通过部署管理面板统一展示跨集群、跨主机的策略实例。如图 4-21 所示，界面记录策略名称、目标对象、部署时间及当前状态等信息，并提供回滚与日志查看操作，便于开展部署结果核查、异常诊断与后续运维管理。

4.4 小结

本章完成了 KPEAgent 原型系统的设计与实现。需求分析部分明确了两类用户角色和五项功能需求；总体设计部分给出了分层架构、数据模型、数据流转和交互时序；详细设计部分说明了知识库管理、策略生成与优化、专家评审与排序以及策略部署管理四个模块的交互流程与实现方式。这些设计为下一章的实验验证提供了平台支撑。

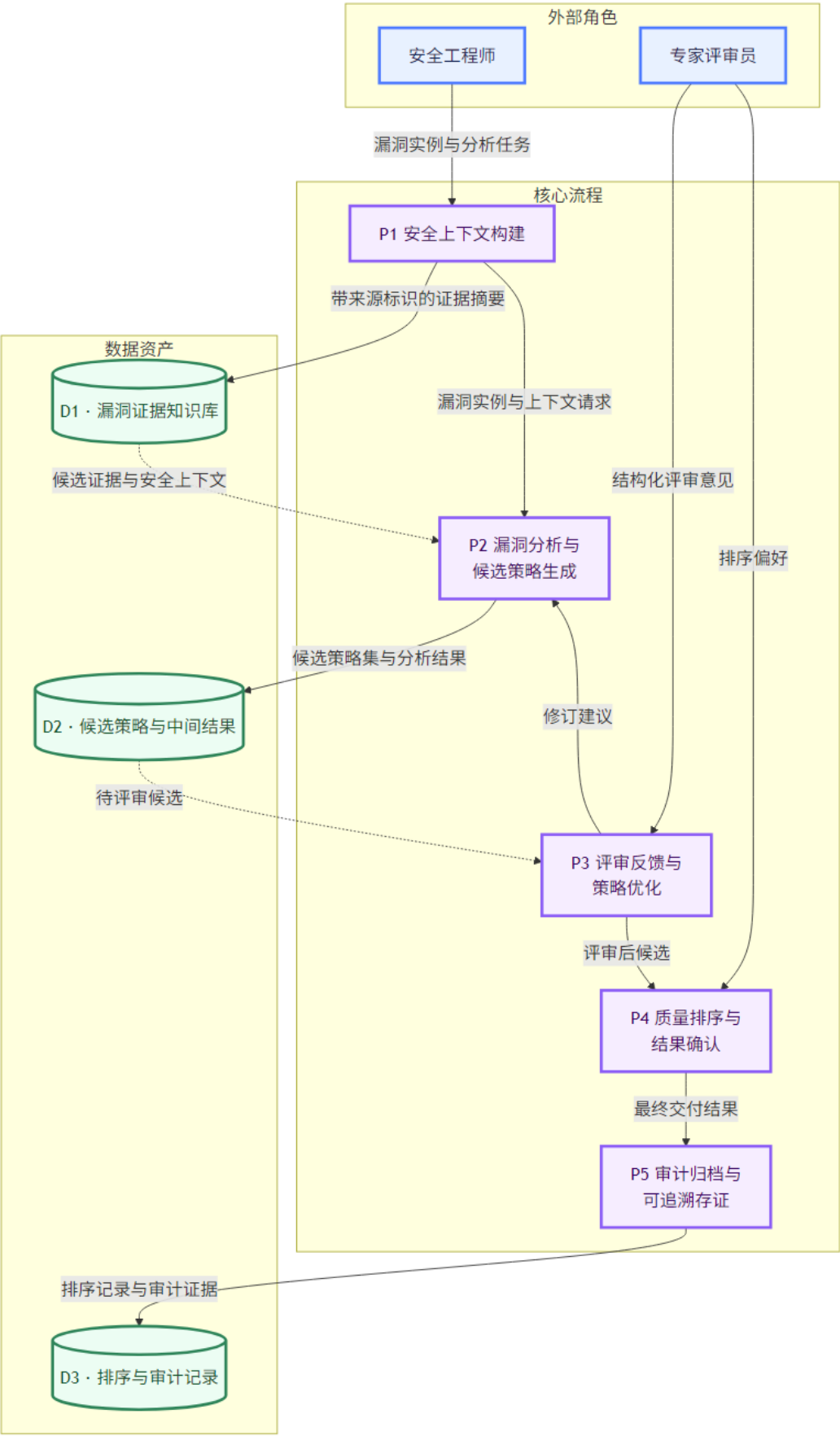


图 4-4: 防控闭环的数据流与审计点

Fig. 4-4: Data flow and audit points in the prevention-and-control loop

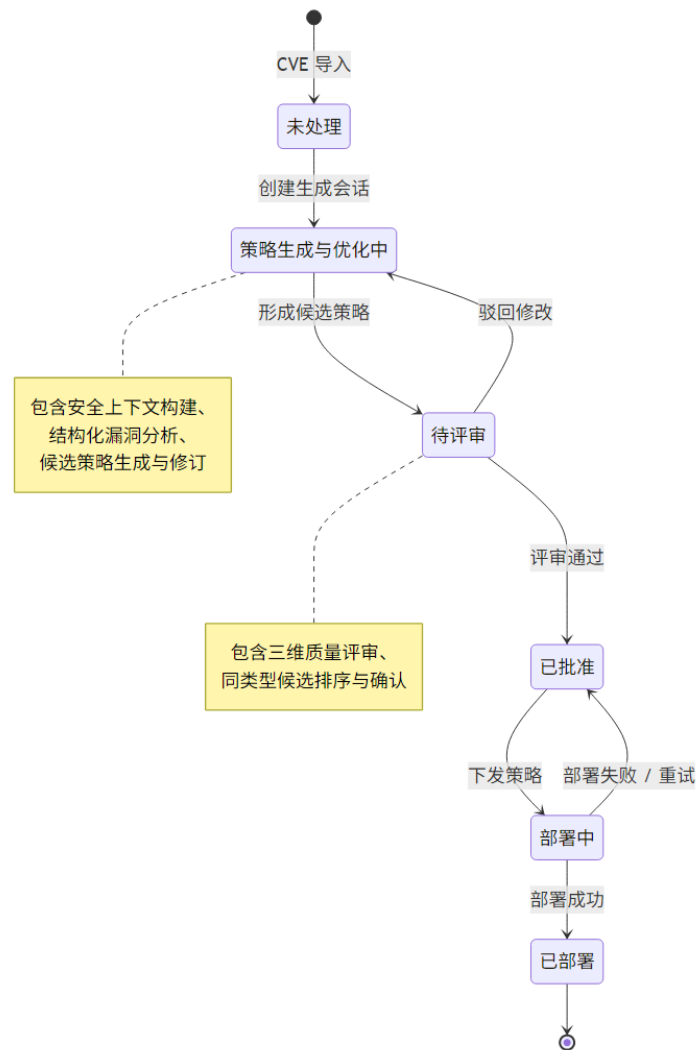


图 4-5: CVE 漏洞处置生命周期状态图 (UML 状态图)

Fig. 4-5: CVE vulnerability handling lifecycle state diagram (UML state diagram)

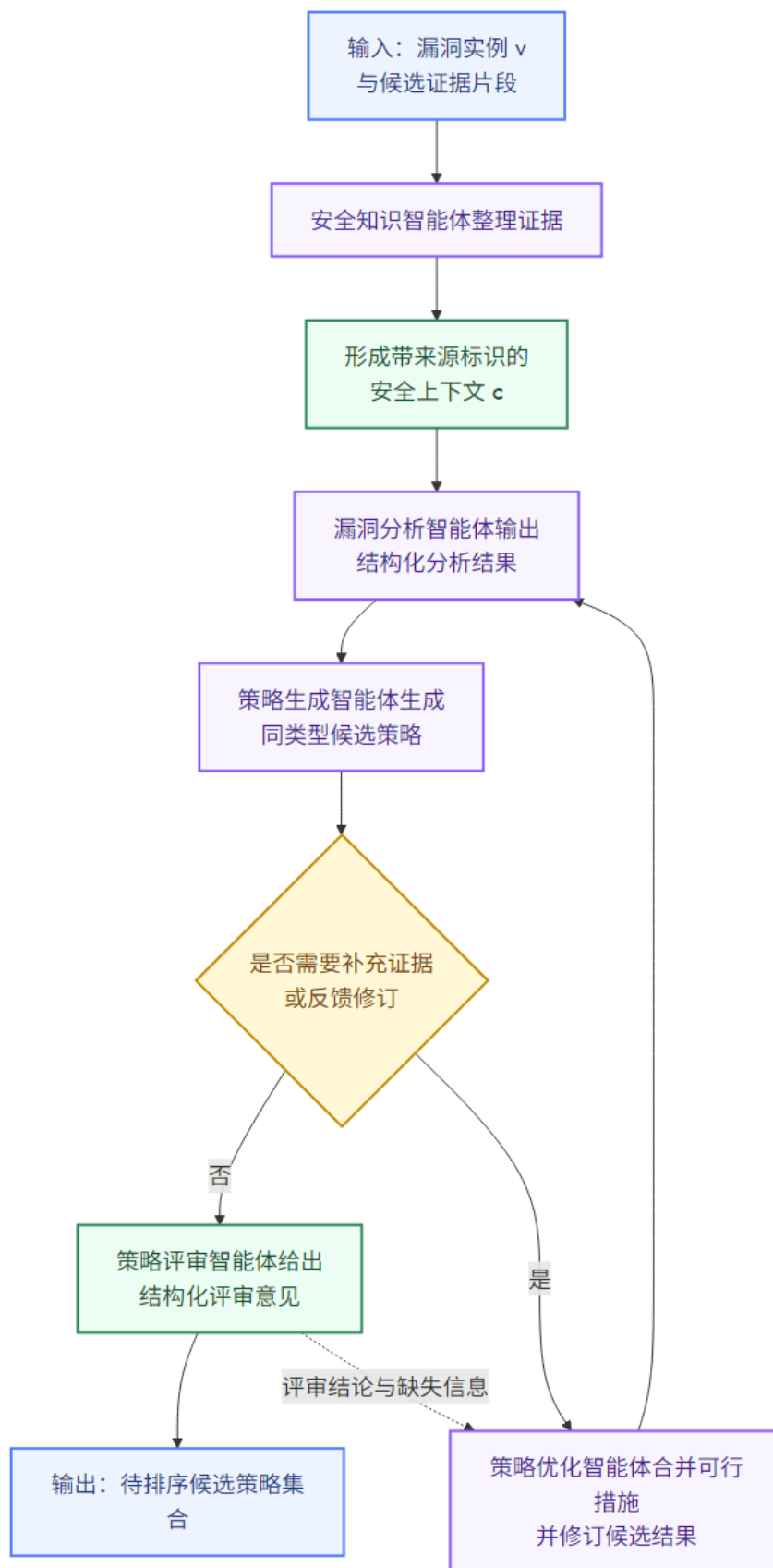


图 4-6: 候选策略生成与优化流程图

Fig. 4-6: Flowchart of candidate strategy generation and refinement

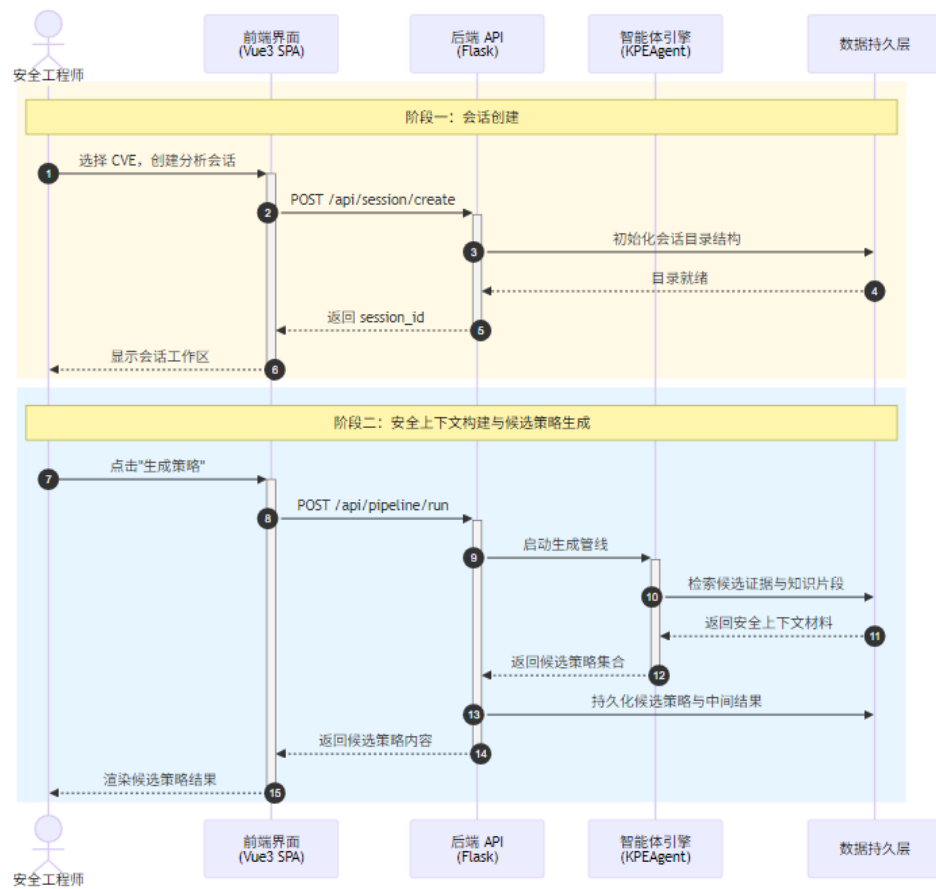


图 4-7: 会话创建与候选策略生成的交互时序 (UML 时序图)

Fig. 4-7: Interaction sequence of session creation and candidate strategy generation (UML sequence diagram)

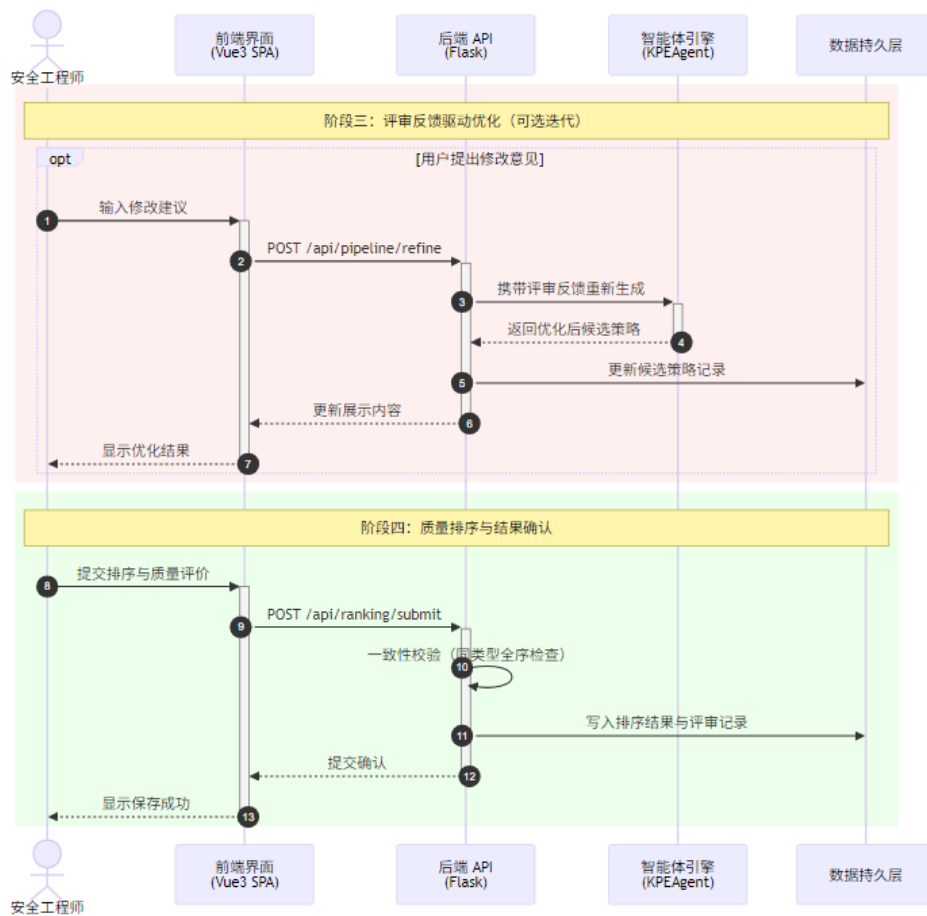


图 4-8: 评审反馈优化与质量排序确认的交互时序 (UML 时序图)

Fig. 4-8: Interaction sequence of feedback-driven refinement and quality ranking confirmation (UML sequence diagram)

[illegible]

图 4-9: CVE 知识库查看与管理界面

Fig. 4-9: Interface for CVE knowledge base browsing and management

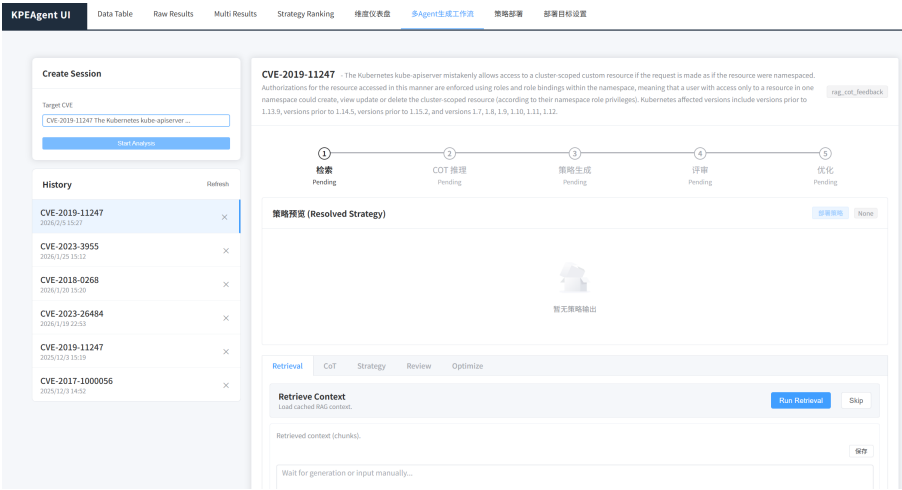


图 4-10: 策略生成会话启动界面
Fig. 4-10: Startup interface of a strategy generation session

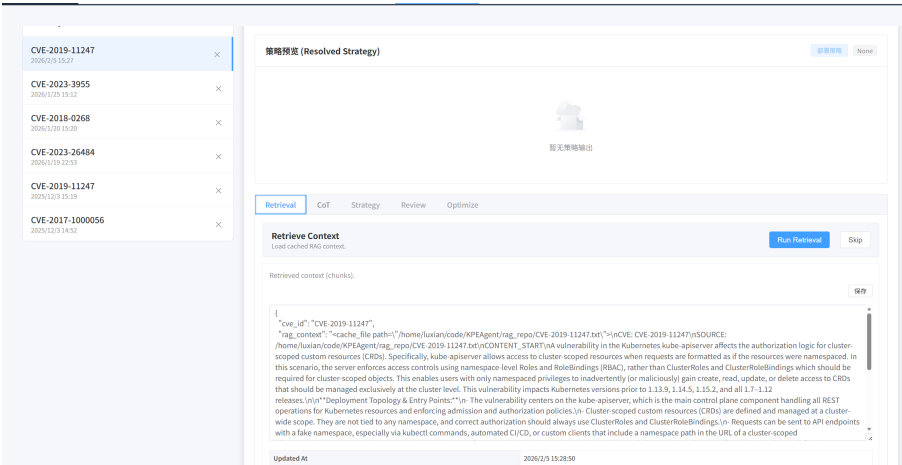


图 4-11: 安全上下文构建阶段的检索结果展示（RAG 实现）
Fig. 4-11: Display of retrieval results in the security-context construction stage (RAG-based implementation)

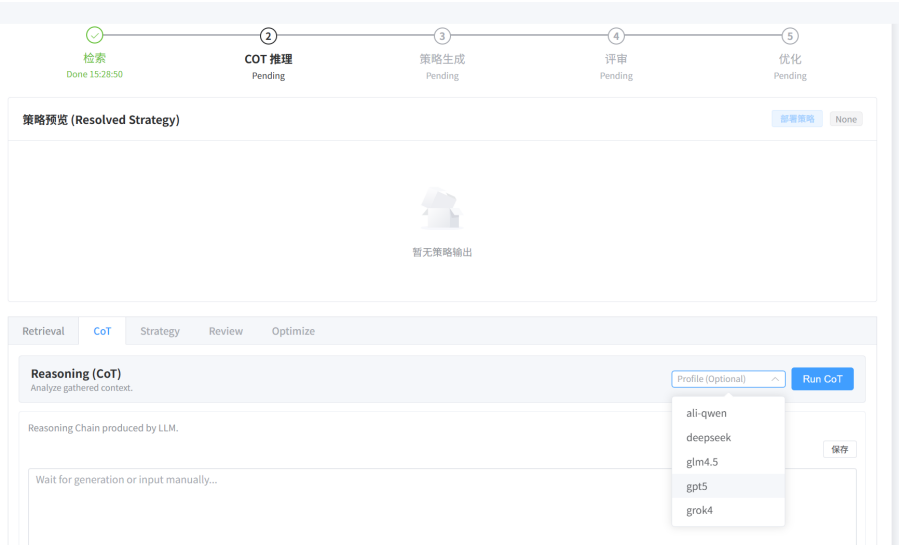


图 4-12: 结构化漏洞分析展示界面（CoT 实现）

Fig. 4-12: Interface for structured vulnerability analysis (CoT-based implementation)



图 4-13: 结构化漏洞分析详情（CoT 实现）

Fig. 4-13: Details of structured vulnerability analysis (CoT-based implementation)

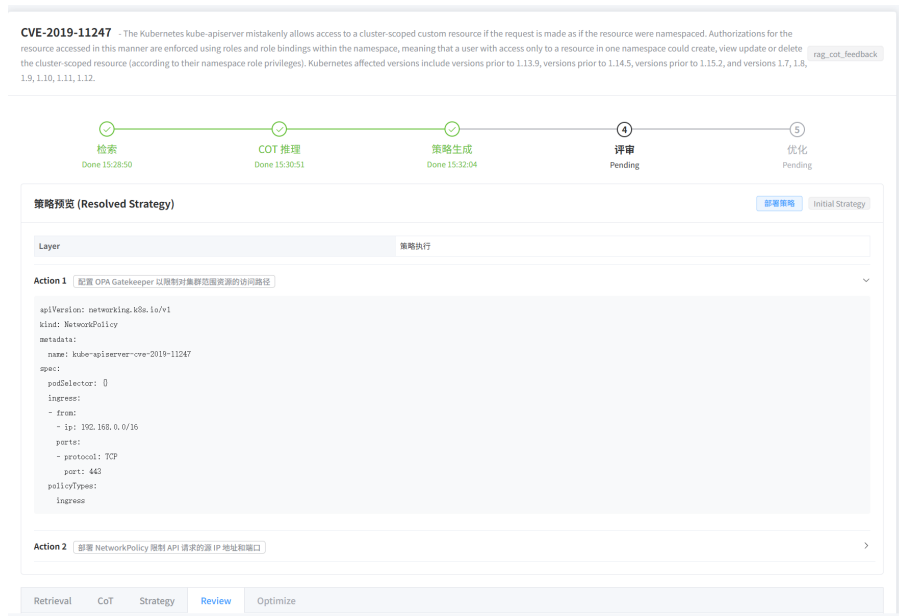


图 4-14: 策略生成初步输出
Fig. 4-14: Initial output of strategy generation

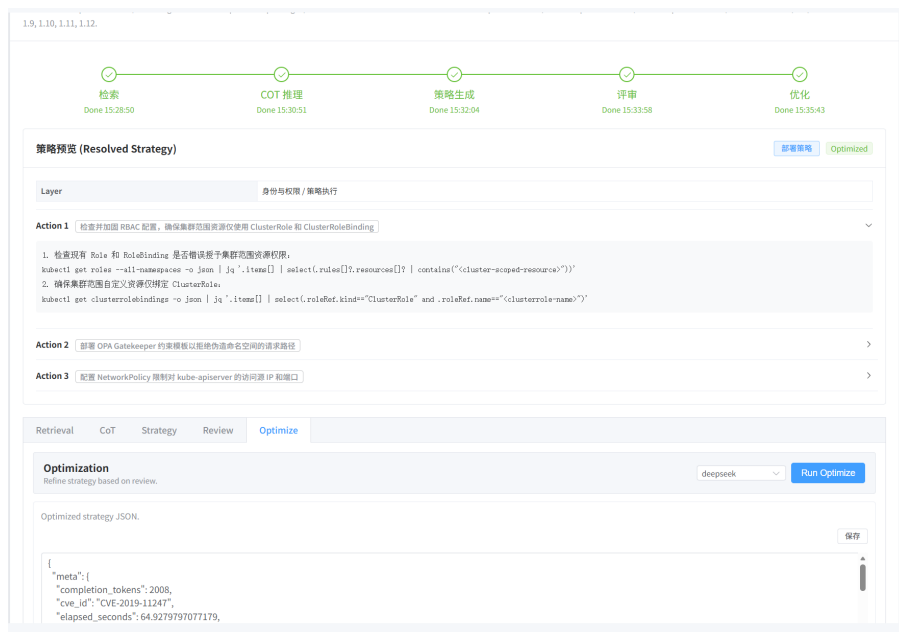


图 4-15: 策略优化后的最终输出
Fig. 4-15: Final output after strategy optimization



图 4-16: 评审阶段的结构化输出示例

Fig. 4-16: Example of structured output in the review stage

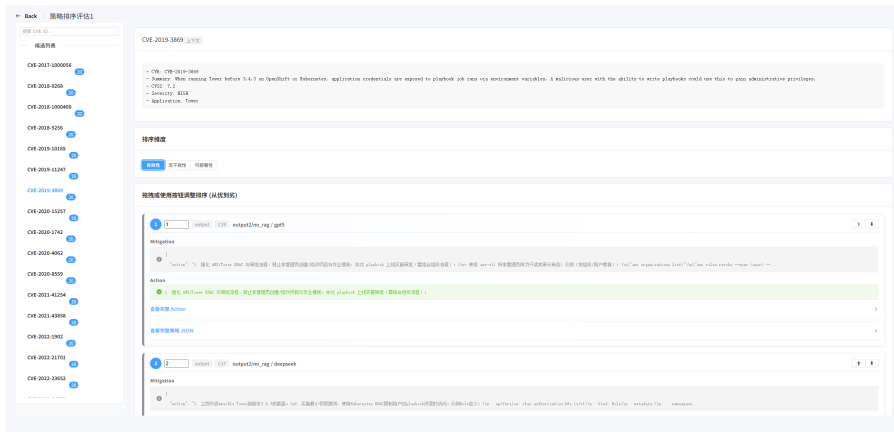


图 4-17: 策略排序主界面

Fig. 4-17: Main interface for strategy ranking

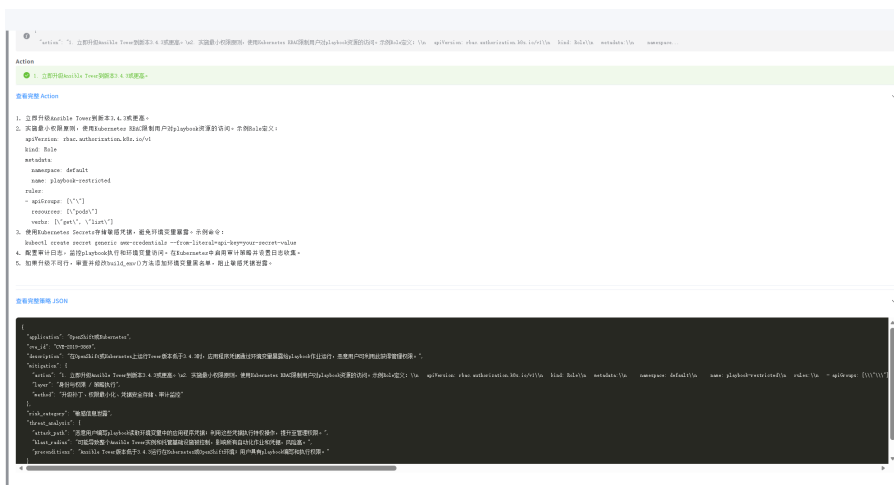


图 4-18: 策略详情查看界面

Fig. 4-18: Interface for viewing strategy details

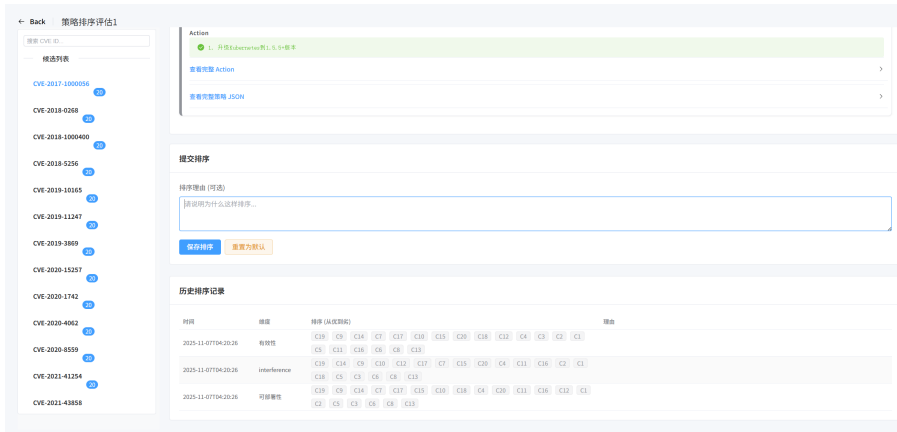


图 4-19: 排序结果提交界面
Fig. 4-19: Interface for submitting ranking results

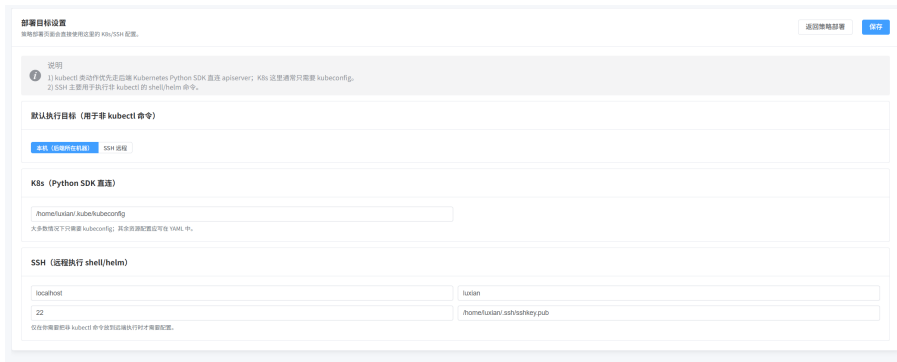


图 4-20: 部署目标配置界面
Fig. 4-20: Interface for configuring deployment targets

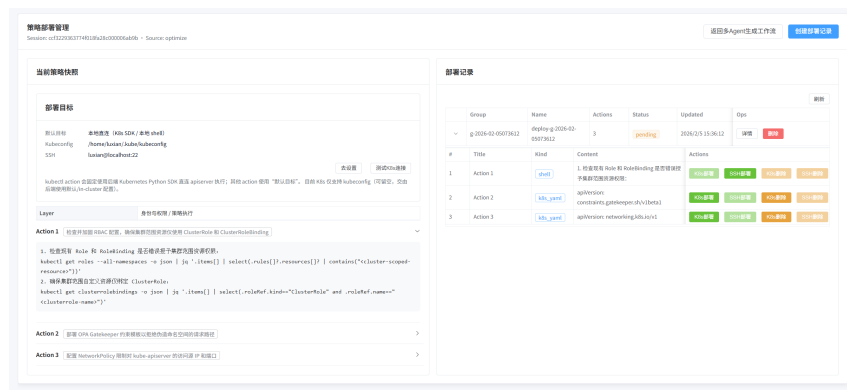


图 4-21: 策略部署管理界面
Fig. 4-21: Interface for strategy deployment management

5 实验研究

本章围绕第三章的方法与第四章的系统实现开展实验验证，从消融实验、模型能力、评审可信度、静态工具对比和资源消耗五个方面展开分析。

5.1 研究问题

本章围绕以下问题展开实验：

- (1) **三个核心模块在三维质量指标上的贡献如何？**需要验证检索增强、思维链推理与反馈优化分别对策略有效性、无干扰性与可部署性的影响，以及这种影响在不同模型下是否稳定。
- (2) **不同大语言模型在策略生成任务中的能力差异如何？**需要评估不同模型在三维质量指标和输出稳定性上的表现差异。
- (3) **基于大语言模型的自动化评审方法是否具备足够的可信度？**需要从评审模型间的一致性以及与人工评审的对照结果两个角度进行验证。
- (4) **相较于现有静态分析工具，本文方法能否提供更具针对性的处置方案？**需要通过主流静态扫描工具的对比，验证本文方法在漏洞处置精度上的优势。
- (5) **不同模型与方法配置在资源消耗上呈现怎样的成本效益特征？**需要量化不同配置下的词元消耗、时间开销和经济成本，为实际部署中的选型提供依据。

5.2 实验对象

本章实验以第三章构建的 55 个 Kubernetes 生态权限提升漏洞实例为对象。该数据集源自 CNCF Landscape 收录的云原生开源应用，经关键词过滤、严重度筛选与人工复核三重过滤获得，覆盖访问控制缺陷、冗余权限、命令注入、身份验证绕过等多种成因类型。每个漏洞实例关联完整的 CVE 公告、CWE 标注、修复提交以及受影响组件的部署清单，构成策略生成所需的输入上下文。

在生成模型方面，本章选取五个具有代表性的大语言模型参与策略生成与评审。表5-1列出各模型的基本信息。五个模型在架构设计、训练数据和参数规模上存在差异，可用于考察方法在不同模型条件下的适用性。

表 5-1: 实验采用的大语言模型
Tab. 5-1: Large language models used in experiments

模型代号	开发机构	发布日期	参数规模
Qwen3-Max	阿里云	2025 年 9 月	1000B+
DeepSeek-V3.1	深度求索	2025 年 9 月	670B
GPT-5	OpenAI	2025 年 6 月	未公开
Grok 4	xAI	2025 年 7 月	未公开
GLM-4.5	智谱 AI	2025 年 7 月	355B

5.3 实验设计

5.3.1 度量指标

本文采用三维评价体系衡量生成策略的质量，对应第三章定义的质量向量 $q(\pi) = (E(\pi), I(\pi), D(\pi))$ 。三个维度的含义如下：

- **有效性** $E(\pi)$ ：衡量策略对漏洞攻击路径的封堵程度，包括是否覆盖关键利用前提、是否消除或限制权限提升的可行路径。得分越高，表示策略对漏洞的处置越充分。
- **无干扰性** $I(\pi)$ ：衡量策略对现有业务功能的影响程度，包括是否引入不必要的访问限制、是否破坏正常服务间通信或资源调度。得分越高，表示策略对业务的副作用越小。
- **可部署性** $D(\pi)$ ：衡量策略在目标环境中落地的难易程度，包括配置片段是否语法正确且可直接应用、是否依赖额外的组件安装或集群变更。得分越高，表示策略越容易部署。

由于大语言模型在绝对评分任务中难以维持稳定的内部尺度^[72]，本文采用**基于排序的相对评价**作为基础信号。具体而言，评审模型对同一批候选策略在每个维度上给出从优到劣的严格排序，而非直接赋予分值。随后依据第三章的多源排名聚合模型（定义 5-6），将各评审模型的排序通过秩函数 $r_{\delta}^{(h)}$ 映射为归一化得分 $s_{\delta}^{(h)}(\pi)$ ，再以等权方式计算共识得分 $\hat{S}_{\delta}(\pi)$ ，据此形成各维度的共识排序。

此外，为衡量评审模型之间的排序一致性，本文引入 **Kendall's W** 协调系数与 **Spearman** 等级相关系数作为辅助度量。为评估资源消耗，记录每次调用的输入与输出词元数以及运行时间，并按模型和方法配置分别聚合统计。

5.3.2 基线方法

为与传统静态安全工具进行比较，本文选取四种代表性工具或文献作为基线。在得到风险合规报告后，再人工评估其对策略生成的实际帮助，包括能否定位与漏洞路径相关的高风险配置，以及是否给出可直接落地的修正建议。

表 5-2: 静态扫描工具
Tab. 5-2: Static scanning tools

工具	来源
kube-linter	https://github.com/stackrox/kube-linter
Kubescape	https://github.com/kubescape/kubescape
KubeSec	https://kubesecc.io/
SLI-Kube	Rahman 等 ^[13]

5.3.3 实验过程

实验围绕候选策略生成、自动评审和人工对照三个环节展开。首先，在统一的提示词模板、输入组织方式和输出结构约束下，分别使用五个大语言模型对 55 个漏洞实例生成候选策略集合 Π_v 。为评估各关键模块的作用，设置三项消融配置：移除检索增强（w/o rag）、移除思维链推理（w/o cot）和移除反馈优化（w/o feedback），并与完整工作流（all）进行对比。除被移除模块外，其余实验条件均保持一致，包括提示词框架、输出格式约束及后处理规则，以保证不同配置之间的可比性。

在评审阶段，将上述五个模型同时作为评审模型，对每个漏洞实例对应的候选策略在三个评价维度上分别进行独立排序。为降低评审偏差，评审前对候选策略进行随机化与匿名化处理，打乱展示顺序并隐去来源信息，以减少顺序效应和模型偏好对结果的影响。随后，对各评审模型给出的排序结果进行等权聚合，得到该漏洞实例上的共识排序，作为自动评审的最终结果。

此外，随机抽取 20 个 CVE 样本进行人工独立评审，以检验自动评审结果与人工判断的一致性。人工评审与自动评审采用相同的评价维度、排序规则和候选展示方式，从而保证两者具有可比性。通过自动评审与人工对照相结合的方式，可以进一步验证评审结果的可靠性，并为分析自动评审方法的

有效性提供依据。

5.4 结果分析与讨论

本节依次从消融实验、生成模型能力、评审可信度、静态工具对比和资源消耗五个方面展开分析。

5.4.1 消融实验与 workflow 模块贡献

消融实验表明，三个模块均能提升策略质量，其中反馈优化的影响最为显著。本节比较完整 workflow `all` 与三种消融配置 `w/o rag`、`w/o cot`、`w/o feedback`，评价维度为有效性 E 、无干扰性 I 与可部署性 D 。55 个漏洞样本先由五个生成模型产出候选策略，再交由五个评审模型独立排序，最后经等权聚合得到各维度共识得分。图 5-1 与表 5-3 给出排名对比和具体数值。

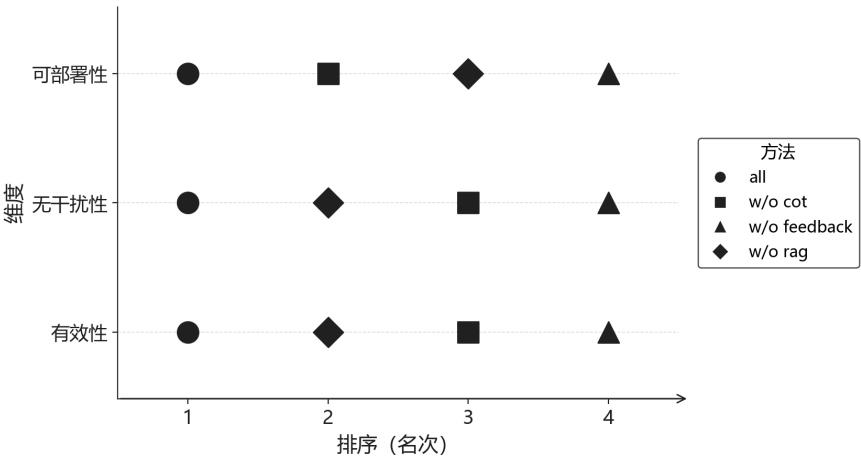


图 5-1: 方法配置在三个维度上的排名对比

Fig. 5-1: Comparison of method configurations across three dimensions

表 5-3: 消融实验方法配置得分

Tab. 5-3: Average scores in ablation study

方法配置	有效性 (E)	无干扰 性 (I)	可部署 性 (D)
all	0.581	0.431	0.403
w/o rag	0.476	0.370	0.321
w/o cot	0.234	0.340	0.398
w/o feedback	0.121	0.269	0.288

完整 workflow 在三个维度上均排名第一。反馈优化的影响最大：移除该模

块后，有效性从 0.581 降至 0.121，降幅达 79%，无干扰性与可部署性也同步出现最大下降。这说明多轮评审反馈能够持续纠偏，使候选方案逐步逼近可用状态。需要说明的是，这一降幅反映的是反馈优化对策略整体可交付质量的综合贡献，而不只是内容有效性的下降，因为去除反馈后，策略在格式规范性与结构完整性上也更弱。

检索增强主要提升策略与具体漏洞场景的贴合度。移除后有效性降至 0.476，降幅约 18%，无干扰性和可部署性也分别下降约 14% 与 20%。这说明外部证据能够帮助模型围绕真实攻击面组织策略。思维链推理则呈现较强的权衡特征：移除后有效性下降至 0.234，但可部署性几乎不变（ $D = 0.398$ 对 $D = 0.403$ ），说明分步推理在提升分析深度的同时，也可能增加部署复杂度。在资源受限的场景下，反馈优化和检索增强应优先保留。从单个生成模型看，结论方向保持一致，只是敏感性有所不同：GPT-5 与 GLM-4.5 在移除反馈后下降更明显，DeepSeek 与 Qwen3-Max 则更依赖检索增强。

5.4.2 生成模型能力评估

本节将讨论对象由 workflow 模块转向生成模型本身，分别从三维质量指标和输出稳定性两个方面展开。

五个生成模型在三个维度上的表现呈现明显的风格分化，而非统一的优劣排序。图 5-2 展示了各模型在有效性 E 、无干扰性 I 与可部署性 D 三个维度上的排名分布。

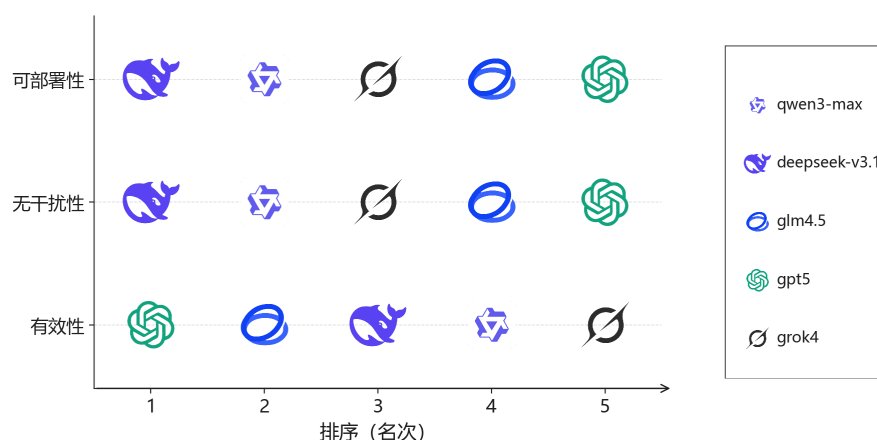


图 5-2: 生成模型在三个维度上的排名对比

Fig. 5-2: Comparison of generation model rankings across three dimensions

表 5-4 显示，五个生成模型的得分并未沿同一方向变化，而是分散在不同

目标之间。GPT-5 在有效性维度上明显领先，得分达到 $E = 0.814$ ，但无干扰性和可部署性分别只有 0.175 和 0.215，说明它更容易给出约束较强、处置力度较大的方案。DeepSeek 则呈现出相反倾向，其无干扰性和可部署性分别达到 $I = 0.497$ 和 $D = 0.605$ ，均为五个模型中最高，但在有效性上并不占优，因此更倾向于生成改动范围较小、部署代价较低的策略。

其余几个模型位于这两种倾向之间。Qwen3-Max 在有效性上的得分不高，但在无干扰性和可部署性上保持了相对均衡；GLM-4.5 三个维度分布接近，没有出现特别突出的单项；Grok 4 的三项得分都不算高，但分布较为平缓。整体来看，这组结果体现的更像风格差异，而非简单的线性优劣排序：有的模型更强调封堵强度，有的模型更强调部署成本与业务扰动。

表 5-4: 生成模型三维质量得分
Tab. 5-4: Quality scores of generation models

生成模型	有效性 (E)	无干扰 性 (I)	可部署 性 (D)
GPT-5	0.814	0.175	0.215
GLM-4.5	0.327	0.324	0.336
DeepSeek	0.284	0.497	0.605
Qwen3-Max	0.193	0.425	0.477
Grok 4	0.147	0.342	0.346

在输出稳定性方面，本文统计了各模型在生成阶段触发重试的次数。重试主要对应三类失败情形：结构化输出中的 JSON 结构错误，生成内容的语法错误，以及内容违规。为便于跨模型对比，本文将计数按固定分母 504 归一化为失误率。表5-5给出五个生成模型的失误率统计。

表 5-5: 生成模型输出稳定性统计
Tab. 5-5: Output stability statistics of generation models

错误类型	DeepSeek 3.1	GLM -4.5	GPT -5	Grok 4	Qwen 3
JSON 格式错误	0.99%	4.56%	0.00%	0.99%	2.58%
语法错误	0.60%	1.98%	0.00%	0.00%	0.99%
内容违规	0.00%	1.98%	0.00%	0.00%	0.00%
合计	1.59%	8.53%	0.00%	0.99%	3.57%

各模型的输出稳定性差异显著。GPT-5 在三类错误上均未触发重试，输

出最为稳定；Grok 4 和 DeepSeek 的总失误差也较低（0.99% 和 1.59%）。GLM-4.5 的总失误差最高（8.53%），主要集中在 JSON 格式错误，且是唯一出现内容违规的模型；Qwen3-Max 的总失误差为 3.57%。

综合三维得分与输出稳定性，五个模型大致呈现两类生成倾向。GPT-5 更偏向高有效性、高封堵强度，在有效性上优势明显且输出零失误，但可部署性和无干扰性较低；DeepSeek 更偏向低干扰、易部署，在无干扰性和可部署性上领先，适合对业务连续性要求较高的场景。Qwen3-Max、GLM-4.5 和 Grok 4 则处于两者之间，提供了不同程度的折中。

5.4.3 评审方法可信度评估

前两节分别讨论了 workflow 差异和模型差异，而这些比较都建立在评审结果足够可靠的前提之上。本节从内部一致性和人工对照两个角度验证这一前提。

本小节使用度量指标中定义的 Kendall's W 协调系数与 Spearman 等级相关系数，考察五个评审模型在三个维度上的排序一致性。表 5-6 给出量化结果，图 5-3 至图 5-3b 展示分布细节。

表 5-6: 不同维度下大语言模型评审一致性统计

Tab. 5-6: Consistency statistics of large-language-model judges across dimensions

维度	Kendall's W	Spearman 相关系数			
		均值	中位数	最小值	最大值
有效性	0.955	0.944	0.944	0.904	0.991
无干扰性	0.846	0.808	0.839	0.585	0.932
可部署性	0.687	0.609	0.689	0.238	0.947

评审模型在有效性维度上的一致性最高，在可部署性维度上的分歧最大。表 5-6 显示，有效性维度的 Kendall's W 达 0.955，Spearman 均值为 0.944，说明不同评审模型对漏洞封堵充分性的判断标准较为接近。无干扰性居中（ $W = 0.846$ ），可部署性最低（ $W = 0.687$ ）。这说明可部署性需要同时权衡工程依赖、部署成本和运维复杂度，评审模型更容易在该维度上出现偏差。

图 5-3 从 Kendall's W 和两两 Spearman 分布两个角度汇总了一致性统计。从整体看，等权聚合后的排序在三个维度上均未出现明显失稳。

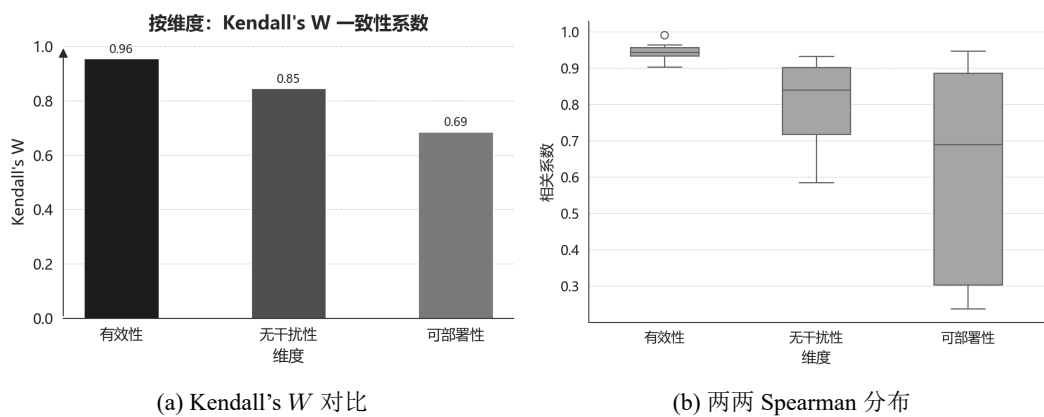


图 5-3: 大语言模型评审一致性统计

Fig. 5-3: Consistency statistics of large-language-model judges

跨维度对比进一步印证了可部署性是评审分歧的主要来源。图5-4从评审模型角度展示了各维度的分歧分布：可部署性上的评审分歧最明显，有效性上排序接近一致，无干扰性介于两者之间。因此，多模型聚合策略在可部署性维度上的纠偏意义最大。

人工对照实验进一步验证了自动评审与专家判断之间的总体一致性。本文随机抽取 20 个 CVE 样本，由安全专家进行独立评审，并通过拟合评审模型权重使自动评审的加权结果逼近人工评审分数，从而学习各评审模型在不同维度上的贡献权重。表5-7 给出学习到的权重分布。

表 5-7: 学习到的评审者权重分布

Tab. 5-7: Learned weight distribution of large-language-model judges

评审者	有效性	无干扰性	可部署性
qwen3-max	0.42	0.38	0.20
deepseek	0.18	0.15	0.20
glm4.5	0.15	0.22	0.20
gpt5	0.20	0.18	0.20
grok4	0.05	0.07	0.20

Qwen3-Max 在有效性和无干扰性维度上与人工判断最为接近。由表5-7可知，该模型在这两个维度上的拟合权重分别为 0.42 和 0.38，远高于其他评审模型。可部署性维度则呈完全均匀分布（各模型权重均为 0.20），表明在该维度上现有评审信号不足以区分各模型的优劣，这与前文一致性分析中可部署性分歧最大的结论相互呼应。

图5-5分别展示了人工评审与自动评审的雷达图对比。两者在生成模型和方法配置的总体趋势上保持一致，差别主要出现在可部署性上——人工评审

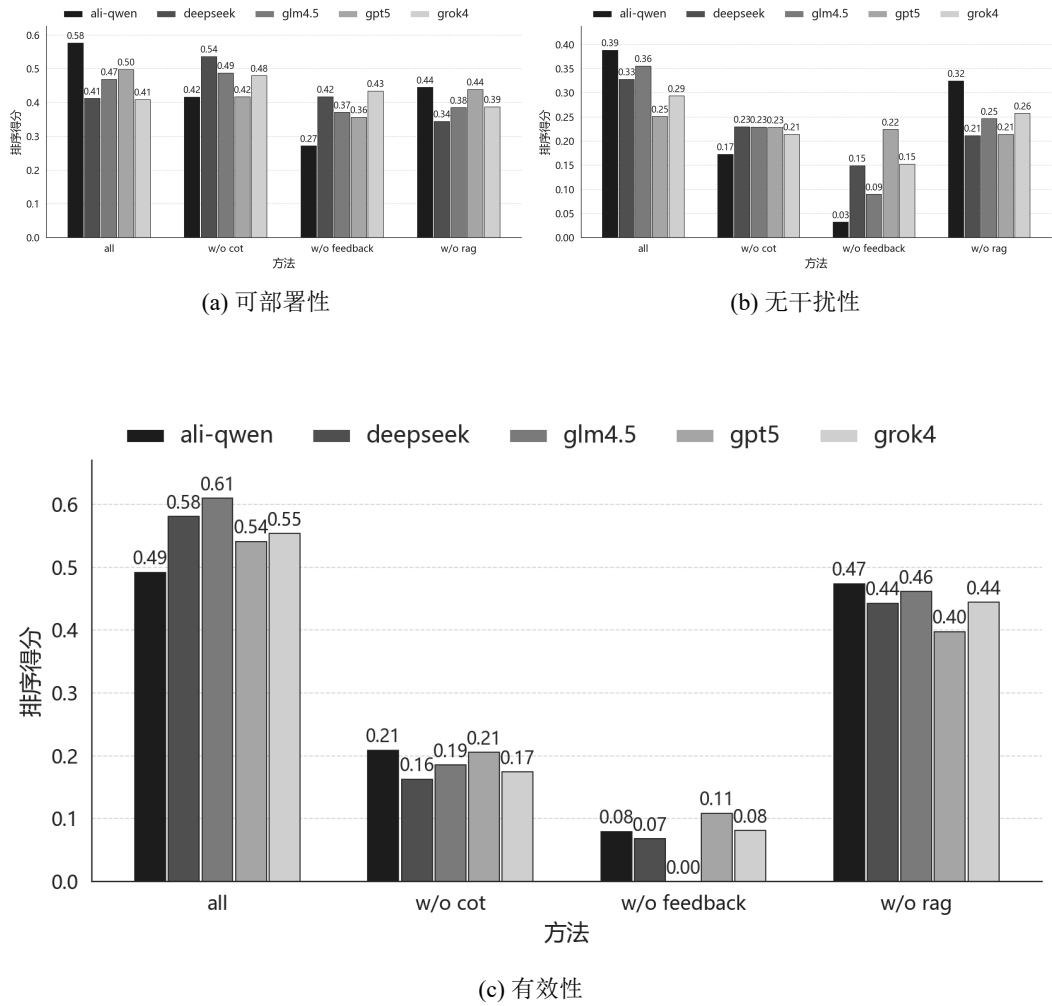


图 5-4: 评审模型单维度评分对比

Fig. 5-4: Comparison of single-dimension scores among judge models

对部署难度的区分更敏感。

多模型聚合评审方案在有效性和无干扰性维度上具有较高可信度，足以支撑前文的主要比较结论。局限主要体现在两点：其一，可部署性维度的评审一致性最低（ $W = 0.687$ ），拟合权重趋于均匀，与人工结果的对应关系也较弱；其二，本实验中生成模型与评审模型均来自同一组大语言模型，可能引入自偏好偏差（self-preference bias），即评审模型对自身生成风格的候选给予偏高排序^[72,73]。本文通过匿名化与随机化处理缓解了这一效应，多模型等权聚合也进一步稀释了单一评审的偏差；但若彻底消除这类影响，仍需引入与生成模型解耦的独立评审来源。当前人工对照仅覆盖 20 个样本，后续仍需扩大样本规模以提高验证精度。

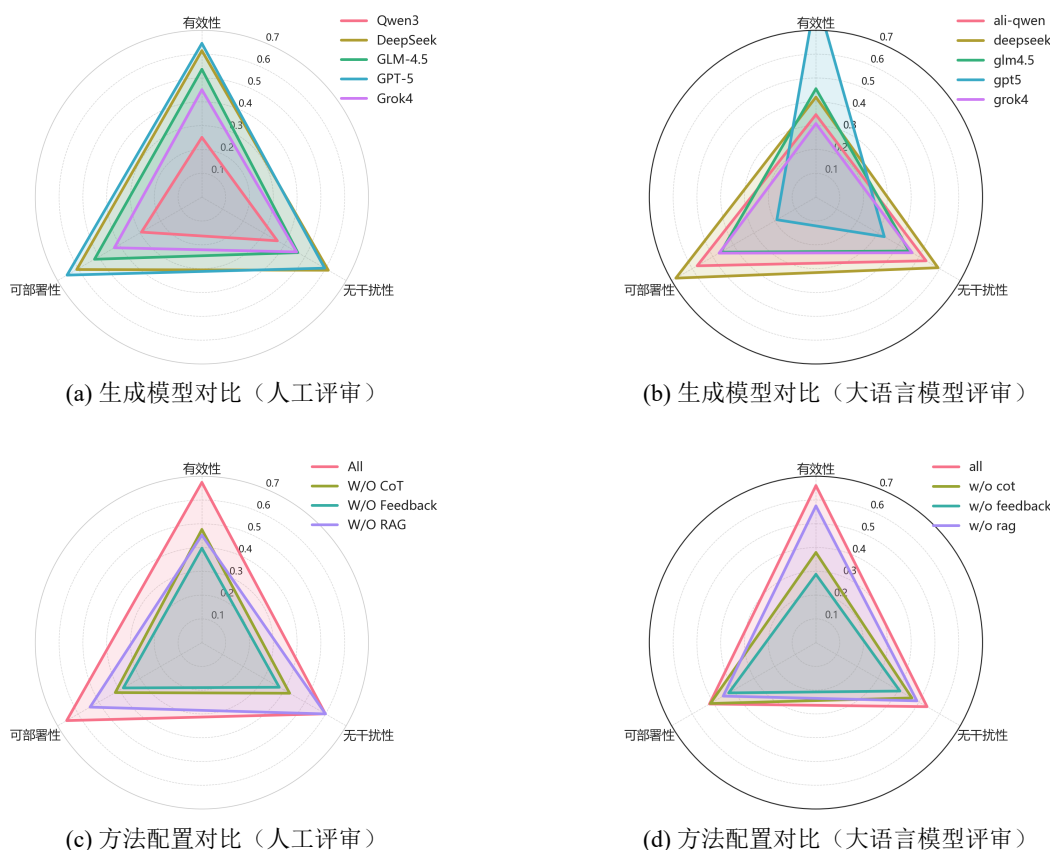


图 5-5: 人工评审与大语言模型评审的三维度雷达图对比

Fig. 5-5: Comparison of three-dimensional radar charts between human and large-language-model evaluation

5.4.4 静态扫描工具的处置价值

本节引入外部基线技术，考察传统静态扫描能否为临时漏洞处置提供有效支持。本文选取 Kubescape、KubeSec、kube-linter 和 SLI-Kube 作为对照工具，在相关 Kubernetes 部署清单上进行扫描，并将报告条目按统一风险类型归并统计，结果见表5-8。四类工具的报告主要集中在配置基线与通用加固项，如资源配置缺失、身份与密钥暴露、网络隔离和非 Root 运行。这类结果对日常治理和合规检查仍有价值，但与本文收集的 55 个权限提升漏洞逐一对应时，能够直接指向漏洞触发前提的情况并不多。只有 CVE-2020-4062 与 CVE-2024-31989 在报告中出现了较明确的关联线索，而且都落在网络隔离项上，分别由 Kubescape 与 SLI-Kube 命中。其余样本的关键攻击路径通常依赖具体组件行为、权限边界或运行时条件，超出了静态规则直接覆盖的范围。就补丁窗口期的漏洞防控而言，静态扫描更适合作为背景性治理工具，而本文方法更适合提供面向具体漏洞场景的处置建议。

表 5-8: 静态扫描工具检测结果
Tab. 5-8: Detection results of static scanning tools

风险类型	关联一级分类	Kubescape	KubeSec	kube-linter	SLI-Kube
资源配置缺失	配置缺陷	110	47	73	0
非 Root 运行	配置缺陷	54	53	51	0
根文件系统非只读	配置缺陷	39	34	59	0
网络隔离	配置缺陷	132	0	0	15
特权与权限提升	配置缺陷	39	0	12	0
主机级暴露	配置缺陷	30	0	9	3
身份与密钥	敏感泄露	192	58	0	2
系统调用隔离	配置缺陷	0	58	0	1
探针配置	配置缺陷	92	0	15	0

5.4.5 资源消耗与效率评估

本节进一步考察生成式流程的资源代价。本文分别统计词元消耗与运行时间，因为前者直接对应调用成本，后者决定在漏洞响应窗口内能否及时完成一轮生成与评审。表5-9 给出了不同方法配置与生成模型下的平均词元消耗及对应成本：

从表5-9可以看出，反馈环节是额外词元开销的主要来源。完整工作流平均成本为 12.99 元，去掉反馈后降至 9.88 元，降幅约 24%。这一差异主要来自反馈轮次中模型需要重新阅读上下文并生成修订策略，因而同时推高输入与输出词元。模型间差异也较为明显：GPT-5 在完整配置下单次处置成本达到 50.40 元，远高于其他模型，其中大部分成本来自输出端（47.11 元），说明其倾向于生成更长、更详尽的策略文本。相比之下，GLM-4.5 的总成本仅为 0.49 元，Qwen3-Max 为 1.50 元，均处于较低水平。从输入输出比例看，DeepSeek 的输出词元量始终高于输入量，说明其在策略生成阶段展开更充分；Qwen3-Max 和 Grok 4 的输出则更紧凑。去掉 RAG 模块后，平均成本为 11.10 元，介于完整工作流和去反馈之间，说明检索增强引入的额外上下文也带来了可观开销。

时间开销呈现出相近结构，见表5-10。完整工作流的方法均值为 146.39 秒，去掉反馈后降至 86.98 秒，降幅约 41%，是所有消融配置中时间缩减最明

表 5-9: 词元消耗与经济成本统计
Tab. 5-9: Token usage and economic cost statistics

方法	模型	输入 Token	输出 Token	输入成本 (CNY)	输出成本 (CNY)	总成本 (CNY)	平均成本 (CNY)
All	qwen3-max	228842	92360	0.57	0.92	1.50	12.99
	deepseek	213174	360923	0.51	2.60	3.11	
	glm4.5	217216	158493	0.17	0.32	0.49	
	gpt5	376812	672931	3.30	47.11	50.40	
	grok4	246549	108188	2.96	6.49	9.45	
W/O CoT	qwen3-max	147128	44759	0.37	0.45	0.82	11.29
	deepseek	143191	310209	0.34	2.23	2.58	
	glm4.5	138905	141923	0.11	0.28	0.39	
	gpt5	201623	646417	1.76	45.25	47.01	
	grok4	165321	61481	1.98	3.69	5.67	
W/O Feedback	qwen3-max	129241	44841	0.32	0.45	0.77	9.88
	deepseek	120138	280609	0.29	2.02	2.31	
	glm4.5	122551	155095	0.10	0.31	0.41	
	gpt5	178449	554871	1.56	38.84	40.40	
	grok4	145538	62736	1.75	3.76	5.51	
W/O RAG	qwen3-max	184820	46291	0.46	0.46	0.92	11.10
	deepseek	170295	318002	0.41	2.29	2.70	
	glm4.5	170290	116108	0.14	0.23	0.37	
	gpt5	338456	603583	2.96	42.25	45.21	
	grok4	203726	64326	2.44	3.86	6.30	

显的。这说明反馈迭代不仅增加词元消耗，也是时间成本的主要瓶颈。GPT-5 在完整配置下单次处置平均耗时 354.48 秒，约 6 分钟，与其较大的输出规模基本一致；去掉反馈后缩短至 228.50 秒，但仍明显高于其他模型。Qwen3-Max 和 Grok 4 在所有配置下均维持在 15~39 秒的区间，具有较强的实时响应能力。DeepSeek 的耗时主要来自 API 响应延迟，而非词元量本身，其完整配置下 225.20 秒的耗时与约 57 万词元总量之间并不呈线性关系。去掉 CoT 后，大部分模型的耗时仅有小幅变化（135.64 对 146.39 秒），说明思维链推理在时间维度上的额外开销有限。

不同模型并不存在对所有场景都占优的统一选择，实际选型仍需结合响应窗口与预算约束权衡。若预算更敏感，Qwen3-Max 或 Grok 4 更合适，两者在完整工作流下的单次成本分别为 1.50 元和 9.45 元，耗时也均不超过 40 秒，能够在漏洞公开后的较短时间内完成一轮策略生成与评审。若更强调封堵效果，GPT-5 虽然代价最高，但在有效性上表现最强，适合高危漏洞的重点处置。DeepSeek 则以较高时间成本换取在可部署性和无干扰性上的优势，更适用于业务连续性要求较高的生产环境。若需进一步压缩成本，最直接的方式

表 5-10: 时间成本统计
Tab. 5-10: Time cost statistics

方法配置	模型	平均耗时 (秒)	方法均值 (秒)
W/O RAG	qwen3-max	27.57	132.48
	deepseek	212.84	
	glm4.5	55.34	
	gpt5	340.42	
	grok4	26.23	
W/O CoT	qwen3-max	26.22	135.64
	deepseek	219.34	
	glm4.5	74.22	
	gpt5	328.47	
	grok4	29.93	
W/O Feedback	qwen3-max	15.94	86.98
	deepseek	140.94	
	glm4.5	35.51	
	gpt5	228.50	
	grok4	14.00	
All	qwen3-max	38.93	146.39
	deepseek	225.20	
	glm4.5	75.78	
	gpt5	354.48	
	grok4	37.55	

仍是去掉反馈模块，但也必须接受策略质量下降。

5.5 小结

本章实验表明，RAG、CoT 与 Feedback 三个模块均能提升策略质量，不同大语言模型在三维指标上也呈现出明显差异。多模型共识排序与人工评审保持了较好一致性；与静态扫描工具相比，本文方法在处置精度上更具优势；资源消耗分析则为不同预算与时限约束下的模型选型提供了参考。总体而言，该方法已经表现出较好的可行性，但其在具体漏洞场景中的阻断效果、业务影响与部署可行性，仍需结合真实案例继续验证。

6 案例分析

在上一章总体实验基础上，本章选取 7 个典型 CVE 案例，进一步考察本文方法在真实云原生权限提升漏洞场景中的表现。每个案例包括**漏洞复现**、**策略生成**与**三维质量验证**三个阶段：在可控环境中重现攻击路径并确认风险；基于本文方法生成**监控策略**与**防护策略**候选；再从有效性 E 、无干扰性 I 与可部署性 D 三个维度评估策略是否能够阻断攻击且具备工程可行性。

6.1 案例分析实验设计

6.1.1 CVE 案例选择标准

为保证案例具有代表性且可复现，本文按以下标准筛选 CVE：

风险等级与影响范围：优先选择 CVSS 评分 ≥ 5.0 且在 Kubernetes 生态中影响较广的漏洞，如核心组件漏洞或高使用率第三方应用漏洞。

成因类型覆盖：确保所选案例覆盖本文两级成因分类框架中的主要类别，包括配置缺陷、安全逻辑缺陷、敏感信息泄露与代码注入等，以检验方法的适用范围。

可复现性：漏洞具备公开的攻击 PoC 或较完整的技术细节，可在隔离环境中安全复现；同时受影响版本与修复版本明确，便于对比验证。

防控价值：漏洞场景具有明确的临时防控需求，如补丁尚未发布、业务无法立即升级或需要多层防御，因此更适合评估智能生成策略的实际价值。

6.1.2 实验环境与复现流程

环境搭建。所有漏洞复现与策略验证均在隔离的 Kubernetes 测试集群中进行。

为便于复核与复现，实验环境的软硬件配置与安全组件版本统一汇总于表 6-2。

漏洞复现流程。对每个 CVE，本文按以下步骤进行复现：

1. **基线环境准备：**部署受影响版本的目标组件，并创建模拟的业务负载与用户身份（如低权限 ServiceAccount）；
2. **攻击执行：**根据公开 PoC 或技术分析，模拟攻击者执行权限提升攻击，记录攻击步骤与观测到的异常行为；

表 6-1: 本章所选 CVE 案例一览
Tab. 6-1: Overview of selected CVE cases in this chapter

序号	CVE 编号	影响组件	一级分类	二级分类
1	CVE-2022-24768	Argo CD	安全逻辑缺陷	访问控制缺陷
2	CVE-2025-2241	Hive	敏感信息泄露	API 端口暴露
3	CVE-2023-30512	CubeFS CSI	配置缺陷	冗余权限
4	CVE-2020-4062	ConjurOSS	配置缺陷	网络隔离不足
5	CVE-2022-24877	Flux	代码注入	路径遍历
6	CVE-2022-29171	Sourcegraph	代码注入	命令注入
7	CVE-2023-22482	Argo CD	安全逻辑缺陷	身份验证绕过

3. **影响确认：**验证攻击是否成功实现权限提升（如获取集群管理员权限、访问受限资源、执行特权操作等）；

策略生成与验证。漏洞复现成功后，使用本文方法生成安全策略并进行三维评价：

1. **安全上下文构建（检索增强（RAG））：**将 CVE 公告、受影响组件文档、攻击日志与社区讨论输入向量知识库，检索与漏洞相关的证据；
2. **策略生成（思维链推理（CoT）与反馈优化（Feedback））：**基于检索增强上下文与漏洞分析结果，生成候选策略并进行评审反馈优化；
3. **策略部署与复测：**在测试集群中部署生成的安全策略（如 NetworkPolicy、RBAC 规则、Gatekeeper 策略等），重新执行攻击以验证策略是否有效阻断；
4. **三维质量评价：**
 - **有效性（E）：**策略能否成功阻断原攻击路径，是否存在绕过可能；
 - **无干扰性（I）：**策略对正常业务流量的影响，如误报率、性能开销与运维复杂度；
 - **可部署性（D）：**策略配置的完整性与可执行性，是否包含清晰的验证与回滚方案。

表 6-2: 实验环境软硬件配置与工具版本汇总
Tab. 6-2: Summary of hardware and software environment configuration and tool versions

项目	配置
计算资源	本地虚拟机集群，96 核 CPU，96GB 内存，200GB 存储
集群工具与版本	Minikube v1.18.1；Kubernetes v1.18.3
节点规模与规格	1 个控制节点与 2 个工作节点；每节点 4 核 CPU/8GB 内存
CNI/网络策略	Calico v3.28.2, Kubernetes-Network-Policy v1.5.5
准入控制	OPA/Gatekeeper v3.14.0；Kyverno v1.11.0
运行时安全监控	Falco v0.38.1
包管理工具	Helm v3

6.2 案例一：CVE-2022-24768 内部权限问题

6.2.1 漏洞详解与复现

漏洞背景

Argo CD 是 Kubernetes 生态中广泛使用的声明式 GitOps 持续交付工具。其典型部署形态是由 Argo CD 控制平面以相对高权限身份持续对齐 Git 仓库中的期望状态，即对集群资源拥有创建、更新与删除能力。该架构提升交付一致性的同时，也引入了新的安全风险：一旦普通用户能够影响 Argo CD 的控制决策，便可能借助控制平面权限放大自身权限边界。

CVE 描述：CVE-2022-24768 为 Argo CD 的不当访问控制（Improper Access Control）漏洞。所有未修复版本（自 1.0.0 起）均存在风险，0.5.x 与 0.8.x 系列中也包含该问题的受限版本。攻击者需要满足以下前置条件之一：(i) 对某个 Application 的源 Git/Helm 仓库具备 push 权限；或 (ii) 在 Argo CD 中对该 Application 具备 sync 与 override 权限。满足条件后，攻击者可通过构造/修改仓库内容或应用配置，诱导 Argo CD 以其控制平面权限执行超出攻击者原始 RBAC 边界的操作，进而可能获得管理员权限。官方已在 Argo CD 2.1.14、2.2.8、2.3.2 中发布补丁。缓解措施可降低风险但不能替代升级。

主要成因分类：安全逻辑缺陷 / 访问控制缺陷（已授权用户的权限放大）。
攻击链条描述：攻击链可概括为：

1. 合法身份阶段：攻击者以普通 Argo CD 用户身份登录；
2. 配置影响阶段：凭借对 Application 的 sync+override 或对源码仓库的

push 权限，注入恶意资源清单；

3. **控制平面滥用阶段：**Argo CD 按 GitOps 同步逻辑将恶意资源应用到集群；
4. **权限提升阶段：**借助被创建的高权限 RBAC 绑定获得集群级管理员能力。

漏洞复现

本节给出可复现的实验流程：构造同时包含正常资源与恶意 RBAC 绑定的 Git 仓库，为低权用户配置 sync 与 override 权限，触发 Argo CD 同步，并在集群中创建管理员级绑定以实现权限提升。

环境配置：Kubernetes 测试集群可选 kind 或 minikube。受影响组件为 Argo CD v2.3.1，补丁版本为 2.3.2、2.2.8、2.1.14。测试负载使用仓库 https://github.com/Ivanbeethoven/bad_repo，包含 app.yaml 与 hack-admin.yaml。

攻击步骤：

本文复现包括以下关键环节：部署漏洞版本并创建宽松 AppProject；授予低权用户 sync+override；创建指向恶意仓库的 Application 并触发同步；最后验证权限变化。为避免正文过长，关键命令与清单见附录B。

攻击结果：综合上述验证可观察到权限边界放大。低权用户 bob 原本仅具备对特定项目范围内应用的 sync/override 能力，但通过影响 Git 期望状态或同步动作，间接诱导控制平面在集群中创建高权限 RBAC 绑定，最终表现为在 --as bob 条件下 kubectl auth can-i '*' '*' 返回 yes。

```
r2cn-ubuntu-01 → ~ kubectl describe clusterrolebinding bob-admin-binding
Name:          bob-admin-binding
Labels:        app.kubernetes.io/instance=bob-app
Annotations:   <none>
Role:
  Kind: ClusterRole
  Name: cluster-admin
Subjects:
  Kind  Name  Namespace
----  ---  -
User   bob
```

图 6-1: 漏洞复现实验结果示意（验证权限变化）

Fig. 6-1: Experimental result of vulnerability reproduction showing privilege change verification

6.2.2 策略部署

本节给出可落地的缓解策略。该策略采用分层防护思路，重点通过限制配置可达性、限制配置表达能力与约束授权来源来阻断攻击路径。

```
# 检查 bob 对 namespaces 的权限
kubectl auth can-i create namespaces --as bob
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  annotations:
    kubernetes.io/last-applied-configuration: |
      {"apiVersion":"rbac.authorization.k8s.io/v1","kind":"ClusterRoleBinding","metadata":{"annotations":{},"labels":{"app.kubernetes.io/instance":"bob-app"},"name":"bob-admin-binding"},"roleRef":{"apiGroup":"rbac.authorization.k8s.io","kind":"ClusterRole","name":"cluster-admin"},"subjects":[{"apiGroup":"rbac.authorization.k8s.io","kind":"User","name":"bob"}]}
creationTimestamp: "2025-12-17T12:02:56Z"
labels:
  app.kubernetes.io/instance: bob-app
  name: bob-admin-binding
resourceVersion: "529144"
uid: 45b0bb4b-605e-4b00-a4d7-9fe650c3171e
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: cluster-admin
subjects:
- apiGroup: rbac.authorization.k8s.io
  kind: User
  name: bob
yes
yes
yes
yes
yes
Warning: resource 'namespaces' is not namespace scoped
```

图 6-2: 漏洞复现实验结果示意（验证是否有命令空间级别权限）

Fig. 6-2: Experimental result of vulnerability reproduction showing namespace-level privilege verification

策略概述：该方案从身份与权限、策略执行、准入控制以及审计治理四个层面分层实施。方法侧以 Kubernetes RBAC 收敛与 Argo CD RBAC 加固为基础，在 AppProject 与权限策略的创建更新环节施加准入约束，并通过身份组映射白名单、审计告警、基线与回滚机制降低权限漂移风险。相关能力可由 Kubernetes RBAC、Argo CD RBAC、Gatekeeper 或 Kyverno 以及审计与告警系统实现。

策略配置与部署步骤：为便于复用与自动化，可按先审计后强制的节奏分阶段落地。集群侧通过 Kubernetes RBAC 收敛 AppProject 的创建、更新与删除权限，应用侧通过 Argo CD RBAC 将默认权限设置为只读，并显式隔离 projects、repositories、clusters、exec 等高风险能力。策略侧在 AppProject 创建更新环节校验策略表达，限制通配符放权并约束权限授予范围。治理侧将关键配置纳入 Git 基线并接入审计告警，降低权限漂移风险。

为避免正文堆叠大量清单，Kubernetes RBAC、Argo CD RBAC、Gatekeeper/Kyverno 准入策略的完整配置与应用命令见附录B。

6.2.3 三因素分析

有效性：该策略在攻击链的多个关键点形成阻断。Kubernetes RBAC 对 AppProject 写入进行限制，可降低放开 sourceRepos、destinations 与白名单导致的爆炸半径。Argo CD RBAC 默认只读并隔离高危资源写权限，使低权用户难以获得可用于放大控制面的关键操作集合。准入控制对策略表达施加硬约束，防止隐蔽通配符放权与权限漂移。身份治理约束授权来源，可降低组注入或错

误映射造成的隐式权限提升。在复现实验中，若上述策略生效，则攻击者即使拥有 `sync` 权限，也难以通过修改 `AppProject`、仓库或策略来诱导 Argo CD 创建集群级高权限绑定，攻击路径受到明显限制。

无干扰性：该策略以最小必要变更为原则，影响主要集中在管理面与配置流程。对业务同步的直接影响总体可控，策略保留项目内正常的 `sync` 能力，仅收紧 `override`、高危资源写权限与高风险策略表达。准入控制建议先审计后强制，以降低误拦截对业务的冲击。额外运维开销主要来自 RBAC 精细化与准入规则维护，但可通过模板化与 GitOps 基线降低长期成本。

可部署性：该策略依赖的能力均为云原生常用组件，具有较强可落地性。Kubernetes RBAC 与 Argo CD RBAC 均为原生能力，配置可版本化并可审计。Gatekeeper 或 Kyverno 属于成熟的准入控制方案，可按项目逐步推广，并与审计治理配合使用。

就 CVE-2022-24768 而言，本文方法生成的策略能够在权限写入、策略表达和身份映射三个环节收紧攻击面。Kubernetes RBAC 收敛、Argo CD RBAC 加固与 Gatekeeper/Kyverno 准入控制协同后，攻击者难以再借助 `sync+override` 权限诱导控制平面创建高权限绑定。复现实验表明，策略生效后权限提升路径被阻断。该方案适用于已采用 GitOps 交付模式的 Kubernetes 集群，但仍需配合版本升级。

6.3 案例二：CVE-2025-2241（Hive ClusterProvision 敏感信息泄露）

6.3.1 漏洞详解与复现

漏洞背景

Hive 是 Red Hat Multicloud Engine（MCE）与 Advanced Cluster Management（ACM）中的集群生命周期管理组件，常用于在多集群场景下自动化创建与托管 OpenShift/Kubernetes 集群。在 vSphere 场景中，Hive 需要使用 vCenter 凭据完成集群的基础设施供应与后续管理。

CVE 描述：CVE-2025-2241 指出 Hive 在完成 vSphere 集群供应（provisioning）后，会将 vCenter 凭据暴露在 ClusterProvision 对象中。由于 ClusterProvision 属于自定义资源（CR），拥有其读取权限的用户可直接从该对象中提取敏感凭据，即使该用户并不具备对 Kubernetes Secret 的访问

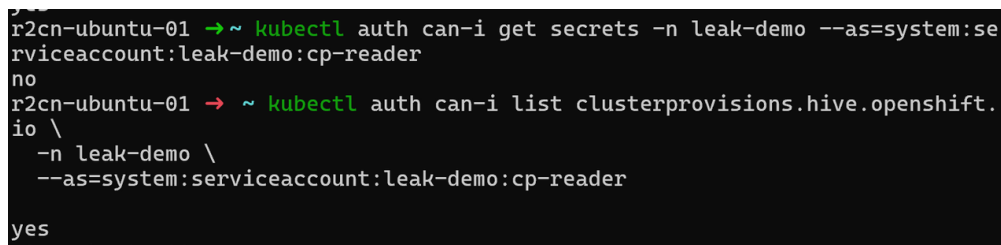
权限。该问题会导致未授权 vCenter 访问、基础设施与集群管理风险，并可能进一步演化为权限提升。

CVSS 信息：CNA (Red Hat) 给出的向量为 CVSS:3.1/AV:N/AC:H/PR:L, 基准分 8.2 (High)。其含义可理解为：攻击者需要一定前置权限 (PR:L) 和较高攻击复杂度 (AC:H)，但一旦满足条件，影响范围跨越安全边界 (S:C)，并造成高机密性与完整性影响 (C:H/I:H)。

主要成因分类：敏感信息泄露 / 配置与对象生命周期缺陷，即敏感数据出现在可读的 CR 状态字段中，形成对 Secret 隔离边界的旁路。

攻击链条描述：攻击链可概括为 Secret 隔离被 CR 可读性绕过：

1. **权限前置阶段：**攻击者拥有对 ClusterProvision 的 get/list/watch 权限（通常来自聚合角色 view/edit 或错误的绑定）；
2. **数据旁路阶段：**Hive 将 vCenter 凭据写入 ClusterProvision, 位于 status 字段；
3. **凭据提取阶段：**攻击者通过读取 CR YAML/JSON 直接获得凭据；
4. **后续风险阶段：**利用 vCenter 凭据对虚拟化平台执行未授权操作，进而影响集群节点与工作负载，造成横向移动或权限提升。



```
r2cn-ubuntu-01 → ~ kubectl auth can-i get secrets -n leak-demo --as=system:serviceaccount:leak-demo:cp-reader
no
r2cn-ubuntu-01 → ~ kubectl auth can-i list clusterprovisions.hive.openshift.io \
-n leak-demo \
--as=system:serviceaccount:leak-demo:cp-reader
yes
```

图 6-3: CVE-2025-2241: vCenter 凭据通过 ClusterProvision 暴露的风险示意

Fig. 6-3: CVE-2025-2241: illustration of vCenter credential exposure through ClusterProvision

漏洞复现

本案例复现目标是验证：无 Secret 访问权限但可读 ClusterProvision 的用户，是否可从对象中提取到 vCenter 凭据。由于该漏洞涉及敏感凭据泄露，本文仅给出受控测试环境的复现要点，详细 YAML 与命令序列见附录 C。

环境配置：测试集群为 OpenShift 或 Kubernetes，并已部署 ACM/MCE 与 Hive，具体版本以厂商公告与实验环境为准。基础设施侧需要可用的 vSphere 环境与测试专用的 vCenter 账号，建议采用最小权限配置。权限模型侧需准备至少一个只读主体，其具备 ClusterProvision 的读取权限但不具备 Secret 的读取权限，用于验证旁路泄露条件。

攻击步骤（摘要）：

1. 创建 vSphere 集群供应配置（Hive ClusterDeployment 等相关对象），触发供应流程；
2. 使用低权限但可读 ClusterProvision 的身份读取对象并导出 YAML；
3. 在输出中定位 vCenter 用户名/密码等敏感字段（例如 `status.vCenter.credentials.*` 或同类结构）。

攻击结果：若在 ClusterProvision 的可读字段中发现明文凭据，则说明存在经由 CR 状态字段的敏感信息旁路泄露；该风险不依赖 Secret 读取权限，属于权限边界外泄。

6.3.2 策略部署

本节给出可落地的缓解策略。本案例侧重敏感信息泄露的多层次控制，即通过**防护策略**收敛读取权限面，通过**监控策略**缩短暴露时间。

策略概述：该方案从**防护策略**（身份与权限、准入检测）与**监控策略**（审计与告警、资源生命周期治理）四个层面分层实施。具体方法包括：

- **防护策略：**收敛 ClusterProvision 的读取权限；采用 Gatekeeper 或 Kyverno 以 dryrun/Audit 模式对对象中疑似敏感字段进行检测提示。
- **监控策略：**启用读取行为审计并联动告警；在供应完成后对对象进行清理以缩短暴露窗口；在发现泄露风险后执行 vCenter 凭据轮换与强化。

策略配置与部署步骤：为避免正文过长，本案例的复现详细步骤、RBAC 清单、审计策略片段、自动化清理示例与 Gatekeeper/Kyverno 检测规则完整代码见附录C。

6.3.3 三因素分析

有效性：RBAC 收敛可直接降低非必要主体读取 ClusterProvision 的概率，是风险抑制的关键手段。审计与告警用于发现异常读取行为，并在凭据轮换后复核风险是否仍然存在。对象清理及其自动化能够进一步缩短凭据暴露窗口。审计型准入检测则用于在配置漂移或版本回退时及时发现敏感字段再次出现。

无干扰性：RBAC 收敛主要影响运维与排障可见性，因此需要以按需、限时、可审计的授权流程替代全局可读。审计与告警对业务链路通常无直接影响，但会增加日志量与告警治理成本。自动化清理需避免破坏 Hive 的对账或

回收流程，建议先在预生产环境验证，并以标签或状态条件精确匹配对象后再删除。

可部署性：防护策略层：RBAC 收敛属于平台原生能力，可在多集群中以模板化方式发布。准入**防护策略**建议从 `dryrun/Audit` 起步，用于检测与辅助治理，通常不建议强制拦截控制器写入 `status` 字段，以避免影响供应流程。

监控策略层：审计与告警对业务链路通常无直接影响，但会增加日志量与告警治理成本。自动化清理属于运维流程治理，建议与 `GitOps` 或流水线集成，在供应完成后打标签并由定时任务触发清理。

就 CVE-2025-2241 而言，本文方法生成的策略通过 RBAC 收敛限制了非必要主体对 `ClusterProvision` 的读取，并配合审计告警与对象自动清理缩短了凭据暴露窗口。该方案适合作为补丁发布前的临时缓解，但根因仍在于 Hive 控制器将凭据写入 CR 状态字段，最终修复仍依赖厂商补丁。

6.4 案例三：CVE-2023-30512（CubeFS CSI 过度权限配置）

6.4.1 漏洞详解与复现

漏洞背景

CubeFS 是面向云原生场景的分布式存储系统，常通过 CSI（Container Storage Interface）插件与 Kubernetes 集成。在 Kubernetes 中，CSI 节点插件通常以 `DaemonSet` 形式在每个节点上运行（例如 `cfs-csi-node`），控制器组件以 `Deployment` 形式运行（例如 `cfs-csi-controller`）。这类组件需要与 Kubernetes API 交互，但其权限应遵循最小权限原则。

CVE 描述：CVE-2023-30512 的核心问题是 CubeFS CSI 插件的服务账号（如 `cfs-csi-service-account`）被绑定到了包含**过度权限**的集群级角色（`ClusterRole`），从而具备对集群范围 `Secret` 的读取能力（如 `get/list`）。一旦攻击者在某节点获得对 CSI 节点插件 Pod 的控制，例如通过节点被入侵、容器逃逸或对系统命名空间特权工作负载的执行能力，便可能滥用该服务账号的身份边界，读取全局敏感信息并进一步演化为集群级权限提升。

主要成因分类：配置缺陷 / 过度授权（Over-privileged RBAC）。

攻击链条描述：攻击链可概括为系统组件身份被过度授权，且在被攻陷后可枚举集群秘密：

1. **身份前置阶段：**CSI 组件使用默认或被误配置的 `ServiceAccount` 运行；

2. **权限放大阶段：**ServiceAccount 被绑定到含 secrets 读取能力的 ClusterRole；
3. **控制获取阶段：**攻击者控制某节点上的 CSI Pod（例如 cfs-csi-node）并获得其运行时上下文；
4. **数据获取阶段：**攻击者以该 ServiceAccount 身份读取集群 Secret，从而获取高价值凭据（管理员 Token、云凭据等），实现从节点或 Pod 控制向集群控制的升级。

[illegible]

漏洞复现（验证性复现）

考虑到本文面向论文复现实验,且该漏洞的攻击细节容易被滥用,本文采用**验证性复现思路**:以RBAC 权限证据与受控权限验证为主,证明 `cfs-csi-service-account` 具备不应存在的 `secrets` 读取能力;同时在策略部署后复测,验证该能力被有效移除。完整命令与策略清单见附录D。

复现步骤（摘要）:

1. 盘点并导出 CSI 的 RBAC 绑定，确认 ClusterRole 规则含 `resources: [secrets]` 且 `verbs` 包含 `get/list/watch`;
2. 以该 `ServiceAccount` 身份执行权限验证 (`kubectl auth can-i`)，确认其具备跨命名空间读取 `Secret` 的能力;
3. 按最小权限、准入防回归与审计检测三项措施部署策略后，重复第 2 步

验证权限应为 no。

```
r2cn-ubuntu-01 → ~ curl -k -H "Authorization: Bearer $TOKEN" $API_SERVER/api/v1/
{
  "kind": "APIResourceList",
  "groupVersion": "v1",
  "resources": [
    {
      "name": "bindings",
      "singularName": "binding",
      "namespaced": true,
      "kind": "Binding",
      "verbs": [
        "create"
      ]
    },
    {
      "name": "componentstatuses",
      "singularName": "componentstatus",
      "namespaced": false,
      "kind": "ComponentStatus",
      "verbs": [
        "get",
        "list"
      ],
      "shortNames": [
        "cs"
      ]
    },
    {
      "name": "configmaps",
      "singularName": "configmap",
      "namespaced": true,
      "kind": "ConfigMap",
      "verbs": [
        "create",
        "delete",
        "deletecollection",
        "get",
        "list",
        "patch",
        "update",

```

图 6-5: CVE-2023-30512: 攻击成功证据 (Secrets 读取能力验证/权限提升结果截图)

Fig. 6-5: CVE-2023-30512: evidence of successful attack showing secret-reading capability and privilege escalation results

攻击结果 (摘要): 若服务账号具备集群范围 Secret 读取权限, 则攻击者一旦控制 CSI Pod/节点, 存在获取高价值凭据并进一步获得集群控制的风险。

6.4.2 策略部署

本案例侧重移除不必要的 Secrets 读取权限并防止权限回归。本文通过防护策略 (RBAC 基线治理 + 准入策略) 与监控策略 (审计检测) 的组合, 在配置漂移或误授权场景下提供分层防护。

策略概述: 该方案从防护策略 (身份与权限、策略执行、网络收敛) 与监控策略 (审计与检测) 等层面分层实施。核心方法包括:

- **防护策略:** 对 CSI 相关 RBAC 进行最小化与清理, 按需评估 ServiceAccount 硬化措施; 在准入侧阻断再次引入含 secrets 读取动词的 ClusterRole; 必要时进一步收敛 CSI 工作负载的出站访问面。

- **监控策略：**对 `secrets` 的枚举行为建立监控和告警。

策略配置与部署步骤：为避免正文过长，RBAC 最小化示例、准入控制规则、审计策略片段与验证命令见附录D。

6.4.3 三因素分析

有效性：RBAC 最小化可直接移除攻击链中的关键条件，即 `secrets` 读取能力，从根源降低风险。准入防回归用于阻断误配置、版本回滚或安装脚本重复执行导致的过度授权再次出现。审计检测可在异常访问出现时提供证据，并触发凭据轮换等止血动作。

无干扰性：RBAC 收敛可能影响 CSI 的部分诊断与自愈路径，因此应在预生产环境验证 PVC/PV 生命周期以及 `attach/detach` 等关键流程。准入策略若采用 `deny` 强制模式，需要避免误拦截平台必需组件的合法权限，建议先以 `dryrun/audit` 运行并结合白名单迭代。审计与检测会增加日志量与告警治理成本，但通常不影响业务链路。

可部署性：RBAC 与审计属于平台原生能力，适合在多集群场景中模板化推广。准入策略与网络收敛则属于长期基线治理内容，可通过 `GitOps` 与策略即代码持续交付。工程落地时建议提供回滚与分批推进路径，先在测试与预生产验证后再逐步推广至生产。

就 CVE-2023-30512 而言，本文方法生成的策略通过 RBAC 最小化直接移除了 CSI ServiceAccount 不应具备的 `secrets` 读取能力，并以准入策略阻止过度授权再次引入。复测显示 `kubectl auth can-i` 的结果由 `yes` 变为 `no`。部署时仍需验证 CSI 的 PVC/PV 生命周期与相关功能不受影响。

6.5 案例四：CVE-2020-4062 Conjur OSS 缺乏网络隔离策略

6.5.1 漏洞详解与复现

漏洞背景

Conjur OSS 是云原生场景下常用的机密管理系统，属于 `Secrets Management` 系统。在典型部署中，Conjur Server 负责鉴权与策略校验，后端数据库 PostgreSQL 用于持久化存储策略、审计与机密相关数据。Kubernetes 默认网络模型在多数容器网络接口 CNI 配置下呈扁平互通状态，Pod 间通信默认允许。因此，对于数据库等敏感组件，需要显式配置 `NetworkPolicy` 等隔离策略，

并建立默认拒绝策略 Default Deny，以控制最小暴露面。

CVE 描述：CVE-2020-4062 影响 Conjur OSS Helm Chart 2.0.0 之前版本。旧版 Chart 默认未提供对 Postgres 的访问控制策略，例如缺失 NetworkPolicy，导致数据库服务在集群内对任意 Pod 可达。即使数据库未通过 NodePort 或 LoadBalancer 暴露到公网，攻击者一旦获得集群内任意工作负载的执行上下文，仍可能横向移动并直连数据库，绕过应用层安全控制，从而造成机密数据泄露、策略被篡改与审计记录被破坏等后果。

严重等级：Critical。

主要成因分类：配置缺陷 / 网络访问控制缺失 Missing NetworkPolicy。

攻击链条描述：该漏洞可抽象为集群内横向移动与直连数据库绕过应用层安全的链条：

1. **初始立足点：**攻击者控制集群内任意 Pod，例如通过业务漏洞或错误授权获得容器执行能力；
2. **侦察发现：**在目标命名空间中发现 Postgres Service 与 5432 端口；
3. **网络直连：**由于缺失 NetworkPolicy，攻击者可从任意 Pod 直连 Postgres；
4. **影响扩展：**若获取数据库凭据或数据密钥（来源可能包括过宽 RBAC、配置泄露或备份泄露），风险可进一步扩展为机密窃取、权限篡改与审计删除。

漏洞复现（验证性复现）

本案例复现以网络暴露面验证为主，即证明在旧版 Chart 部署下，Postgres 在集群内可被非 Conjur 组件访问，随后再通过部署 NetworkPolicy 将该访问面收敛。考虑到该场景涉及数据库直连与敏感数据风险，本文不展开可被滥用的数据库操作细节，完整的环境准备与策略清单见附录E。

环境配置：测试集群为 Kubernetes（Minikube）与 Helm v3，受影响组件为 Conjur OSS Helm Chart < 2.0.0。同时要求集群 CNI 支持 NetworkPolicy；若不支持，则本文所述隔离策略无法生效。

复现步骤：

1. 在独立命名空间部署旧版 Conjur OSS Chart，并确认 Postgres Service 端口 5432 存在；
2. 证明该命名空间中不存在限制 Postgres 入站的 NetworkPolicy；
3. 进行受控连通性验证：从非 Conjur 组件工作负载发起到 Postgres 5432 的 TCP 探测，用于证明集群内可达；

4. 部署默认拒绝并仅允许 Conjur Server 访问 Postgres 的 NetworkPolicy, 复测应不可达。

攻击结果（摘要）：若未隔离 Postgres，攻击者一旦获得集群内任意 Pod 的执行上下文，即存在横向移动并直连数据库的机会，构成从业务容器到机密管理后端的跨域突破路径。

6.5.2 策略部署

该漏洞的治理重点是**收敛数据库暴露面并防止隔离策略回归**。本文采用以 NetworkPolicy 为核心的缓解路径：通过默认拒绝策略 Default Deny 收敛数据库入站访问面，仅允许 Conjur Server 访问 Postgres 5432 端口，并与命名空间隔离、RBAC 收敛及审计治理配套，以降低横向直连与配置漂移带来的风险。

策略概述：从网络隔离、身份与权限、审计与治理三个层面分层实施。网络层通过 NetworkPolicy 实现默认拒绝与精确放通，身份层通过 RBAC 限制对数据库凭据相关资源的读取面，治理层以基线检测与告警机制保证策略长期有效。

策略配置与部署步骤：为避免正文过长，NetworkPolicy 示例、验证命令与注意事项见附录E。

6.5.3 三因素分析

有效性：NetworkPolicy 能直接收敛数据库暴露面，将任意 Pod 可达降至仅 Conjur Server 可达，从网络层阻断横向直连路径。命名空间隔离与 RBAC 收敛可进一步降低数据库凭据被读取或滥用的概率，并与网络隔离形成配合。基线检测则用于在 Helm 回滚或配置漂移时，及时发现 NetworkPolicy 缺失、规则被弱化等回归风险。

无干扰性：NetworkPolicy 的引入可能影响 Conjur 组件间通信与运维排障，因此需在预生产完成连通性与回归验证，并准备回滚方案。若集群 CNI 不支持 NetworkPolicy，则相关策略无法执行，应优先通过命名空间隔离等手段降低暴露面，或在平台层面选用支持策略的 CNI。精确放通依赖正确的标签选择器，标签不一致可能造成误拦截，因此需要与 Chart 模板保持一致。

可部署性：NetworkPolicy 属于 Kubernetes 原生资源，便于纳入 GitOps 流程并在多集群环境中复制推广。由于数据库服务与端口相对稳定，规则维护成

本较低。为避免遗漏与回归，可将 NetworkPolicy 纳入 Helm values 或部署流水线的强制检查项，并与基线检测配合使用。

就 CVE-2020-4062 而言，本文方法生成的策略以 NetworkPolicy 建立默认拒绝，将数据库访问面从集群内任意 Pod 可达收敛为仅 Conjur Server 可达。复现实验表明，策略部署后非授权 Pod 无法连通 Postgres 5432 端口。该方案依赖集群 CNI 对 NetworkPolicy 的支持，不支持时需采用命名空间隔离等替代手段。

6.6 案例五：CVE-2022-24877 (Flux kustomize-controller 路径穿越)

6.6.1 漏洞详解与复现

漏洞背景

Flux 是云原生 GitOps 交付体系中的常用组件，典型部署包含 kustomize-controller 等控制器，用于在集群中持续对齐 Git 仓库的期望状态。在该工作流中，控制器会拉取仓库内容并调用 Kustomize 解析 kustomization.yaml，进而生成并应用资源清单。由于 kustomization.yaml 以文件路径为核心配置对象，控制器必须将其中指定的资源、补丁与组件路径解析为实际文件系统访问。若路径规范化与边界检查不足，则恶意构造的 kustomization.yaml 可能触发路径穿越，使控制器读取超出预期工作目录的文件，从而造成敏感信息暴露，并在多租户部署中形成隔离边界破坏的风险。

CVE 描述：CVE-2022-24877 是 Flux kustomize-controller 的路径穿越漏洞。攻击者可通过恶意 kustomization.yaml 触发控制器读取其 Pod 文件系统中的非预期文件，从而暴露敏感数据；在多租户部署下，该能力可能进一步放大为权限提升风险。该漏洞已在 kustomize-controller v0.24.0 中修复，并随 flux2 v0.29.0 集成发布。工程侧常见缓解手段包括在 CI/CD 流水线中自动校验 kustomization.yaml 是否符合安全策略。

主要成因分类：输入校验缺陷 / 路径穿越 (Path Traversal, 路径规范化与目录边界检查不足)。

攻击链条描述：攻击链可概括为仓库内容被信任、路径解析越界以及敏感数据外泄。攻击者将恶意 kustomization.yaml 提交至可被控制器同步的仓库；控制器在构建阶段解析并访问其中指定的路径；若路径可越界访问控

制器容器文件系统，则非预期文件内容可能被注入到构建产物、事件输出或后续流程中，造成敏感信息暴露，并在多租户环境中破坏隔离边界。

漏洞复现（验证性复现）

本案例复现采用**验证性复现**思路，重点验证控制器是否会在构建过程中读取超出预期目录边界的文件，并以构建日志与产物行为作为证据。复现的观测点包括：控制器在构建阶段出现与异常路径解析相关的错误或告警信息，或在严格受控的测试条件下出现构建产物包含非预期内容的迹象。为避免涉及敏感文件读取的可滥用细节，正文仅给出复现边界与证据判据。完整的环境准备、受控验证步骤与证据判据见附录F。

6.6.2 策略部署

本节给出针对路径解析越界所致数据暴露的优先级缓解策略。其治理思路以前置校验与 GitOps 变更治理为核心，用于降低多租户场景下的输入面风险。

策略概述：第一，在 CI/CD 或代码评审环节对 `kustomization.yaml` 进行策略化校验，阻断异常路径引用进入仓库。第二，在多租户环境中收敛对 Git 仓库写入与 Flux 资源变更的权限。第三，通过运行时硬化与最小权限配置降低异常路径被利用后的后果。

策略配置与部署步骤：为避免正文堆叠大量清单，CI 校验示例、权限收敛要点与验证建议见附录F。

6.6.3 三因素分析

有效性：CI 校验用于在提交阶段阻断异常路径引用进入仓库，从而显著降低触发概率。权限与变更治理用于限制恶意 `kustomization.yaml` 进入同步面，运行时硬化与最小权限配置则在多租户场景下缩小爆炸半径。

无干扰性：CI 校验可能对历史仓库中不规范写法产生阻断，因此应采用先告警后强制的方式逐步推进，并提供例外与审批流程。多租户下的权限收敛可能影响自助交付效率，需要以标准化模板与可审计的变更机制替代高权限直写。运行时硬化通常对业务逻辑影响较小，但仍需验证其与现有控制器权限、文件访问行为及构建流程的兼容性。

可部署性：上述措施以工程流程与平台能力为主，因而便于推广应用：CI 校验可作为流水线准入检查复用；多租户权限收敛与运行时硬化可通过 GitOps

模板化交付，实现持续改进。

就 CVE-2022-24877 而言，本文方法生成的策略通过 CI 流水线中的 `kustomization.yaml` 校验在提交阶段拦截异常路径引用，再配合多租户权限收敛与运行时硬化限制路径穿越被利用后的影响范围。该方案可作为补丁发布前的辅助缓解措施，最终修复仍依赖升级至 `kustomize-controller v0.24.0` 及以上版本。

6.7 案例六：CVE-2022-29171（Sourcegraph gitserver 远程代码执行）

6.7.1 漏洞详解与复现

漏洞背景

Sourcegraph 是面向大规模代码仓库的搜索与导航平台，其典型部署包含负责仓库拉取与 Git 操作的 `gitserver` 服务。在企业集成场景中，Sourcegraph 可接入多种代码托管与协作系统。为获得额外元数据，部分集成允许管理员在系统中配置外部命令，由后端服务在特定路径下执行并返回结果。此类机制天然具有较高风险：若命令可被高权限用户任意改写，且执行环境与网络边界缺少隔离，则可能形成配置驱动的命令执行入口。

CVE 描述：CVE-2022-29171 指出 Sourcegraph `gitserver` 在 Gitolite 与 Phabricator 联动集成中存在远程代码执行风险。在受影响版本（< 3.38.0）中，站点管理员可配置用于获取 Phabricator 元数据的 `callsignCommand`；若攻击者同时具备对 Sourcegraph 实例的站点管理员权限，以及对其内置 Grafana 监控实例的管理员权限，则可进一步获取内部服务地址信息，并将上述命令改写为任意系统命令，从而在 `gitserver` 上触发远程执行。该问题已在 3.38.0 版本修复，官方仍建议即使采取临时缓解也应尽快升级。

主要成因分类：安全逻辑缺陷 / 命令注入（配置驱动的命令执行缺乏约束）与权限边界缺陷（高敏感配置权限与观测面权限向执行面耦合）。

攻击链条描述：攻击链可概括为高权限配置、执行面触达以及基础设施控制。攻击者首先获得可编辑或新增 Gitolite code host 的管理权限；随后借助 Grafana 管理权限获取 `gitserver` 的内部寻址信息；最终改写 `callsignCommand` 并触发执行。风险的核心在于管理面配置、观测面信息与执行面能力发生了耦合。

漏洞复现（验证性复现）

本文采用**验证性复现**思路，在隔离环境中验证外部命令执行风险。复现不提供可复用攻击载荷，而以审计日志与任务执行记录为主：在受影响版本中，若可观察到 `gitserver` 因配置项变更而执行外部命令的迹象，则可判定存在执行面风险；升级至修复版本或禁用相关集成后，该迹象应消失。完整环境准备、验证判据与回归检查步骤见附录G。

6.7.2 策略部署

本案例的治理重点是**切断命令执行入口并隔离执行面与观测面**。考虑到攻击链对权限前置条件要求较高，治理目标应落在降低误授权限后的可达性与影响范围，避免配置权限与观测面权限共同指向执行面。

策略概述：第一，若业务不依赖 Gitolite 集成，禁用或移除 Gitolite code host，消除 `callsignCommand` 对应的输入面。第二，对 `gitserver` 实施网络隔离，采用默认拒绝并按需放通的方式收敛可达面，降低内部寻址信息被利用后的横向触达能力。第三，进行权限分离与最小化，严格限制外部服务配置的变更主体，并收敛 Grafana 的暴露面与管理员权限，降低观测面泄露内部拓扑的风险。**策略配置与部署步骤：**为避免正文过长，策略配置要点与验证建议见附录G。

上述措施可形成分层防护：功能禁用用于收敛输入面，网络隔离与权限最小化用于限制影响范围。落地时应将相关功能开关、网络策略与权限配置纳入基线，并在变更后复测关键判据，避免因回退或旁路路径导致风险回归。

6.7.3 三因素分析

有效性：禁用 Gitolite 集成可直接移除关键输入面，适用于不依赖该功能的部署。网络隔离与权限最小化属于分层防护：即使出现站点管理员权限误授或观测面越权，也能显著提高攻击链的达成难度并降低爆炸半径。

无干扰性：禁用 Gitolite 会影响依赖该集成的仓库同步与元数据功能，因此需要评估替代接入方式。网络隔离需要精确梳理 `gitserver` 的入站/出站依赖，初期可能带来误拦截风险，建议先在预生产环境逐步验证并配套监控。权限分离会提升变更门槛，但对高敏感配置属于必要治理。

可部署性：功能禁用在多数部署中可通过标准化配置管理实现。`NetworkPolicy` 等网络隔离能力依赖集群 CNI 支持，但在 Kubernetes 环境中易于模板化推广。权限分离与 Grafana 暴露面治理属于组织流程与平台基线配置，适合

纳入 GitOps 基线与审计机制持续管理。

就 CVE-2022-29171 而言，本文方法生成的策略通过禁用 Gitolite 集成切断了 `callsignCommand` 对应的命令执行入口，并以网络隔离和权限分离限制误授权限后的可达范围与影响半径。由于该攻击链同时依赖站点管理员与 Grafana 管理员权限，防护效果与权限治理的严格程度直接相关，最终仍建议升级至修复版本。

6.8 案例七：CVE-2023-22482（Argo CD OIDC aud 未校验导致越权访问）

6.8.1 漏洞详解与复现

漏洞背景

Argo CD 是 Kubernetes 场景中常用的声明式 GitOps 持续交付工具，提供 Web 与 API 接口供用户进行应用同步、回滚与差异审计等操作。在企业环境中，Argo CD 常通过 OIDC 与统一身份提供方对接，由身份提供方为 Argo CD 签发包含用户身份与组信息的令牌。在 OIDC 令牌中，`aud`（audience）声明用于标识该令牌的预期接收方。通常情况下，服务端除验证签名与发行者外，还需要校验 `aud` 是否包含本服务的受众标识，以避免将为其他服务签发的令牌误用到本服务上。

CVE 描述：CVE-2023-22482 指出 Argo CD 在部分版本中存在不恰当授权缺陷。其能够验证令牌由已配置的 OIDC 提供方签名，但未对令牌的 `aud` 声明进行校验，从而可能接受并非面向 Argo CD 的有效令牌。若同一身份提供方还为其他系统签发令牌，攻击者一旦获得其他系统的有效令牌，就可能将其用于访问 Argo CD。Argo CD 将基于令牌中的 `groups` 声明授予权限，即使这些组信息并非为 Argo CD 的授权场景设计。该问题在 2.6.0-rc3、2.5.6、2.4.19 与 2.3.13 及以上版本修复；受影响范围包含 $\geq 1.8.2$ 且低于上述修复版本的版本分支。

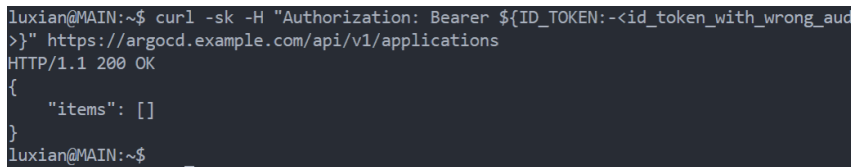
主要成因分类：安全逻辑缺陷 / 身份与授权校验缺失（令牌受众 `aud` 未校验导致跨受众令牌复用）。

攻击链条描述：攻击链可概括为多受众身份提供、跨受众令牌复用以及基于组声明的越权访问。攻击者获得同一 OIDC 提供方为其他受众签发的有效令牌；将其提交给 Argo CD 的接口；若 Argo CD 未校验 `aud`，则会接受该

令牌并根据 `groups` 映射授予权限，进而造成对应用资源与交付流程的非预期访问。该缺陷同时放大了令牌泄露事件的影响范围。

漏洞复现（验证性复现）

本文采用**验证性复现**思路，验证 `aud` 不匹配的有效令牌是否会被 Argo CD 接受这一关键安全性质。复现过程不展开可直接复用的令牌获取与请求构造细节，而以配置证据、令牌声明证据与接口返回结果为主：在受影响版本中，若 Argo CD 在已验签前提下接受 `aud` 不匹配的令牌并返回受 RBAC 控制的资源，则可判定存在受众校验缺失；升级至修复版本后，同样令牌应在校验阶段被拒绝。完整的环境准备、验证判据与回归检查步骤见附录H。



```
luxian@MAIN:~$ curl -sk -H "Authorization: Bearer ${ID_TOKEN:-<id_token_with_wrong_aud>}" https://argocd.example.com/api/v1/applications
HTTP/1.1 200 OK
{
  "items": []
}
luxian@MAIN:~$
```

图 6-6: 验证性复现证据示意（审计/接口响应截图的经处理摘要）

Fig. 6-6: Illustration of validation-oriented reproduction evidence based on processed summaries of audit and API response screenshots

6.8.2 策略部署

该漏洞的治理重点是**降低跨系统令牌复用的可行性与后果**。在补丁窗口期内，可通过身份提供方配置、前置网关校验与 RBAC 收敛构建补充措施，阻断攻击路径并降低风险暴露面。

策略概述：第一，在身份提供方侧为 Argo CD 配置独立客户端与唯一受众，限制该客户端签发的组声明范围，避免与其他系统的组命名空间混用。第二，在 Argo CD 前置入口实施令牌受众强制校验，例如通过 API 网关对 `iss`、`aud` 与签名进行一致性验证，拒绝 `aud` 不匹配的请求。第三，收敛 Argo CD 的 RBAC 与项目边界，限制高权限组的授予范围，并将关键操作纳入审计与告警，降低令牌误用后的爆炸半径。**策略配置与部署步骤：**为避免正文过长，策略配置要点与验证建议见附录H。

6.8.3 三因素分析

有效性：身份提供方侧的客户端隔离可减少可被复用的令牌来源，前置网关的 `aud` 强制校验可在应用层之前阻断异常令牌进入。RBAC 与项目边界收敛则用于限制误用令牌的权限上限，三者配合可形成分层防护。

无干扰性：前置网关校验对访问路径有一定侵入性，因此需评估其对 CLI、Web 与自动化流水线的影响，并提供分批实施方案。身份提供方的客户端隔离与组声明裁剪可能影响既有用户组映射，需要配套变更沟通与回滚预案。RBAC 收敛可能降低自助交付效率，应通过标准化角色与审批流程平衡安全与效率。

可部署性：身份提供方侧配置与网关校验可作为平台能力复用，在多集群场景中可模板化推广。RBAC 与项目边界策略可通过 GitOps 管理并持续审计，作为长期基线治理的组成部分。

就 CVE-2023-22482 而言，本文方法生成的策略通过身份提供方侧的客户端隔离和前置网关的 aud 强制校验，在应用层之前阻断了跨受众令牌复用，并以 RBAC 收敛限制令牌误用后的权限上限。该方案可作为补丁窗口期的补充缓解措施，最终修复仍需升级至官方修复版本。

表 6-3 从有效性、无干扰性与可部署性三个维度汇总了 7 个案例的验证结论。

表 6-3: 案例验证三维质量汇总
Tab. 6-3: Summary of three-dimensional quality verification across cases

序号	CVE 编号	有效性 <i>E</i>	无干扰性 <i>I</i>	可部署性 <i>D</i>
1	CVE-2022-24768	多环节阻断攻击链，复测确认提权失败	保留正常同步能力，建议先审计后强制	RBAC/准入控制均为原生能力
2	CVE-2025-2241	RBAC 收敛 + 审计告警 + 对象清理降低凭据暴露	需以按需授权替代全局可读	原生能力，准入建议 dryrun 起步
3	CVE-2023-30512	直接移除 secrets 读取能力，复测 can-i 为 no	需验证 CSI 的 PVC/PV 生命周期	RBAC/准入可模板化推广
4	CVE-2020-4062	默认拒绝策略阻断横向移动，复测端口不可达	需验证组件间通信与标签一致性	NetworkPolicy 原生资源，维护成本低
5	CVE-2022-24877	CI 校验阻断异常路径引用进入仓库	历史不规范写法可能触发误报	CI 准入检查可复用，GitOps 交付
6	CVE-2022-29171	禁用 Gitolite 消除命令执行入口	影响依赖该集成的仓库同步功能	标准化配置管理，可模板化推广
7	CVE-2023-22482	前置网关 aud 校验阻断跨受众令牌复用	需评估对 CLI/Web/流水线的影	身份侧配置可跨集群复用

6.9 小结

本章案例验证表明，7 个覆盖不同成因类型的典型 CVE 在隔离集群中均可完成漏洞复现与策略部署验证。如表 6-3 所示，生成的监控策略与防护策略在有效性维度上均能阻断或显著缓解攻击路径；在无干扰性维度上，多数策略对业务影响可控，但仍需在预生产环境验证；在可部署性维度上，策略主要依赖 Kubernetes 原生能力与成熟准入控制组件，具备进一步模板化推广的条件。

7 结论与展望

7.1 总结

本文面向 Kubernetes 集群第三方开源应用中的权限提升漏洞，围绕漏洞披露到补丁在生产环境中就绪之间的窗口期问题，提出了多智能体协同的策略化漏洞防控技术。该技术能够面向权限提升相关 CVE 自动生成可执行的监控策略与防护策略，并在 55 个真实漏洞样本上验证了其可行性。本文的主要贡献可概括为以下四点：

(1) 构建了面向权限提升漏洞的向量化安全知识库。针对漏洞成因多样、上下文分散的问题，本文从代码仓库、漏洞公告、议题与合并请求等多源渠道组织带来源标识的安全证据，并提出面向策略生成的两级成因分类框架，为后续漏洞分析与策略生成提供结构化支撑。

(2) 提出了面向策略生成与评估的完整方法链条。本文给出了安全策略的三元组形式化表示与三维质量排序评价方法，并设计了由六类智能体协同完成的生成 workflow。该 workflow 通过检索增强约束事实依据，通过思维链推理细化分析过程，并借助评审反馈驱动定向修正，同时引入多评审主体共识聚合机制，以降低单一评审偏差。

(3) 实现了 KPEAgent 系统原型。该系统集成了 CVE 知识库管理、策略生成 workflow、评审反馈交互、专家排序与策略部署等功能模块，能够为安全工程师与专家评审员提供从漏洞分析到策略部署的可操作工具支持。

(4) 验证了所提方法在真实漏洞场景中的可行性。本文在 55 个云原生权限提升漏洞样本上开展了消融实验与模型对比实验，结果表明检索增强、思维链推理与反馈优化三个模块均对策略质量有正向贡献，不同模型在策略生成任务上也呈现出明显的能力差异。进一步地，本文选取访问控制缺陷、冗余权限、命令注入、身份验证绕过等多种成因类型下的 7 个典型 CVE，在隔离环境中完成漏洞复现与策略部署，从三个评价维度验证了策略在真实攻击场景下的实际效果。

总体来看，本文形成了从理论建模、方法设计到工程实现的较完整技术方案，为补丁窗口期内的权限提升漏洞处置提供了一条可落地的临时防控路径，也为大语言模型在安全运维场景中的应用积累了实践基础。

7.2 展望

尽管本文已完成方法设计、系统实现与实验验证，但仍有进一步完善和扩展的空间。后续工作可从以下三个方向继续推进：

（1）**高危漏洞的自动化监控与知识库实时更新**。当前知识库采用离线批量构建方式，无法及时响应新披露漏洞。未来可对 GitHub Security Advisory、NVD 等公开数据源建立持续监控与订阅机制，实现增量知识库更新与策略重评估的自动化触发，缩短从漏洞披露到策略交付的响应时间。

（2）**漏洞利用的自动化复现与验证**。当前策略验证主要依赖人工模拟或静态推理，缺乏真实利用环境下的自动化验证能力。未来可基于漏洞披露信息自动搭建包含漏洞的 Kubernetes 测试集群，结合 PoC 代码执行攻击重放，量化评估策略的阻断成功率与误报率，为策略质量评价提供更客观的证据支撑。

（3）**研究范围向其他漏洞类型扩展**。本文聚焦于权限提升漏洞，未来可将方法体系扩展至拒绝服务漏洞、供应链与镜像安全漏洞等云原生环境中的其他高危类型，通过扩展分类框架与优化评价维度，构建更全面的智能化安全防控能力。

若上述方向能够持续推进，云原生安全防控有望进一步由被动应对走向主动防御，并由单点防护走向更稳定的体系化防御。

参考文献

- [1] BOUGUETTAYA A, SINGH M, HUHNS M, et al. A service computing manifesto: the next 10 years[J/OL]. Commun. ACM, 2017, 60(4): 64 – 72. <https://doi.org/10.1145/2983528>.
- [2] ZHONG C, LI S, HUANG H, et al. Domain-driven design for microservices: An evidence-based investigation[J/OL]. IEEE Transactions on Software Engineering, 2024, 50(6): 1425-1449. DOI: 10.1109/TSE.2024.3385835.
- [3] Cloud Native Computing Foundation. Cncf cloud native definition v1.1[Z]. 2024.
- [4] Cloud Native Computing Foundation. Who we are[Z]. 2026.
- [5] 吴逸文, 张洋, and 王怀民王涛. 从 Docker 容器看容器技术的发展: 一种系统文献综述的视角[J]. 软件学报, 2023, 34(12): 5527-5551.
- [6] VERMA A, PEDROSA L, KORUPOLU M, et al. Large-scale cluster management at google with borg[C/OL]//EuroSys ' 15: Proceedings of the Tenth European Conference on Computer Systems. ACM, 2015: 1 – 17. <http://dx.doi.org/10.1145/2741948.2741964>.
- [7] LAWSON A, SICA J. Cncf annual cloud native survey: The infrastructure of ai's future[R/OL]. The Linux Foundation, 2026. <https://www.cncf.io/reports/the-cncf-annual-cloud-native-survey/>.
- [8] FERRAILOLO D, CUGINI J, KUHN D R, et al. Role-based access control (rbac): Features and motivations[C]//Proceedings of the 11th Annual Computer Security Applications Conference. 1995: 241-48.
- [9] GU Y, TAN X, ZHANG Y, et al. Epscan: Automated detection of excessive rbac permissions in kubernetes applications[C/OL]//Proceedings of the 2025 IEEE Symposium on Security and Privacy (SP). 2025: 3199-3217. DOI: 10.1109/SP61157.2025.00011.
- [10] SUN C A, HAN X, GONG Y. Kubeguard: A systematic permission-oriented risk detection approach for kubernetes applications[C/OL]//Proceedings of the 2025 IEEE International Conference on Web Services (ICWS). 2025: 855-865. DOI: 10.1109/ICWS67624.2025.00111.

- [11] YANG N, SHEN W, LI J, et al. Take over the whole cluster: Attacking kubernetes via excessive permissions of third-party applications[C/OL]//CCS '23: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security. New York, NY, USA: Association for Computing Machinery, 2023: 3048–3062. <https://doi.org/10.1145/3576915.3623121>.
- [12] SHAMIM S I, HU H, RAHMAN A. On prescription or off prescription? an empirical study of community-prescribed security configurations for kubernetes[C/OL]//Proceedings of the 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE). 2025: 2432-2444. DOI: 10.1109/ICSE55347.2025.00170.
- [13] RAHMAN A, SHAMIM S I, BOSE D B, et al. Security misconfigurations in open source kubernetes manifests: An empirical study[J/OL]. ACM Trans. Softw. Eng. Methodol., 2023, 32(4). <https://doi.org/10.1145/3579639>.
- [14] Falco (CNCF). Falco: Cloud native runtime security[Z]. 2025.
- [15] Open Policy Agent (CNCF). Open policy agent: Policy-based control for cloud native environments[Z]. 2025.
- [16] Kyverno (CNCF). Kyverno: Kubernetes native policy management[Z]. 2025.
- [17] Kubernetes. Network policies | kubernetes documentation[Z]. 2025.
- [18] MINNA F, BLAISE A, REBECCHI F, et al. Understanding the security implications of kubernetes networking[J/OL]. IEEE Security & Privacy, 2021, 19(5): 46-56. DOI: 10.1109/MSEC.2021.3094726.
- [19] BUDIGIRI G, BAUMANN C, MÜHLBERG J T, et al. Network policies in kubernetes: Performance evaluation and security analysis[C/OL]//Proceedings of the 2021 Joint European Conference on Networks and Communications & 6G Summit (EuCNC/6G Summit). 2021: 407-412. DOI: 10.1109/EuCNC/6GSummit51104.2021.9482526.
- [20] WRIGHT C P, DAVE J, GUPTA P, et al. Versatility and unix semantics in namespace unification[J/OL]. ACM Trans. Storage, 2006, 2(1): 74 – 105. <https://doi.org/10.1145/1138041.1138045>.
- [21] Kubernetes. Kubernetes documentation[Z]. 2024.
- [22] MEAD N R. Computer security: Art and science[J]. IEEE Security & Privacy, 2003, 1(3): 14-14.

- [23] Aqua Security. Trivy: A simple and comprehensive vulnerability scanner for containers[Z]. 2025.
- [24] Anchore. Gype: A vulnerability scanner for container images and filesystems [Z]. 2025.
- [25] Anchore. Syft: A cli tool and library for generating sboms from container images[Z]. 2025.
- [26] Kubescape. Kubescape: Kubernetes security scanning and hardening tool[Z]. 2025.
- [27] kube-linter. kube-linter: Static analysis for kubernetes yaml and helm charts [Z]. 2025.
- [28] kube-score. kube-score: Kubernetes object analysis tool for best practices[Z]. 2025.
- [29] KubeSec. Kubsec: Security risk analysis for kubernetes resources[Z]. 2025.
- [30] Fairwinds. Polaris: Kubernetes best-practices and configuration checks[Z]. 2025.
- [31] Bridgecrew (Checkov). Checkov: Infrastructure as code (iac) static analysis tool[Z]. 2025.
- [32] CYBERARK. Kubiscan: Kubernetes risk assessment tool[Z]. 2024.
- [33] APPVIA. Krane: Kubernetes rbac static analysis & visualisation tool[Z]. 2024.
- [34] NETWORKS P A. Kubernetes privilege escalation: Excessive permissions in popular platforms[Z]. 2022.
- [35] HØILAND-JØRGENSEN T, BROUER J D, BORKMANN D, et al. The express data path: Fast programmable packet processing in the operating system kernel[C]//Proceedings of the 14th international conference on emerging networking experiments and technologies. 2018: 54-66.
- [36] SALTZER J, SCHROEDER M. The protection of information in computer systems[J/OL]. Proceedings of the IEEE, 1975, 63(9): 1278-1308. DOI: 10.1109/PROC.1975.9939.
- [37] O'SHEA G. Redundant access rights[J]. Computers & Security, 1995, 14(4): 323-348.
- [38] MOLLOY I, PARK Y, CHARI S. Generative models for access control poli-

- cies: applications to role mining over logs with attribution[C/OL]//SACMAT '12: Proceedings of the 17th ACM Symposium on Access Control Models and Technologies. New York, NY, USA: Association for Computing Machinery, 2012: 45–56. <https://doi.org/10.1145/2295136.2295145>.
- [39] SANDERS M W, YUE C. Minimizing privilege assignment errors in cloud services[C/OL]//CODASPY '18: Proceedings of the Eighth ACM Conference on Data and Application Security and Privacy. New York, NY, USA: Association for Computing Machinery, 2018: 2–12. <https://doi.org/10.1145/3176258.3176307>.
- [40] DAVIL L, DMITRIENKO A, SADEGHI A R, et al. Privilege escalation attacks on android[C]//Proceedings of the Information Security Conference. 2010.
- [41] HUTCHINSON S, VAROL C. A survey of privilege escalation detection in android[C]//Proceedings of the 2018 9th IEEE Annual Ubiquitous Computing, Electronics & Mobile Communication Conference (UEMCON). 2018.
- [42] WU D, GAO D, LI Y, et al. Seccomp: Towards practically defending against component hijacking in android applications[C]//Proceedings of the 32nd Annual Conference on Computer Security Applications (ACSAC). 2016.
- [43] EL-ZAWAWY M A, HAMDY A. Detection of hidden privilege escalations in android[J]. Automated Software Engineering, 2025, 32(2).
- [44] TAYLOR V F, BERESFORD A R, MARTINOVIC I. Intra-library collusion: A potential privacy nightmare on smartphones[C]//Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS). 2017.
- [45] AU K W Y, ZHOU Y F, HUANG Z, et al. Pscout: analyzing the android permission specification[C/OL]//CCS '12: Proceedings of the 2012 ACM Conference on Computer and Communications Security. New York, NY, USA: Association for Computing Machinery, 2012: 217 – 228. <https://doi.org/10.1145/2382196.2382222>.
- [46] BAGHERI H, SADEGHI A, GARCIA J, et al. Deldroid: An automated approach for determination and enforcement of least-privilege architecture in android[J]. Journal of Systems and Software, 2019.
- [47] XU Y, WANG G, REN J, et al. An adaptive and configurable protection frame-

- work against android privilege escalation threats[J]. *Future Generation Computer Systems*, 2019, 92: 210-224.
- [48] HUSSAIN A K H, KAKAVAND M, SILVAL M. A novel android security framework to prevent privilege escalation attacks[J]. *International Journal of Computer Network and Information Security (IJCNIS)*, 2020, 12(1): 20-26.
- [49] SHARMEEN S, HUDA S, ABAWAJY J H, et al. An adaptive framework against android privilege escalation threats using deep learning and semi-supervised approaches[J]. *Applied Soft Computing*, 2020.
- [50] TRIPP O, PISTOIA M, FINK S J, et al. Taj: effective taint analysis of web applications[C/OL]//*Proceedings of the 30th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2009)*. New York, NY, USA, 2009: 87-97. DOI: 10.1145/1542476.1542486.
- [51] ENCK W, GILBERT P, HAN S, et al. Taintdroid: An information-flow tracking system for realtime privacy monitoring on smartphones[J/OL]. *ACM Transactions on Computer Systems*, 2014, 32(2): 1-29. DOI: 10.1145/2619091.
- [52] YOU W, LIANG B, SHI W, et al. Taintman: An art-compatible dynamic taint analysis framework on unmodified and non-rooted android devices[J/OL]. *IEEE Transactions on Dependable and Secure Computing*, 2020, 17(1): 209-222. DOI: 10.1109/TDSC.2017.2740169.
- [53] ZHANG J, YANG L, HAN Y, et al. A small leak will sink many ships: Vulnerabilities related to mini-programs permissions[C]//*Proceedings of the 2023 IEEE 47th Annual Computers, Software, and Applications Conference (COMPSAC)*. IEEE, 2023: 595-606.
- [54] WANG Y, FAN M, ZHOU H, et al. Minichecker: Detecting data privacy risk of abusive permission request behavior in mini-programs[C/OL]//*ASE '24: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. New York, NY, USA: Association for Computing Machinery, 2024: 1667-1679. <https://doi.org/10.1145/3691620.3695534>.
- [55] ISLAM SHAMIM M S, AHAMED BHUIYAN F, RAHMAN A. Xi commandments of kubernetes security: A systematization of knowledge related to kubernetes security practices[C/OL]//*Proceedings of the 2020 IEEE Secure*

- Development (SecDev). 2020: 58-64. DOI: 10.1109/SecDev45635.2020.21525.
- [56] SHAMIM S I. Mitigating security attacks in kubernetes manifests for security best practices violation[C/OL]//ESEC/FSE 2021: Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. New York, NY, USA: Association for Computing Machinery, 2021: 1689 – 1690. <https://doi.org/10.1145/3468264.3473495>.
- [57] CURTIS J A, EISTY N U. The kubernetes security landscape: Ai-driven insights from developer discussions[C/OL]//Proceedings of the 2025 IEEE/ACIS 23rd International Conference on Software Engineering Research, Management and Applications (SERA). 2025: 179-186. DOI: 10.1109/SERA65747.2025.11154503.
- [58] SHAMIM S I, HU H, RAHMAN A. Dynamic application security testing for kubernetes deployment: An experience report from industry[C/OL]//FSE Companion '25: Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering. New York, NY, USA: Association for Computing Machinery, 2025: 514–519. <https://doi.org/10.1145/3696630.3728573>.
- [59] ZEROUALI A, OPDEBEECK R, DE ROOVER C. Helm charts for kubernetes applications: Evolution, outdatedness and security risks[C/OL]//Proceedings of the 2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR). 2023: 523-533. DOI: 10.1109/MSR59073.2023.00078.
- [60] BLAISE A, REBECCHI F. Stay at the helm: secure kubernetes deployments via graph generation and attack reconstruction[C/OL]//Proceedings of the 2022 IEEE 15th International Conference on Cloud Computing (CLOUD). 2022: 59-69. DOI: 10.1109/CLOUD55607.2022.00022.
- [61] MINNA F, MASSACCI F, TUMA K. Analyzing and mitigating (with llms) the security misconfigurations of helm charts from artifact hub[J]. Empirical Software Engineering, 2025, 30(5): 132.
- [62] UL HAQUE M, KHOLOOSI M M, BABAR M A. Kgsecconfig: A knowl-

- edge graph based approach for secured container orchestrator configuration [C/OL]//Proceedings of the 2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER). 2022: 420-431. DOI: 10.1109/SANER53432.2022.00057.
- [63] DELL'IMMAGINE G, SOLDANI J, BROGI A. Kubehound: Detecting microservices' security smells in kubernetes deployments[J/OL]. Future Internet, 2023, 15(7). <https://www.mdpi.com/1999-5903/15/7/228>. DOI: 10.3390/fi15070228.
- [64] KIM B, KIM J, LEE S. Exploring security enhancements in kubernetes cni: A deep dive into network policies[J/OL]. IEEE Access, 2025, 13: 35322-35338. DOI: 10.1109/ACCESS.2025.3543841.
- [65] PECKA N, BEN OTHMANE L, VALANI A. Privilege escalation attack scenarios on the devops pipeline within a kubernetes environment[C/OL]// ICSSP '22: Proceedings of the International Conference on Software and System Processes and International Conference on Global Software Engineering. New York, NY, USA: Association for Computing Machinery, 2022: 45-49. <https://doi.org/10.1145/3529320.3529325>.
- [66] CESARANO C, NATELLA R. Kubefence: Security hardening of the kubernetes attack surface[A/OL]. 2025. arXiv: 2504.11126. <https://arxiv.org/abs/2504.11126>.
- [67] ZHU H, GEHRMANN C. Kub-sec, an automatic kubernetes cluster apparmor profile generation engine[C/OL]//Proceedings of the 2022 14th International Conference on COMMunication Systems & NETWORKS (COMSNETS). 2022: 129-137. DOI: 10.1109/COMSNETS53615.2022.9668504.
- [68] LE M V, AHMED S, WILLIAMS D, et al. Securing container-based clouds with syscall-aware scheduling[C/OL]//ASIA CCS '23: Proceedings of the 2023 ACM Asia Conference on Computer and Communications Security. New York, NY, USA: Association for Computing Machinery, 2023: 812-826. <https://doi.org/10.1145/3579856.3582835>.
- [69] REIMERS N, GUREVYCH I. Sentence-bert: Sentence embeddings using siamese bert-networks[C]//Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Con-

- ference on Natural Language Processing (EMNLP-IJCNLP). 2019: 3982-3992.
- [70] SONG K, TAN X, QIN T, et al. Mpnet: Masked and permuted pre-training for language understanding[J]. Advances in Neural Information Processing Systems, 2020, 33: 16857-16867.
- [71] DOUZE M, et al. The faiss library[A/OL]. 2024. arXiv: 2401.08281. <https://arxiv.org/abs/2401.08281>.
- [72] WU X, SARAF P P, LEE G, et al. Unveiling scoring processes: Dissecting the differences between llms and human graders in automatic scoring[J]. Technology, Knowledge and Learning, 2025: 1-16.
- [73] WANG P, LI L, et al. Large language models are not fair evaluators[A/OL]. 2023. arXiv: 2305.17926. <https://arxiv.org/abs/2305.17926>.

附录 A 云原生权限提升漏洞分类与 CWE 对应关系

表 A-1: 云原生权限提升漏洞分类框架与 CWE 对应关系
Tab. A-1: Mapping between the classification framework of cloud-native privilege escalation vulnerabilities and CWE

一级分类	二级分类	主要 CWE	CWE 描述
配置缺陷	冗余权限	CWE-269	不当访问控制（通用型）
		CWE-266	权限分配错误
		CWE-732	关键资源权限分配错误
	网络隔离不足	CWE-653	隔离措施不足
		CWE-668	资源暴露至错误安全域
		CWE-669	资源在不同安全域间转移错误
安全逻辑缺陷	访问控制缺陷	CWE-863	授权验证错误
		CWE-284	不当访问控制
		CWE-862	缺失授权机制
	身份验证绕过	CWE-287	不当身份验证
		CWE-290	硬编码密码进行身份验证
		CWE-327	使用存在缺陷或高风险的加密算法
	输入验证不足	CWE-20	不当输入验证
		CWE-74	输出内容中特殊元素的中和处理不当
		CWE-79	Web 页面生成过程中输入的中和处理不当
	授权验证不完善	CWE-285	授权验证不当
敏感信息泄露	API 端口暴露	CWE-250	以非必要权限执行操作
		CWE-268	特权操作执行不当
		CWE-200	敏感信息泄露给未授权主体
	日志泄露	CWE-358	动态识别资源的操作限制不当
		CWE-276	默认权限配置错误
		CWE-532	敏感信息写入日志文件
	配置文件泄露	CWE-214	存储/传输前敏感信息清除不当
		CWE-497	敏感系统信息泄露至未授权控制域
		CWE-922	敏感信息存储不安全
	命令注入	CWE-200	敏感信息泄露给未授权主体
		CWE-270	权限上下文切换错误
		CWE-78	操作系统命令中特殊元素的中和处理不当
代码注入	命令注入	CWE-94	代码生成控制不当
		CWE-601	URL 重定向至不可信站点
	路径遍历	CWE-22	路径名限制不当，未限定在指定目录
		CWE-36	绝对路径遍历
		CWE-74	输出内容中特殊元素的中和处理不当

A.1 KubeGuard 权限提升检测规则汇总

为便于正文聚焦方法主线，本节补充给出 KubeGuard 在四类权限提升场景下采用的代表性检测规则汇总。

表 A-2: 多种权限提升类型及其检测规则
Tab. A-2: Types of privilege escalation and their detection rules

类型	机制	检测规则
凭证窃取	密钥访问	检测请求的源 IP 地址或服务账户与资源所属 Pod 不一致的 API 请求
	Pod 执行	
	容器逃逸	
账户冒用	服务账户模拟	检测服务账户、操作和资源的组合是否属于该账户的正常使用行为集合
RBAC 操纵	角色修改	检测对角色或角色绑定资源的创建、修补或更新操作
	角色绑定修改	
间接执行	工作负载修改	检测对工作负载资源的创建、修补或更新操作，以及对 Pod 的 exec 操作
	Pod 执行	

附录 B CVE202224768 复现步骤与策略清单

本附录给出案例一的复现详细步骤与纵深防御策略代码清单，用于复核与工程复现。为降低对生产环境的风险，建议仅在隔离测试集群中执行。

B.1 漏洞复现详细步骤

步骤 1：部署漏洞版本 Argo CD (v2.3.1)

```
kubectl create namespace argocd
kubectl apply -n argocd -f
↪ https://raw.githubusercontent.com/argoproj/argo-cd/v2.3.1/manifests/install.yaml
kubectl rollout status deployment/argocd-server -n argocd
```

步骤 2：获取初始管理员密码并登录 UI/CLI

```
kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}" | base64
↪ -d && echo
kubectl port-forward svc/argocd-server -n argocd 8080:443
```

步骤 3：创建宽松的 AppProject（用于演示风险）

```
cat <<EOF | kubectl apply -f -
apiVersion: argoproj.io/v1alpha1
kind: AppProject
metadata:
  name: demo-project
  namespace: argocd
spec:
  destinations:
  - namespace: '*'
    server: '*'
  sourceRepos:
  - '*'
  clusterResourceWhitelist:
  - group: '*'
    kind: '*'
EOF
```

步骤 4：创建低权用户 bob，并授予其对目标 Application 的 sync 与 override 权限在 argocd-rbac-cm 中添加如下策略（注意缩进）：

```
policy.default: role:readonly
policy.csv: |
p, role:syncer, applications, sync, demo-project/*, allow
p, role:syncer, applications, override, demo-project/*, allow
g, bob, role:syncer
```

并在 argocd-cm 中开启账户：

```
accounts.bob: apiKey, login
```

随后重启 Argo CD Server 使配置生效：

```
kubectl rollout restart deployment/argocd-server -n argocd
```

步骤 5：创建 Application 指向恶意仓库

```
cat <<EOF | kubectl apply -f -
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: bob-app
  namespace: argocd
spec:
  project: demo-project
  source:
    repoURL: https://github.com/Ivanbeethoven/bad_repo
    path: .
    targetRevision: HEAD
  destination:
    server: https://kubernetes.default.svc
    namespace: default
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
EOF
```

步骤 6：模拟 bob 触发同步（或依赖自动同步）

```
argocd login localhost:8080 --username bob --password <token-or-password>
argocd app sync bob-app --force
```

步骤 7：验证是否权限提升成功

```
kubectl auth can-i '*' '*' --as bob
```

若输出为 yes，说明 bob 获得了集群范围的高权限（该结果用于说明风险，生产环境严禁在真实集群中进行）。

B.2 策略代码清单

B.2.1 Kubernetes RBAC：收敛 AppProject 的写权限

将 appprojects.argoproj.io 的创建/更新/删除限制在平台管理员组，租户仅可只读（示例中组名可按实际替换）：

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: argocd-appproject-edit
rules:
- apiGroups: ["argoproj.io"]
  resources: ["appprojects"]
  verbs: ["get", "list", "watch", "create", "update", "patch", "delete"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: argocd-appproject-readonly
rules:
- apiGroups: ["argoproj.io"]
  resources: ["appprojects"]
  verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: argocd-appproject-edit-platform-admins
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: argocd-appproject-edit
subjects:
- kind: Group
  name: argo:platform-admin
  apiGroup: rbac.authorization.k8s.io
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: argocd-appproject-readonly-tenants
roleRef:
  apiGroup: rbac.authorization.k8s.io
```

```

kind: ClusterRole
name: argocd-appproject-readonly
subjects:
- kind: Group
  name: argo:tenant-readonly
  apiGroup: rbac.authorization.k8s.io

```

B.2.2 Argo CD RBAC：默认只读 + 高风险能力隔离

建议将 `argocd-rbac-cm` 纳入 Git 基线管理，并在修改后滚动重启 `argocd`

-server:

```

apiVersion: v1
kind: ConfigMap
metadata:
  name: argocd-rbac-cm
  namespace: argocd
data:
  policy.csv: |
    g, argo:platform-admin, role:admin
    g, argo:tenant-readonly, role:readonly

    p, role:admin, projects, *, *, allow
    p, role:admin, repositories, *, *, allow
    p, role:admin, clusters, *, *, allow
    p, role:admin, applications, *, *, allow
    p, role:admin, exec, *, *, allow
    p, role:admin, logs, *, *, allow

    p, role:readonly, applications, get, *, allow
    p, role:readonly, repositories, get, *, allow
    p, role:readonly, projects, get, *, allow
    p, role:readonly, clusters, get, *, allow
    p, role:readonly, logs, get, *, allow
    p, role:readonly, exec, *, *, deny
  policy.default: role:readonly

```

B.2.3 准入控制：Gatekeeper（推荐）

目标：在 AppProject 创建/更新时，对 `spec.roles[].policies[]` 进行一致性校验。本策略覆盖三类约束：

- 禁止出现 `*, *, allow` 的全局通配符放权；
- 要求 policy subject 绑定到项目域（`proj:<project>:<role>` 前缀）；
- 禁止在 project 内授予 `projects/repositories/clusters/exec` 等高危资源写权限。

（1）ConstraintTemplate

```

apiVersion: templates.gatekeeper.sh/v1beta1
kind: ConstraintTemplate
metadata:
  name: k8sargocdappprojectpolicies
spec:
  crd:
    spec:
      names:
        kind: K8sArgoCDAppProjectPolicies
      validation:
        openAPIV3Schema:
          properties:
            enforceProjPrefix:
              type: boolean
  targets:
    - target: admission.k8s.gatekeeper.sh
      rego: |
        package k8sargocdappprojectpolicies

        trim_space(s) = out {
          out := regex.replace("^\\s+|\\s+$", "", s)
        }

        # 1) 禁止出现 "... , *, *, allow" 的全局通配符放权

```

```
deny[msg] {
  input.review.kind.group == "argoproj.io"
  input.review.kind.kind == "AppProject"
  roles := input.review.object.spec.roles
  some i
  policies := roles[i].policies
  some j
  policy := policies[j]
  regex.match(".*\\s*\\s*\\s*\\s*\\s*allow\\s*$", policy)
  msg := sprintf("AppProject policy contains global wildcard allow: %s", [policy])
}

# 2) 要求 subject 采用 proj:<project>:<role> 前缀 (可选开关)
deny[msg] {
  input.review.kind.group == "argoproj.io"
  input.review.kind.kind == "AppProject"
  input.parameters.enforceProjPrefix == true
  ap := input.review.object.metadata.name

  roles := input.review.object.spec.roles
  some i
  policies := roles[i].policies
  some j
  policy := policies[j]

  startswith(policy, "p,")
  parts := split(policy, ",")
  count(parts) >= 2
  subject := trim_space(parts[1])
  not startswith(subject, sprintf("proj:%s:", [ap]))
  msg := sprintf("AppProject policy subject must start with 'proj:%s:' %s", [ap, policy])
}

# 3) 禁止授予 projects/repositories/clusters/exec 的写权限
deny[msg] {
  input.review.kind.group == "argoproj.io"
  input.review.kind.kind == "AppProject"
  roles := input.review.object.spec.roles
  some i
  policies := roles[i].policies
  some j
  policy := policies[j]
  regex.match("^p,.*(projects|repositories|clusters|exec),\\s*(create|update|patch|delete|\\|
↵ \\s*),.*(allow)\\s*$", policy)
  msg := sprintf("AppProject policy grants write on global resource: %s", [policy])
}
```

(2) Constraint

```
apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sArgoCDAppProjectPolicies
metadata:
  name: restrict-argocd-appproject-policies
spec:
  match:
    kinds:
      - apiGroups: ["argoproj.io"]
        kinds: ["AppProject"]
  parameters:
    enforceProjPrefix: true
```

B.2.4 准入控制: Kyverno (替代方案)

若集群已使用 Kyverno, 可用如下 ClusterPolicy 实现同类校验。建议先将 validationFailureAction 设为 Audit, 稳定后再切换为 Enforce。

```
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: restrict-argocd-appproject-policies
spec:
  validationFailureAction: Audit
  background: true
  rules:
    - name: forbid-global-wildcard-allow
      match:
        any:
          - resources:
              kinds:
                - argoproj.io/v1alpha1/AppProject
      validate:
        message: "forbid global wildcard allow in AppProject role policies"
        foreach:
          - list: "request.object.spec.roles[].policies[]"
            deny:
              conditions:
                any:
```

```

- key: "{{ regex_match('.*\\s*\\s*\\s*\\s*allow\\s*$', element) }}"
  operator: Equals
  value: true

- name: enforce-proj-prefix
  match:
    any:
      - resources:
          kinds:
            - argoproj.io/v1alpha1/AppProject
  validate:
    message: "policy subject must use proj:<project>:<role> prefix"
    foreach:
      - list: "request.object.spec.roles[].policies[]"
        deny:
          conditions:
            all:
              - key: "{{ regex_match('^p,\\s*proj:[^:]+:[^,]+$', element) }}"
                operator: Equals
                value: false

- name: forbid-global-resource-write
  match:
    any:
      - resources:
          kinds:
            - argoproj.io/v1alpha1/AppProject
  validate:
    message: "forbid write on projects/repositories/clusters/exec in AppProject policies"
    foreach:
      - list: "request.object.spec.roles[].policies[]"
        deny:
          conditions:
            any:
              - key: "{{ regex_match('^p,.*(projects|repositories|clusters|exec),\\s*(create|update|patch|delete|\\s*),.*(allow)\\s*$', element) }}"
                operator: Equals
                value: true

```


附录 C CVE-2025-2241 复现步骤与策略清单

本附录给出案例二（CVE-2025-2241）的复现要点与缓解策略清单，用于复核与工程落地。该漏洞属于**敏感信息泄露**：vCenter 凭据出现在 Hive 的 ClusterProvision 对象可读字段中，导致“无 Secret 权限也可读到凭据”的旁路风险。请仅在隔离测试环境中操作。

C.1 漏洞复现详细步骤

前提条件

- 已部署 ACM/MCE 与 Hive，并启用 vSphere 集群供应能力（具体版本与安装方式依赖环境，以厂商公告与实验环境为准）；
- 至少存在一个“只读身份”。该身份**不具备** secrets 读取权限，但具备 clusterprovisions.hive.openshift.io 的 get/list/watch 权限（该条件用于验证旁路泄露）。

步骤 1：触发一次 vSphere 集群供应通过 ACM 控制台或 YAML 创建 Hive 的 ClusterDeployment（示例仅展示关键字段，真实环境中密码应引用 Secret；本漏洞的问题在于后续对象中出现明文暴露）：

```
apiVersion: hive.openshift.io/v1
kind: ClusterDeployment
metadata:
  name: vsphere-cluster
  namespace: your-namespace
spec:
  baseDomain: example.com
  platform:
    vsphere:
      vCenter: vc.example.com
      username: administrator@vsphere.local
      password: your-password
  provisioning:
    installConfigSecretRef:
      name: your-install-config
    imageSetRef:
      name: img4.14.0
```

步骤 2：用“可读 ClusterProvision 但不可读 Secret”的身份读取对象先确认权限面，再读取 ClusterProvision：

```
# 谁能读 clusterprovisions (OpenShift)
oc adm policy who-can get clusterprovision
oc adm policy who-can list clusterprovision

# 低权限用户自检 (Kubernetes 通用)
kubectl auth can-i get clusterprovision --as=<user> --all-namespaces
kubectl auth can-i get secrets --as=<user> --all-namespaces

# 导出对象 (名称可能带随机后缀，按实际替换)
kubectl get clusterprovision -n your-namespace
kubectl get clusterprovision <name> -n your-namespace -o yaml
```

步骤 3：定位敏感字段在输出中查找类似结构（位置因版本实现而异，示例以 status.vCenter.credentials 为代表）：

```
status:
  vCenter:
    credentials:
      username: ...
      password: ...
```

若凭据以明文形式出现，即满足漏洞复现判据。

C.2 策略代码清单

本节策略以“收敛读取面 + 缩短暴露窗口 + 审计发现与轮换止血”为主线。由于敏感信息出现在控制器写入的 `status` 字段中，准入控制更适合作为审计/检测手段，不建议强制拦截控制器更新（否则可能影响供应流程）。

C.2.1 1) 立刻收敛 ClusterProvision 的读取权限 (RBAC)

风险盘点命令 (OCP)

```
oc adm policy who-can get clusterprovision
oc adm policy who-can list clusterprovision
oc adm policy who-can watch clusterprovision
```

最小可读权限示例（仅绑定 Hive 控制器服务账号）说明：Hive 控制器的具体 `ServiceAccount/namespace` 需按实际部署确认。

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: hive-clusterprovision-read-min
rules:
- apiGroups: ["hive.openshift.io"]
  resources: ["clusterprovisions"]
  verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: hive-clusterprovision-read-min-binding
subjects:
- kind: ServiceAccount
  name: hive-controllers
  namespace: hive
roleRef:
  kind: ClusterRole
  name: hive-clusterprovision-read-min
  apiGroup: rbac.authorization.k8s.io
```

C.2.2 2) 对读取行为启用审计并联动告警

Kubernetes 审计策略片段（加载方式依平台而定）：

```
apiVersion: audit.k8s.io/v1
kind: Policy
rules:
- level: Metadata
  verbs: ["get", "list", "watch"]
  resources:
  - group: "hive.openshift.io"
    resources: ["clusterprovisions"]
  omitStages: ["RequestReceived"]
```

建议在 SIEM/日志平台中将 Hive 控制器主体加入白名单，仅对非白名单主体的读取行为告警。

C.2.3 3) 缩短暴露窗口：供应完成后清理 ClusterProvision

手动清理

```
oc get clusterprovision -n <ns>
oc delete clusterprovision -n <ns> <name>
```

自动化清理（示例 CronJob）说明：删除条件建议通过标签或状态字段精确匹配，避免误删；示例以标签 `hive.openshift.io/retain=false` 为条件。

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: cleanup-sa
  namespace: hive
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: cleanup-role
  namespace: hive
rules:
- apiGroups: ["hive.openshift.io"]
  resources: ["clusterprovisions"]
  verbs: ["list", "get", "delete"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: cleanup-rb
  namespace: hive
subjects:
- kind: ServiceAccount
  name: cleanup-sa
roleRef:
  kind: Role
  name: cleanup-role
  apiGroup: rbac.authorization.k8s.io
---
apiVersion: batch/v1
kind: CronJob
metadata:
  name: cleanup-clusterprovision
  namespace: hive
spec:
  schedule: "0 * * * *"
  jobTemplate:
    spec:
      template:
        spec:
          serviceAccountName: cleanup-sa
          containers:
            - name: cleaner
              image: registry.access.redhat.com/ubi9/ubi:latest
              command: ["/bin/sh", "-c"]
              args:
                - |
                  set -eu
                  oc delete clusterprovision -n hive -l hive.openshift.io/retain=false
                  ↪ --ignore-not-found
          restartPolicy: OnFailure
```

C.2.4 4) vCenter 凭据轮换与强化（止血）

建议在确认泄露后立即轮换 vCenter 凭据，并启用最小权限、MFA、来源 IP 允许列表等约束。轮换完成后更新 Hive 使用的 Secret（示例命令需按实际 Secret 名称/namespace 调整）：

```
oc -n hive create secret generic vcenter-credentials \
  --from-literal=username=<new-user> \
  --from-literal=password=<new-pass> \
  --dry-run=client -o yaml | oc apply -f -
```

C.2.5 5) “敏感字段守护”检测（Gatekeeper/Kyverno, 建议审计模式）

Gatekeeper (dryrun)：检测 ClusterProvision 中是否出现敏感键名 (password/token/secret/credential)。

```
apiVersion: templates.gatekeeper.sh/v1beta1
kind: ConstraintTemplate
metadata:
  name: k8sclustprovkeysensitive
spec:
  crd:
    spec:
      names:
        kind: K8sClustProvKeySensitive
      targets:
        - target: admission.k8s.gatekeeper.sh
          rego: |
            package k8sclustprovkeysensitive

            sensitive_key(k) {
              lk := lower(k)
              contains(lk, "password")
            }
            sensitive_key(k) {
              lk := lower(k)
              contains(lk, "token")
            }
            sensitive_key(k) {
              lk := lower(k)
              contains(lk, "secret")
            }
            sensitive_key(k) {
              lk := lower(k)
              contains(lk, "credential")
            }

            # 深度遍历 object, 收集所有键
            walk(obj, path, val) {
              is_object(obj)
              some k
              v := obj[k]
              walk(v, array.concat(path, [k]), val)
            }
            walk(obj, path, val) {
              is_array(obj)
              some i
              v := obj[i]
              walk(v, array.concat(path, [sprintf("%v", [i])]), val)
            }
            walk(obj, path, obj) {
              not is_object(obj)
              not is_array(obj)
            }

            deny[msg] {
              input.review.kind.group == "hive.openshift.io"
              input.review.kind.kind == "ClusterProvision"
              walk(input.review.object, [], _)
              some k
              _ = input.review.object[k]
              sensitive_key(k)
              msg := sprintf("ClusterProvision contains sensitive key at top-level: %s", [k])
            }

            deny[msg] {
              input.review.kind.group == "hive.openshift.io"
              input.review.kind.kind == "ClusterProvision"
              walk(input.review.object, path, _)
              some i
              k := path[i]
              sensitive_key(k)
              msg := sprintf("ClusterProvision contains sensitive key on path %v", [path])
            }
            ---
apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sClustProvKeySensitive
metadata:
  name: audit-clusterprovision-sensitive-keys
spec:
  enforcementAction: dryrun
  match:
    kinds:
      - apiGroups: ["hive.openshift.io"]
        kinds: ["ClusterProvision"]
```

Kyverno (Audit)：对对象文本进行检测，以提示可能存在的敏感键名。

该策略更适用于告警与排查，不建议直接用于阻断控制器更新。

```
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: audit-clusterprovision-sensitive-keys
spec:
  validationFailureAction: Audit
  background: true
  rules:
    - name: detect-secret-like-keys
      match:
        any:
          - resources:
              kinds:
                - hive.openshift.io/v1/ClusterProvision
      validate:
        message: ClusterProvision 可能包含敏感字段 password token secret credential。
        deny:
          conditions:
            any:
              - key: "{{ to_string(request.object) }}"
                operator: AnyIn
                value:
                  - password
                  - token
                  - secret
                  - credential
```


附录 D CVE-2023-30512 复现步骤与策略清单

本附录给出案例三（CVE-2023-30512, CubeFS CSI 插件过度授权导致的集群权限提升风险）的**验证性复现与策略清单**。为避免提供可被滥用的细粒度利用操作，复现部分以“权限证据 + 受控权限验证”为主，重点证明 CSI ServiceAccount 被绑定了不必要的 secrets 读取能力；策略部分给出可落地的 RBAC 收敛、准入防回归与审计检测。

D.1 漏洞复现详细步骤

变量约定（按实际替换）

```

NAMESPACE=<cubeFS-namespace>
SA=cfs-csi-service-account
CLUSTERROLE=cfs-csi-cluster-role

# 如 SA 名称不同，请先定位：
kubectl get sa -A | findstr /i "csi cfs cubeFS"
kubectl get clusterrole,clusterrolebinding -A | findstr /i "csi cfs cubeFS"

```

步骤 1：导出 RBAC 证据（确认 secrets 权限存在）

```

kubectl get clusterrole $CLUSTERROLE -o yaml
kubectl get clusterrolebinding -o yaml | findstr /i "cfs-csi cubeFS"

# 重点检查 ClusterRole 规则中是否出现：
# resources: ["secrets"]
# verbs: ["get","list","watch"]

```

步骤 2：受控权限验证（不执行实际数据提取）

```

# 直接验证该 SA 是否具备跨命名空间读取 secrets 的能力
kubectl auth can-i get secrets --as=system:serviceaccount:$NAMESPACE:$SA --all-namespaces
kubectl auth can-i list secrets --as=system:serviceaccount:$NAMESPACE:$SA --all-namespaces
kubectl auth can-i watch secrets --as=system:serviceaccount:$NAMESPACE:$SA --all-namespaces

```

权限提升路径（文字图，便于复盘）

```

CubeFS CSI 安装环境
|
+--> DaemonSet: cfs-csi-node （节点插件）
|
+--> Deployment: cfs-csi-controller （控制器）
      |
      v
ServiceAccount: cfs-csi-service-account
      |
      v
ClusterRole: cfs-csi-cluster-role
              （误包含 secrets 的 get/list/watch）
              |
              v
一旦攻击者控制节点或 CSI Pod => 可滥用该 SA 的身份边界
（风险：读取高价值凭证 -> 演化为集群级权限提升）

```

D.2 策略代码清单

本节策略遵循“最小权限 + 防回归 + 可观测”主线：先移除 CSI 不必要的 secrets 读取权限；再用准入策略阻断未来再次引入过度授权；最后启用审计检测以便快速发现异常访问并触发止血流程。

D.2.1 1) RBAC 最小化：移除 ClusterRole 中的 secrets 权限

说明：CSI 需要的资源/动词与具体实现有关，建议在预生产以 PVC/PV 创建、挂载、扩容、attach/detach 等流程回归验证后再推广到生产。以下示例展示“覆盖式”更新 ClusterRole 的方式，明确不包含 `resources: ["secrets"]`。

```
cat <<'EOF' | kubectl apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: cfs-csi-cluster-role
  labels:
    app.kubernetes.io/name: cfs-csi
rules:
- apiGroups: [""]
  resources: ["nodes","persistentvolumes","pods"]
  verbs: ["get","list","watch"]
- apiGroups: ["storage.k8s.io"]
  resources: ["volumeattachments","csinodes"]
  verbs: ["get","list","watch"]
# 注意：不应包含 resources:["secrets"]
EOF

# 复测（期望输出 no）
kubectl auth can-i list secrets --as=system:serviceaccount:$NAMESPACE:$SA --all-namespaces
```

D.2.2 2) ServiceAccount 硬化（按需评估）

若 CSI 驱动不需要直接调用 Kubernetes API，可评估禁用自动挂载令牌以降低“被攻陷即可滥用身份”的风险：

```
# 将 automountServiceAccountToken 设为 false（需评估 CSI 功能是否依赖 token）
kubectl -n $NAMESPACE patch serviceaccount $SA --type='merge' -p
↪ '{"automountServiceAccountToken":false}'
```

D.2.3 3) 准入防回归（OPA Gatekeeper）：禁止创建包含 secrets 读取动词的 ClusterRole

说明：该策略用于防止未来误配置再次引入过度授权。建议先以 dryrun 运行观察影响，再逐步强制。

```
cat <<'EOF' | kubectl apply -f -
apiVersion: templates.gatekeeper.sh/v1beta1
kind: ConstraintTemplate
metadata:
  name: k8sdenysecretsrbac
spec:
  crd:
    spec:
      names:
        kind: K8sDenySecretsRBAC
  targets:
  - target: admission.k8s.gatekeeper.sh
    rego: |
      package k8sdenysecretsrbac

      has_secrets_rule(rule) {
        rule.resources[_] == "secrets"
        rule.verbs[_] == "get"
      }
      has_secrets_rule(rule) {
        rule.resources[_] == "secrets"
        rule.verbs[_] == "list"
      }
      has_secrets_rule(rule) {
        rule.resources[_] == "secrets"
        rule.verbs[_] == "watch"
      }

      violation[{"msg": msg}] {
        input.review.kind.kind == "ClusterRole"
```

```

        some i
        rule := input.review.object.rules[i]
        has_secrets_rule(rule)
        msg := "ClusterRole must not grant get/list/watch on secrets"
    }
EOF

cat <<'EOF' | kubectl apply -f -
apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sDenySecretsRBAC
metadata:
  name: deny-clusterrole-secrets-read
spec:
  enforcementAction: dryrun
  match:
    kinds:
      - apiGroups: ["rbac.authorization.k8s.io"]
        kinds: ["ClusterRole"]
EOF

```

D.2.4 4) 审计与检测：对 secrets 枚举行为进行可观测化

Kubernetes 审计策略片段（加载方式依平台而定）：

```

apiVersion: audit.k8s.io/v1
kind: Policy
rules:
- level: Metadata
  verbs: ["get","list","watch"]
  resources:
    - group: ""
      resources: ["secrets"]
  omitStages: ["RequestReceived"]

```

建议在 SIEM/日志平台中对来自 CSI ServiceAccount 的 secrets 枚举进行聚合告警，并结合阈值与白名单降低噪音。

D.2.5 5) 网络收敛（可选）：限制 CSI Pod 出站访问面

说明：NetworkPolicy 能降低被攻陷工作负载的横向移动能力，但在不同 CNI 下支持情况不同；同时 API Server 可能位于外部 LB，需按实际环境调整。

```

cat <<'EOF' | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: cfs-csi-egress-lockdown
  namespace: $NAMESPACE
spec:
  podSelector:
    matchLabels:
      app: cfs-csi
  policyTypes: ["Egress"]
  egress:
    - to:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: kube-system
          podSelector:
            matchLabels:
              k8s-app: kube-dns
      ports:
        - protocol: UDP
          port: 53
EOF

```


附录 E CVE-2020-4062 复现步骤与策略清单

本附录给出案例四（CVE-2020-4062，Conjur OSS Helm Chart 旧版本缺失网络隔离导致 Postgres 在集群内可达）的复现要点与缓解策略清单。考虑到该问题涉及数据库直连带来的敏感数据风险，复现环节仅进行网络暴露面验证，用于确认 5432 端口在集群内的可达性以及网络策略的生效性，不展开数据库操作细节。

E.1 漏洞复现详细步骤

前提条件

- 集群 CNI 支持 NetworkPolicy，例如 Calico。
- Helm v3 可用。
- 受影响的 Conjur OSS Helm Chart 版本为 < 2.0.0。

步骤 1：在隔离命名空间部署旧版 Chart，并进行必要的兼容性适配旧版 Chart 可能使用过时 API，例如 apps/v1beta2。在较新 Kubernetes 版本上复现时，可在保持网络策略缺失这一特性不变的前提下，将 Deployment API 适配为 apps/v1 并补齐 selector 字段以完成安装。

```
NAMESPACE=conjur-vuln
helm repo add cyberark https://cyberark.github.io/helm-charts
helm repo update
helm search repo cyberark/conjur-oss --versions

helm install conjur-oss cyberark/conjur-oss \
  --namespace $NAMESPACE \
  --create-namespace \
  --version <2.0.0>

kubectl -n $NAMESPACE get pods
```

步骤 2：确认 Postgres Service 与 5432 端口暴露

```
NAMESPACE=conjur-vuln
kubectl get ns $NAMESPACE
kubectl -n $NAMESPACE get svc

# 重点确认 Postgres Service（名称以实际为准）与 5432 端口存在
kubectl -n $NAMESPACE get svc | findstr /i "postgres 5432 conjur"
```

步骤 3：确认未配置 NetworkPolicy

```
kubectl -n $NAMESPACE get networkpolicy
```

若不存在任何约束 Postgres 入站的策略，且集群默认允许 Pod 间互通，则可判定数据库对集群内工作负载开放。

步骤 4：连通性验证在不执行数据库操作的前提下，使用 TCP 连通性探测验证非 Conjur 组件工作负载是否能够与 Postgres 的 5432 端口建立连接，以此作为暴露面存在的验证依据。

```
POSTGRES_SVC=<postgres-service-name>

kubectl -n $NAMESPACE run netcheck \
  --image=busybox:1.36 \
  --restart=Never \
  --rm -i -- sh -c "nc -vz $POSTGRES_SVC 5432"
```

E.2 策略代码清单

E.2.1 1) 缓解措施：NetworkPolicy，默认拒绝并仅放通 Conjur Server 访问 Postgres 5432

下述策略在命名空间内实现两点：其一，默认拒绝 Postgres Pod 的入站流量；其二，仅允许 Conjur Server 访问 Postgres 的 5432 端口。其中标签选择器需与实际部署对象一致，示例使用 app=conjur-oss 与 app=conjur-oss-postgres，可按实际标签调整。

```
cat <<'EOF' | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: conjur-postgres-default-deny
  namespace: conjur-vuln
spec:
  podSelector:
    matchLabels:
      app: conjur-oss-postgres
  policyTypes:
  - Ingress
  ingress: []
EOF

cat <<'EOF' | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: conjur-postgres-allow-conjur
  namespace: conjur-vuln
spec:
  podSelector:
    matchLabels:
      app: conjur-oss-postgres
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: conjur-oss
      ports:
      - protocol: TCP
        port: 5432
EOF
```

策略验证

```
kubectl -n conjur-vuln get networkpolicy

# 复测：从非 Conjur 组件工作负载到 Postgres 5432 的连通性应失败
# 从 Conjur Server 到 Postgres 5432 的连通性应成功
```

E.2.2 2) 命名空间隔离与 RBAC 收敛

- 将 Conjur 部署在独立命名空间，并通过 NetworkPolicy 限制跨命名空间访问。

- 收敛对数据库凭据相关 **Secret** 的读取权限，降低网络可达与凭据可读叠加带来的风险。

附录 F CVE-2022-24877 复现步骤与策略清单

本附录给出案例五（CVE-2022-24877，Flux kustomize-controller 通过恶意 kustomization.yaml 触发路径穿越，导致控制器 Pod 文件系统敏感数据暴露，并在多租户部署中可能引发权限提升）的**验证性复现与缓解策略清单**。为避免提供可直接滥用的利用细节，复现部分以“路径越界行为证据与判据”为主，不提供可复用的攻击脚本。

F.1 漏洞复现详细步骤

复现目标验证控制器在构建过程中是否会因配置文件中的异常路径引用而产生目录边界越界读取风险。重点关注构建阶段的路径规范化与边界约束，并以日志行为与构建产物的异常特征作为证据判据。本案例涉及的控制器为 kustomize-controller，相关配置文件为 kustomization.yaml。

环境与前提

建议在隔离测试集群中进行，并使用受影响版本的 kustomize-controller。该类问题在多租户集群中可能放大隔离破坏风险，故不建议在生产环境进行实验。修复信息：该漏洞已在 kustomize-controller v0.24.0 修复。该版本随 flux v0.29.0 集成发布。

步骤要点

1. 在隔离仓库中准备最小化示例，包含 kustomization.yaml 及少量合法资源清单，用于建立可对比的基线构建产物。
2. 在受控条件下引入异常路径引用（例如包含父目录引用或绝对路径风格的条目），触发控制器重新拉取并执行构建。
3. 观察控制器日志与事件输出，记录与路径解析、文件读取失败或异常引用相关的报错/告警。
4. 对比构建产物与基线差异，检查是否出现“非预期文件内容进入产物”的迹象。该步骤应仅使用无敏感性的测试素材进行验证。

参考命令（用于采集受控证据）

```
FLUX_NS=flux-system
APP_NS=<kustomization-namespace>
KS_NAME=<kustomization-name>

kubectl -n $FLUX_NS get deploy kustomize-controller -o wide
kubectl -n $APP_NS get kustomization $KS_NAME -o yaml

# 在隔离仓库中准备异常路径测试样例后，触发一次受控重调和
```

```
flux reconcile kustomization $KS_NAME -n $APP_NS --with-source
```

采集控制器日志与事件作为证据

```
kubect1 -n $FLUX_NS logs deploy/kustomize-controller --tail=200
```

```
kubect1 -n $APP_NS get events --sort-by=.lastTimestamp | findstr /i "Kustomization Reconciliation"
```

证据判据与逻辑解释 该类漏洞的关键证据在于“路径解析是否受目录边界约束”。若控制器在构建阶段对异常路径引用给出明确的拒绝与报错，并且构建产物未出现越界读取痕迹，可视为风险被抑制。若在严格受控的测试条件下观察到异常路径引用导致非预期内容进入构建产物，则可判定目录边界约束存在缺口，需立即升级组件并补齐流程治理措施。

F.2 策略代码清单（优先级方案）

本节选取对该类风险**最直接且工程可落地**的缓解措施：CI/CD 前置校验，并辅以多租户下的变更治理与运行时硬化。

F.2.1 1) 工作区缓解：在 CI/CD 中校验 `kustomization.yaml` 路径策略

在补丁窗口期或多团队协作场景下，可在 CI/CD 流水线中加入配置校验门禁。校验对象为 `kustomization.yaml`，要求其资源路径满足安全策略，例如：禁止绝对路径风格、禁止父目录引用、限制引用的文件类型与目录范围。下述示例为 `conftest` (OPA) 策略骨架，用于展示如何对 `kustomization.yaml` 的路径字段进行约束；可按组织规范扩展字段列表与白名单策略。

```
package kustomizepolicy
deny[msg] {
  some p
    p := input.resources[_]
    risky_path(p)
    msg := sprintf("kustomization.yaml: resources contains risky path: %s", [p])
}
deny[msg] {
  some i
    p := input.patches[i].path
    risky_path(p)
    msg := sprintf("kustomization.yaml: patches[%d].path contains risky path: %s", [i, p])
}
risky_path(p) {
  startswith(p, "/")
}
risky_path(p) {
  contains(p, "..")
}
```

说明：为降低误报，可进一步将 `contains(p, "..")` 改为按路径分段匹配“父目录段”，并对允许的相对路径前缀建立白名单。

F.2.2 2) 多租户变更治理与运行时硬化（纵深防御）

在多租户集群中，应严格控制谁可以向 Flux 同步源提交变更、谁可以修改 Flux 相关资源，并通过代码评审与分支保护降低恶意配置进入同步面的概率。同时可对控制器工作负载启用非特权运行、只读根文件系统、默认 seccomp 配置与能力集收敛，以降低异常路径下的后续风险。

附录 G CVE-2022-29171 复现步骤与策略清单

本附录给出案例六（CVE-2022-29171，Sourcegraph 在 Gitolite 与 Phabricator 联动集成中存在配置驱动的命令执行风险）的验证性复现与缓解策略清单。为避免提供可直接滥用的利用细节，复现部分以执行路径证据与回归判据为主，不给出可直接用于远程执行的命令载荷与请求模板。

G.1 漏洞复现详细步骤

复现目标验证在受影响版本中，特定集成配置项（callsignCommand）是否会被 gitserver 在处理仓库元数据获取流程时执行。复现以日志、审计与可观测证据为主，并在禁用集成后复测，确认风险被消除。

环境与前提仅在隔离测试环境开展。前置条件包括：（i）Sourcegraph 版本处于受影响范围；（ii）启用了 Gitolite code host 并与 Phabricator 元数据获取路径相关联；（iii）存在可编辑/新增 Gitolite code host 的管理权限；（iv）存在对内置 Grafana 的高权限访问，用于说明观测面可能降低内部寻址成本。

步骤要点

1. 部署受影响版本 Sourcegraph，并确认 gitserver 作为独立工作负载运行（记录其命名空间与标签信息）。
2. 在管理面将 Gitolite 与 Phabricator 集成配置为可用状态，保证仓库元数据获取流程可被触发。
3. 在严格受控条件下对 callsignCommand 进行无害探针式变更（例如仅返回固定标识、且不访问外部网络、不读取敏感文件）。
4. 触发一次与该集成相关的元数据获取/同步流程，并观察 gitserver 日志、任务记录或审计日志中是否出现外部命令执行的迹象。
5. 禁用 Gitolite code host 后重复触发流程，确认相同迹象不再出现。

参考命令（用于日志与工作负载证据采集）

```

NAMESPACE=sourcegraph
kubectl -n $NAMESPACE get deploy,pod,svc | findstr /i "gitserver"
kubectl -n $NAMESPACE logs deploy/gitserver --tail=200
kubectl -n $NAMESPACE get events --sort-by=.lastTimestamp | findstr /i "gitserver"

# 在管理面触发一次元数据获取/同步后，重复采集日志并与禁用集成后的结果对比
kubectl -n $NAMESPACE logs deploy/gitserver --since=10m | findstr /i "gitolite phabricator"
↪ callsign
  
```

证据判据与逻辑解释若在受影响版本中，gitserver 的运行日志或可观测数据中出现因集成配置而执行外部命令的明确迹象，可判定存在执行面风

险。禁用相关集成后，该执行路径应被移除或被显式限制；若仍可观察到可疑执行迹象，则需进一步检查是否存在配置残留或旁路路径。

G.2 策略清单

本节仅选取 3 条对该类风险最直接且工程可落地的策略：功能禁用、网络隔离与权限分离/暴露面收敛。

G.2.1 1) 快速缓解：禁用或移除 Gitolite code host（若业务允许）

若业务不依赖 Gitolite，可移除所有 Gitolite code host 配置并保持禁用状态，从输入面直接消除 `callsignCommand` 的风险来源。

G.2.2 2) 网络隔离：限制 gitserver 的可达面（默认拒绝并按需放通）

在 Kubernetes 部署中，建议对 gitserver 应用 NetworkPolicy：仅允许 Sourcegraph 内部必要组件访问其入站端口；出站仅放通 DNS 与必要的代码托管出口。该策略用于降低内部拓扑信息被获取后的横向触达能力，并为权限误授提供纵深缓冲。

策略示例分别约束入站与出站。入站仅允许 Sourcegraph 内部调用方访问 gitserver；出站仅放通 DNS，并对代码托管出口按需单独放通。选择器与端口应按实际部署对象调整。

```

NAMESPACE=sourcegraph

# 1) 入站收敛：仅允许 Sourcegraph 内部组件访问 gitserver
cat <<'EOF' | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: gitserver-ingress-allow-internal
  namespace: sourcegraph
spec:
  podSelector:
    matchLabels:
      app: gitserver
  policyTypes: ["Ingress"]
  ingress:
    - from:
      - podSelector:
          matchLabels:
            app: sourcegraph-frontend
      - podSelector:
          matchLabels:
            app: repo-updater
      ports:
        - protocol: TCP
          port: 3178
EOF

# 2) 出站收敛：仅放通 DNS；代码托管出口按需另行放通
cat <<'EOF' | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy

```

```

metadata:
  name: gitserver-egress-dns-only
  namespace: sourcegraph
spec:
  podSelector:
    matchLabels:
      app: gitserver
  policyTypes: ["Egress"]
  egress:
    - to:
      - namespaceSelector:
          matchLabels:
            kubernetes.io/metadata.name: kube-system
        podSelector:
          matchLabels:
            k8s-app: kube-dns
      ports:
        - protocol: UDP
          port: 53
        - protocol: TCP
          port: 53
EOF

# 验证: 策略已创建, 且 gitserver 的出站/入站符合预期
kubectl -n $NAMESPACE get networkpolicy | findstr /i "gitserver"

```

验证要点建议先在预生产灰度启用, 并结合 gitserver 的错误日志与调用链路指标观察误拦截情况。对确需访问的代码托管出口, 按最小集合逐项放通, 并保留变更记录。

G.2.3 3) 权限分离与暴露面收敛: 限制管理面与观测面的高权限

严格限制可编辑外部服务配置 (尤其是 Gitolite code host 配置) 的主体数量, 并启用变更审批与审计。同时收敛 Grafana 的暴露面, 例如仅内网访问、受控 Ingress、强认证与最小管理员集合, 并将 Grafana 管理员与 Sourcegraph 站点管理员职责分离, 避免观测面权限放大攻击链的信息优势。

落地要点在平台流程上, 可将外部服务配置变更与观测面账号授权纳入同一类高风险变更, 要求双人复核、强制留痕与定期回收。在技术控制上, 可对管理入口启用强认证并限制来源网络, 同时在日志平台中对关键配置项变更进行集中审计与告警。

附录 H CVE-2023-22482 复现步骤与策略清单

本附录给出案例七（CVE-2023-22482，Argo CD 在 OIDC 场景下未校验令牌受众 aud，导致跨受众令牌可能被接受）的**验证性复现与缓解策略清单**。为避免提供可直接复用的越权访问细节，复现部分以配置证据、令牌声明证据与接口返回判据为主，不展开可直接用于获取令牌与构造请求的具体命令。

H.1 漏洞复现详细步骤

复现目标验证在受影响版本中，Argo CD 是否会接受 aud 不匹配但签名与发行者合法的 OIDC 令牌，并据此返回受 RBAC 控制的资源响应。

环境与前提仅在隔离测试环境开展。前置条件包括：（i）部署受影响版本的 Argo CD（例如 2.5.4），并启用 OIDC 登录；（ii）OIDC 提供方能够为至少两个不同受众签发令牌，且其中一个受众并非 Argo CD；（iii）Argo CD 已配置为信任该 OIDC 提供方，并启用 groups 相关声明用于 RBAC 映射；（iv）具备可观察的审计与日志渠道，用于对登录校验与资源访问结果进行证据采集。

步骤要点

1. 固化基线配置：记录 Argo CD 版本与 OIDC 配置要点（issuer、clientId、所请求的 scope），确认 Argo CD 能够正常使用 OIDC 完成会话建立。
2. 获取一枚由同一 OIDC 提供方签发、但 aud 指向其他受众的有效令牌。该令牌应满足验签可通过且包含可用于 RBAC 映射的 groups 声明。
3. 提取令牌声明证据：对该令牌解析其 payload，保留 iss、aud、exp 与 groups 等字段的记录，证明其受众并非 Argo CD。
4. 使用该令牌访问 Argo CD 的受保护接口，选择一个只读且便于判定的 API 作为验证点，记录响应结果与 Argo CD 侧日志。

参考命令（用于配置、令牌与响应证据采集）

```
ARGO_NS=argocd
ARGOCD_URL=https://argocd.example.com
TOKEN=<oidc-token-with-wrong-aud>

kubectl -n $ARGO_NS get cm argocd-cm -o yaml
kubectl -n $ARGO_NS get deploy argocd-server -o wide

# 保留令牌 payload 作为 aud/iss 证据（必要时补齐 base64 padding）
printf '%s' "$TOKEN" | cut -d. -f2 | tr '_-' '/' | base64 -d

# 选择只读接口作为验证点
curl -k -H "Authorization: Bearer $TOKEN" "$ARGOCD_URL/api/v1/applications"
kubectl -n $ARGO_NS logs deploy/argocd-server --since=10m | findstr /i "oidc token aud"
```

证据判据与逻辑解释若在受影响版本中，Argo CD 在验签通过的前提下接受 aud 不匹配的令牌，并返回受 RBAC 控制的资源响应，可判定存在受众校验缺失。

H.2 策略清单

本节仅选取 3 条对该类风险最直接且工程可落地的策略：身份提供方客户端隔离、前置受众校验与 RBAC 收敛。

H.2.1 1) 身份提供方治理：独立客户端与唯一受众

为 Argo CD 配置独立 OIDC 客户端与唯一受众标识，避免与其他业务系统共享同一受众空间。同时对该客户端签发的 groups 声明进行裁剪，确保仅包含与 Argo CD 授权相关的组，并建立清晰的组命名规范，避免跨系统复用带来的权限含义漂移。

Argo CD OIDC 配置示例(唯一 clientID)说明:以下示例展示 argocd-cm 中的关键配置项，重点在于为 Argo CD 使用独立客户端标识，并显式声明基于 groups 的 RBAC 映射来源。

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: argocd-cm
  namespace: argocd
data:
  url: https://argocd.example.com
  oidc.config: |
    name: CorporateOIDC
    issuer: https://idp.example.com/realms/platform
    clientID: argocd-prod
    clientSecret: $oidc.corporate.clientSecret
    requestedScopes: ["openid", "profile", "email", "groups"]
    requestedIDTokenClaims:
      groups:
        essential: true
    logoutURL: https://idp.example.com/realms/platform/protocol/openid-connect/logout
```

身份提供方客户端侧约束示意说明:不同身份提供方的配置格式不同，以下仅示意“为 Argo CD 单独分配客户端与受众、限制组声明来源”的关键字

```
clientId: argocd-prod
audience: argocd-prod
redirectUri:
  - https://argocd.example.com/auth/callback
protocol: openid-connect
defaultClientScopes:
  - openid
  - profile
  - email
  - groups
groupClaimFilter:
  prefix: argo:
  allow:
    - argo:platform-admin
    - argo:project-readonly
    - argo:project-deployer
```

H.2.2 2) 入口补偿控制：前置强制校验 aud

在 Argo CD 前置入口（如 API 网关或反向代理）对令牌进行一致性校验，至少包含发行者、签名与受众校验。落地时应提供灰度与回滚机制，并覆盖 Web 与自动化访问路径。

前置网关示例：Envoy JWT 认证过滤器说明：该示例展示如何在进入 Argo CD 前对 JWT 的 issuer 与 audiences 执行强制校验。若令牌的 aud 不包含 argocd-prod，请求将在到达 Argo CD 之前被拒绝。

```
static_resources:
  listeners:
  - name: argocd_listener
    address:
      socket_address:
        address: 0.0.0.0
        port_value: 8443
    filter_chains:
    - filters:
      - name: envoy.filters.network.http_connection_manager
        typed_config:
          "@type": type.googleapis.com/envoy.extensions.filters.network.http_connection_manager.v3
          .HttpConnectionManager
          stat_prefix: ingress_https
          route_config:
            virtual_hosts:
            - name: argocd
              domains: ["argocd.example.com"]
              routes:
              - match: { prefix: "/" }
                route:
                  cluster: argocd_server
          http_filters:
          - name: envoy.filters.http.jwt_authn
            typed_config:
              "@type":
                type.googleapis.com/envoy.extensions.filters.http.jwt_authn.v3.JwtAuthentication
            providers:
            corporate_oidc:
              issuer: https://idp.example.com/realms/platform
              audiences:
              - argocd-prod
              remote_jwks:
                http_uri:
                  uri: https://idp.example.com/realms/platform/protocol/openid-connect/certs
                  cluster: idp_jwks
                  timeout: 5s
            rules:
            - match: { prefix: "/api/" }
              requires:
                provider_name: corporate_oidc
          - name: envoy.filters.http.router
      - name: envoy.filters.http.router
  clusters:
  - name: argocd_server
    type: STRICT_DNS
    load_assignment:
      cluster_name: argocd_server
    endpoints:
    - lb_endpoints:
      - endpoint:
          address:
            socket_address:
              address: argocd-server.argocd.svc.cluster.local
              port_value: 80
```

H.2.3 3) 权限收敛：RBAC 最小化与项目边界约束

收敛 argocd-rbac-cm 中的角色授权，将高权限组限制在最小集合，并通过 AppProject 限定可部署的命名空间、资源类型与来源仓库。同时将关键接口的访问失败与异常登录事件纳入审计与告警，以便在令牌泄露或误用时快速发现与止血。

Argo CD RBAC 收敛示例

说明：以下示例将高权限组限制为 `argo:platform-admin`。普通项目用户仅保留只读与有限同步能力，从而降低跨受众令牌误用后的权限上限。

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: argocd-rbac-cm
  namespace: argocd
data:
  policy.default: role:readonly
  policy.csv: |
    g, argo:platform-admin, role:admin
    g, argo:project-readonly, role:readonly
    g, argo:project-deployer, role:deployer

    p, role:admin, applications, *, /*, allow
    p, role:admin, projects, *, *, allow
    p, role:admin, repositories, *, *, allow
    p, role:admin, clusters, *, *, allow

    p, role:readonly, applications, get, /*, allow
    p, role:readonly, projects, get, *, allow
    p, role:readonly, repositories, get, *, allow

    p, role:deployer, applications, get, my-project/*, allow
    p, role:deployer, applications, sync, my-project/*, allow
    p, role:deployer, applications, action/*, my-project/*, deny
```

AppProject 边界约束示例说明：通过限制目标命名空间、来源仓库与可管理资源范围，进一步压缩误用令牌在项目维度上的横向影响面。

```
apiVersion: argoproj.io/v1alpha1
kind: AppProject
metadata:
  name: my-project
  namespace: argocd
spec:
  description: bounded project for tenant applications
  sourceRepos:
    - https://github.com/example-org/app-configs.git
  destinations:
    - namespace: tenant-a
      server: https://kubernetes.default.svc
  clusterResourceWhitelist:
    - group: ""
      kind: Namespace
  namespaceResourceWhitelist:
    - group: apps
      kind: Deployment
    - group: ""
      kind: Service
  roles:
    - name: readonly
      description: read-only access for tenant reviewers
      policies:
        - p, proj:my-project:readonly, applications, get, my-project/*, allow
      groups:
        - argo:project-readonly
    - name: deployer
      description: deployment access for approved operators
      policies:
        - p, proj:my-project:deployer, applications, get, my-project/*, allow
        - p, proj:my-project:deployer, applications, sync, my-project/*, allow
      groups:
        - argo:project-deployer
```

致 谢

作者简历及在学研究成果

一、主要教育经历/工作经历（从大学起，到硕士入学止）

起止年月	学习或工作单位	备注
2019 年 9 月至 2023 年 6 月	在北京科技大学计算机科学与技术专业攻读学士学位	

二、在学期间从事的科研工作

三、在学期间所获的科研奖励

四、在学期间发表的论文

[1] 第二作者（导师一作）.ICWS.CCF-B.2025.

独创性声明

本人声明所呈交的论文是我个人在导师指导下进行的研究工作及取得的
研究成果。尽我所知，除了文中特别加以标注和致谢的地方外，论文中不包
含其他人已经发表或撰写过的研究成果，也不包含为获得北京科技大学或其
它教育机构的学位或证书而使用过的材料。与我一同工作的同志对本研究所
做的任何贡献均已在论文中作了明确的说明并表示了谢意。

作者签名：_____ 日期：_____ 年_____ 月_____ 日

备注：提交时须有作者亲笔签名！

关于论文使用授权的说明

本人完全了解北京科技大学有关保留、使用学位论文的规定，即：学校有权保留送交论文的复印件，允许论文被查阅和借阅；学校可以公布论文的全部或部分内容，可以采用影印、缩印或其他复制手段保存论文。

（保密的论文在解密后应遵循此规定）

签名：_____ 导师签名：_____ 日期：_____

学位论文数据集

关键词 *	密级 *	中图分类号 *	UDC	论文资助
学位授予单位名称 *		学位授予单位代 码 *	学位类别 *	学位级别 *
北京科技大学		10008		
论文题名 *		并列题名		论文语种 *
作者姓名 *			学号 *	
培养单位名称 *		培养单位代码 *	培养单位地址	邮编
北京科技大学		10008	北京市海淀区学 院路 30 号	100083
学科专业 *		研究方向 *	学制 *	学位授予年 *
论文提交日期 *				
导师姓名 *			职称 *	
评阅人	答辩委员会主席	答辩委员会成员		
	*			
电子版论文提交格式文本 () 图像 () 视频 () 音频 () 多媒体 () 其他 () 推荐格式: application/msword; application/pdf				
电子论文出版 (发布) 者		电子论文出版 (发布) 地		权限声明
论文总页数 *				
共 33 项, 其中带 * 为必填数据, 为 22 项。				